Constructions

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Calculus **3**
- 1.1 Assumptions . . . 3
- 1.2 Differentiation . . . 5
- 1.3 Integration . . . 8
- 1.4 Ordinary differential equations . . . 12
- 1.5 Fourier series of periodic functions . . . 14

2 Plotting **17**
- 2.1 Plotting functions in 2D . . . 17
- 2.2 Plotting curves . . . 18
- 2.3 openmath . . . 21
- 2.4 Tachyon 3D plotting . . . 22
- 2.5 gnuplot . . . 22
- 2.6 Plotting surfaces . . . 24

3 Groups **25**
- 3.1 Permutation groups . . . 25
- 3.2 Conjugacy classes . . . 27
- 3.3 Normal subgroups . . . 28
- 3.4 Centers . . . 29
- 3.5 The group id database . . . 30
- 3.6 Construction instructions for every group of order less than 32 . . . 31
- 3.7 Construction instructions for every finitely presented group of order 15 or less . . . 44

4 Linear algebra **47**
- 4.1 Choosing the base ring: exact vs. numerical . . . 47
- 4.2 Vector spaces . . . 49
- 4.3 Matrix powers . . . 49
- 4.4 Kernels . . . 50
- 4.5 Eigenvectors and eigenvalues . . . 53
- 4.6 Row reduction . . . 57
- 4.7 Characteristic polynomial . . . 58
- 4.8 Solving systems of linear equations . . . 59

5 Linear codes and ciphers **61**
- 5.1 Codes . . . 61
- 5.2 Ciphers . . . 64

6 Representation theory **69**
- 6.1 Ordinary characters . . . 69
- 6.2 Brauer characters . . . 73

7 Rings **77**
7.1 Matrix rings . . . 77
7.2 Polynomial rings . . . 78
7.3 p-adic numbers . . . 79
7.4 Quotient rings of polynomials . . . 80

8 Polynomials **83**
8.1 Polynomial powers . . . 83
8.2 Factorization . . . 84
8.3 Polynomial GCD's . . . 85
8.4 Roots of polynomials . . . 87
8.5 Evaluation of multivariate functions . . . 88
8.6 Roots of multivariate polynomials . . . 89
8.7 Gröbner bases . . . 90

9 Elementary number theory **93**
9.1 Taking modular powers . . . 93
9.2 Discrete logs . . . 93
9.3 Prime numbers . . . 94
9.4 Divisors . . . 95
9.5 Quadratic residues . . . 95

10 Modular forms **97**
10.1 Cusp forms . . . 97
10.2 Coset representatives . . . 97
10.3 Modular symbols and Hecke operators . . . 98
10.4 Genus formulas . . . 99

11 Elliptic curves **101**
11.1 Conductor . . . 101
11.2 j-invariant . . . 101
11.3 The $GF(q)$-rational points on E . . . 102
11.4 Modular form associated to an elliptic curve over **Q** . . . 103

12 Number fields **105**
12.1 Ramification . . . 105
12.2 Class numbers . . . 106
12.3 Integral basis . . . 109

13 Algebraic Geometry **111**
13.1 Point counting on curves . . . 111
13.2 Riemann-Roch spaces using Singular . . . 120

14 Interface Issues **129**
14.1 Background jobs . . . 129
14.2 Referencing Sage . . . 129
14.3 Logging your Sage session . . . 129
14.4 LaTeX conversion . . . 129
14.5 Sage and other computer algebra systems . . . 130
14.6 Command-line Sage help . . . 130
14.7 Reading and importing files into Sage . . . 131
14.8 Python language program code for Sage commands . . . 132
14.9 "Special functions" in Sage . . . 133
14.10 What is Sage? . . . 134

15 Contributions to this document 135

16 Indices and tables 137

Index 139

This document collects the answers to some questions along the line "How do I construct … in Sage?" Though much of this material can be found in the manual or tutorial or string documentation of the Python code, it is hoped that this will provide the casual user with some basic examples on how to start using Sage in a useful way.

This work is licensed under a Creative Commons Attribution-Share Alike 3.0 License.

Please send suggestions, additions, corrections to the `sage-devel` Google group!

The Sage wiki http://wiki.sagemath.org/ contains a wealth of information. Moreover, all these can be tested in the Sage interface http://sagecell.sagemath.org/ on the web.

CHAPTER

ONE

CALCULUS

Here are some examples of calculus symbolic computations using Sage.

1.1 Assumptions

By default a symbolic variable is as general as possible: it may stand for a positive, negative, or complex value. Many calculus results hold only under extra hypotheses, and Sage will not apply them unless you say so. Use `assume()` to attach a hypothesis to a variable, `assumptions()` to list what is currently assumed, and `forget()` to remove assumptions again. For example, $\sqrt{x^2} = x$ holds only for positive x:

```
sage: assume(x > 0)
sage: bool(sqrt(x^2) == x)
True
sage: assumptions()
[x > 0]
sage: forget()
sage: bool(sqrt(x^2) == x)
False
```

```
>>> from sage.all import *
>>> assume(x > Integer(0))
>>> bool(sqrt(x**Integer(2)) == x)
True
>>> assumptions()
[x > 0]
>>> forget()
>>> bool(sqrt(x**Integer(2)) == x)
False
```

Assumptions decide integrals and sums whose answers depend on a sign or range condition:

```
sage: n = var('n')
sage: assume(n + 1 > 0)
sage: integral(x^n, x)
x^(n + 1)/(n + 1)
sage: forget()
```

```
>>> from sage.all import *
>>> n = var('n')
>>> assume(n + Integer(1) > Integer(0))
```

(continues on next page)

(continued from previous page)

```
>>> integral(x**n, x)
x^(n + 1)/(n + 1)
>>> forget()
```

```
sage: var('q, a, k')
(q, a, k)
sage: assume(abs(q) < 1)
sage: sum(a*q^k, k, 0, oo)
-a/(q - 1)
sage: forget()
```

```
>>> from sage.all import *
>>> var('q, a, k')
(q, a, k)
>>> assume(abs(q) < Integer(1))
>>> sum(a*q**k, k, Integer(0), oo)
-a/(q - 1)
>>> forget()
```

A variable can also be declared to have a property such as `'integer'`, `'real'`, `'positive'`, or `'even'`:

```
sage: n = var('n')
sage: assume(n, 'integer')
sage: sin(n*pi)
0
sage: forget()
```

```
>>> from sage.all import *
>>> n = var('n')
>>> assume(n, 'integer')
>>> sin(n*pi)
0
>>> forget()
```

Pass each relationship as a separate assumption. A chained comparison is not an error, but it is silently truncated to its first relation (Python evaluates `0 < x < 1` as `(0 < x) and (x < 1)`, and the undecided first comparison short-circuits the `and`), so only `0 < x` gets recorded:

```
sage: assume(0 < x < 1)
sage: assumptions()
[0 < x]
sage: forget()
sage: assume(0 < x, x < 1)
sage: assumptions()
[0 < x, x < 1]
sage: forget()
```

```
>>> from sage.all import *
>>> assume(Integer(0) < x < Integer(1))
>>> assumptions()
[0 < x]
```

(continues on next page)

(continued from previous page)

```
>>> forget()
>>> assume(Integer(0) < x, x < Integer(1))
>>> assumptions()
[0 < x, x < 1]
>>> forget()
```

For a temporary hypothesis, use `assuming` in a `with` block: the assumption is in force inside the block and automatically forgotten afterwards:

```
sage: solve(x^2 == 4, x)
[x == -2, x == 2]
sage: with assuming(x > 0):
....:     solve(x^2 == 4, x)
[x == 2]
sage: assumptions()
[]
```

```
>>> from sage.all import *
>>> solve(x**Integer(2) == Integer(4), x)
[x == -2, x == 2]
>>> with assuming(x > Integer(0)):
...     solve(x**Integer(2) == Integer(4), x)
[x == 2]
>>> assumptions()
[]
```

A restriction can also be built into a variable when it is created, with the `domain` keyword (this too registers an assumption):

```
sage: t = var('t', domain='positive')
sage: abs(t)
t
sage: forget()
```

```
>>> from sage.all import *
>>> t = var('t', domain='positive')
>>> abs(t)
t
>>> forget()
```

If you have used Maple's `assume`/`about` or Mathematica's `Assuming`, this is the corresponding mechanism in Sage; see `sage.symbolic.assumptions` for the full reference, including `forget()` of individual assumptions.

1.2 Differentiation

Differentiation:

```
sage: var('x k w')
(x, k, w)
sage: f = x^3 * e^(k*x) * sin(w*x); f
x^3*e^(k*x)*sin(w*x)
sage: f.diff(x)
```

(continues on next page)

(continued from previous page)

```
w*x^3*cos(w*x)*e^(k*x) + k*x^3*e^(k*x)*sin(w*x) + 3*x^2*e^(k*x)*sin(w*x)
sage: latex(f.diff(x))
w x^{3} \cos\left(w x\right) e^{\left(k x\right)} + k x^{3} e^{\left(k x\right)} \sin\
↪left(w x\right) + 3 \, x^{2} e^{\left(k x\right)} \sin\left(w x\right)
```

```
>>> from sage.all import *
>>> var('x k w')
(x, k, w)
>>> f = x**Integer(3) * e**(k*x) * sin(w*x); f
x^3*e^(k*x)*sin(w*x)
>>> f.diff(x)
w*x^3*cos(w*x)*e^(k*x) + k*x^3*e^(k*x)*sin(w*x) + 3*x^2*e^(k*x)*sin(w*x)
>>> latex(f.diff(x))
w x^{3} \cos\left(w x\right) e^{\left(k x\right)} + k x^{3} e^{\left(k x\right)} \sin\
↪left(w x\right) + 3 \, x^{2} e^{\left(k x\right)} \sin\left(w x\right)
```

If you type `view(f.diff(x))` another window will open up displaying the compiled output. In the notebook, you can enter

```
var('x k w')
f = x^3 * e^(k*x) * sin(w*x)
show(f)
show(f.diff(x))
```

into a cell and press `shift-enter` for a similar result. You can also differentiate and integrate using the commands

```
R = PolynomialRing(QQ,"x")
x = R.gen()
p = x^2 + 1
show(p.derivative())
show(p.integral())
```

in a notebook cell, or

```
sage: R = PolynomialRing(QQ,"x")
sage: x = R.gen()
sage: p = x^2 + 1
sage: p.derivative()
2*x
sage: p.integral()
1/3*x^3 + x
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ,"x")
>>> x = R.gen()
>>> p = x**Integer(2) + Integer(1)
>>> p.derivative()
2*x
>>> p.integral()
1/3*x^3 + x
```

on the command line. At this point you can also type `view(p.derivative())` or `view(p.integral())` to open a new window with output typeset by LaTeX.

1.2.1 Critical points

You can find critical points of a piecewise defined function:

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f1 = x^0
sage: f2 = 1-x
sage: f3 = 2*x
sage: f4 = 10*x-x^2
sage: f = piecewise([((0,1),f1), ((1,2),f2), ((2,3),f3), ((3,10),f4)])
sage: f.critical_points()
[5.0]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f1 = x**Integer(0)
>>> f2 = Integer(1)-x
>>> f3 = Integer(2)*x
>>> f4 = Integer(10)*x-x**Integer(2)
>>> f = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2),␣
↪((Integer(2),Integer(3)),f3), ((Integer(3),Integer(10)),f4)])
>>> f.critical_points()
[5.0]
```

1.2.2 Power series

Sage offers several ways to construct and work with power series.

To get Taylor series from function expressions use the method `.taylor()` on the expression:

```
sage: var('f0 k x')
(f0, k, x)
sage: g = f0/sinh(k*x)^4
sage: g.taylor(x, 0, 3)
-62/945*f0*k^2*x^2 + 11/45*f0 - 2/3*f0/(k^2*x^2) + f0/(k^4*x^4)
```

```
>>> from sage.all import *
>>> var('f0 k x')
(f0, k, x)
>>> g = f0/sinh(k*x)**Integer(4)
>>> g.taylor(x, Integer(0), Integer(3))
-62/945*f0*k^2*x^2 + 11/45*f0 - 2/3*f0/(k^2*x^2) + f0/(k^4*x^4)
```

Formal power series expansions of functions can be had with the `.series()` method:

```
sage: (1/(2-cos(x))).series(x,7)
1 + (-1/2)*x^2 + 7/24*x^4 + (-121/720)*x^6 + Order(x^7)
```

```
>>> from sage.all import *
>>> (Integer(1)/(Integer(2)-cos(x))).series(x,Integer(7))
1 + (-1/2)*x^2 + 7/24*x^4 + (-121/720)*x^6 + Order(x^7)
```

Certain manipulations on such series are hard to perform at the moment, however. There are two alternatives: either use the Maxima subsystem of Sage for full symbolic functionality:

```
sage: f = log(sin(x)/x)
sage: f.taylor(x, 0, 10)
-1/467775*x^10 - 1/37800*x^8 - 1/2835*x^6 - 1/180*x^4 - 1/6*x^2
sage: maxima(f).powerseries(x,0)._sage_()
sum(2^(2*i... - 1)*(-1)^i...*x^(2*i...)*bern(2*i...)/(i...*factorial(2*i...)), i...,␣
↪1, +Infinity)
```

```
>>> from sage.all import *
>>> f = log(sin(x)/x)
>>> f.taylor(x, Integer(0), Integer(10))
-1/467775*x^10 - 1/37800*x^8 - 1/2835*x^6 - 1/180*x^4 - 1/6*x^2
>>> maxima(f).powerseries(x,Integer(0))._sage_()
sum(2^(2*i... - 1)*(-1)^i...*x^(2*i...)*bern(2*i...)/(i...*factorial(2*i...)), i...,␣
↪1, +Infinity)
```

Or you can use the formal power series rings for fast computation. These are missing symbolic functions, on the other hand:

```
sage: R.<w> = QQ[[]]
sage: ps = w + 17/2*w^2 + 15/4*w^4 + O(w^6); ps
w + 17/2*w^2 + 15/4*w^4 + O(w^6)
sage: ps.exp()
1 + w + 9*w^2 + 26/3*w^3 + 265/6*w^4 + 413/10*w^5 + O(w^6)
sage: (1+ps).log()
w + 8*w^2 - 49/6*w^3 - 193/8*w^4 + 301/5*w^5 + O(w^6)
sage: (ps^1000).coefficients()
[1, 8500, 36088875, 102047312625, 1729600092867375/8]
```

```
>>> from sage.all import *
>>> R = QQ[['w']]; (w,) = R._first_ngens(1)
>>> ps = w + Integer(17)/Integer(2)*w**Integer(2) + Integer(15)/
↪Integer(4)*w**Integer(4) + O(w**Integer(6)); ps
w + 17/2*w^2 + 15/4*w^4 + O(w^6)
>>> ps.exp()
1 + w + 9*w^2 + 26/3*w^3 + 265/6*w^4 + 413/10*w^5 + O(w^6)
>>> (Integer(1)+ps).log()
w + 8*w^2 - 49/6*w^3 - 193/8*w^4 + 301/5*w^5 + O(w^6)
>>> (ps**Integer(1000)).coefficients()
[1, 8500, 36088875, 102047312625, 1729600092867375/8]
```

1.3 Integration

Numerical integration is discussed in *Riemann and trapezoid sums for integrals* below.

Sage can integrate some simple functions on its own:

```
sage: f = x^3
sage: f.integral(x)
1/4*x^4
sage: integral(x^3,x)
1/4*x^4
sage: f = x*sin(x^2)
```

(continues on next page)

(continued from previous page)

```
sage: integral(f,x)
-1/2*cos(x^2)
```

```
>>> from sage.all import *
>>> f = x**Integer(3)
>>> f.integral(x)
1/4*x^4
>>> integral(x**Integer(3),x)
1/4*x^4
>>> f = x*sin(x**Integer(2))
>>> integral(f,x)
-1/2*cos(x^2)
```

Sage can also compute symbolic definite integrals involving limits.

```
sage: var('x, k, w')
(x, k, w)
sage: f = x^3 * e^(k*x) * sin(w*x)
sage: f.integrate(x)
((24*k^3*w*e^(k*x) - 24*k*w^3*e^(k*x) - (k^6*w*e^(k*x) + 3*k^4*w^3*e^(k*x) + 3*k^2*w^
↪5*e^(k*x) + w^7*e^(k*x))*x^3 + 6*(k^5*w*e^(k*x) + 2*k^3*w^3*e^(k*x) + k*w^5*e^
↪(k*x))*x^2 - 6*(3*k^4*w*e^(k*x) + 2*k^2*w^3*e^(k*x) - w^5*e^(k*x))*x)*cos(w*x) -␣
↪(6*k^4*e^(k*x) - 36*k^2*w^2*e^(k*x) + 6*w^4*e^(k*x) - (k^7*e^(k*x) + 3*k^5*w^2*e^
↪(k*x) + 3*k^3*w^4*e^(k*x) + k*w^6*e^(k*x))*x^3 + 3*(k^6*e^(k*x) + k^4*w^2*e^(k*x) -␣
↪k^2*w^4*e^(k*x) - w^6*e^(k*x))*x^2 - 6*(k^5*e^(k*x) - 2*k^3*w^2*e^(k*x) - 3*k*w^4*e^
↪(k*x))*x)*sin(w*x))/(k^8 + 4*k^6*w^2 + 6*k^4*w^4 + 4*k^2*w^6 + w^8)
sage: integrate(1/x^2, x, 1, infinity)
1
```

```
>>> from sage.all import *
>>> var('x, k, w')
(x, k, w)
>>> f = x**Integer(3) * e**(k*x) * sin(w*x)
>>> f.integrate(x)
((24*k^3*w*e^(k*x) - 24*k*w^3*e^(k*x) - (k^6*w*e^(k*x) + 3*k^4*w^3*e^(k*x) + 3*k^2*w^
↪5*e^(k*x) + w^7*e^(k*x))*x^3 + 6*(k^5*w*e^(k*x) + 2*k^3*w^3*e^(k*x) + k*w^5*e^
↪(k*x))*x^2 - 6*(3*k^4*w*e^(k*x) + 2*k^2*w^3*e^(k*x) - w^5*e^(k*x))*x)*cos(w*x) -␣
↪(6*k^4*e^(k*x) - 36*k^2*w^2*e^(k*x) + 6*w^4*e^(k*x) - (k^7*e^(k*x) + 3*k^5*w^2*e^
↪(k*x) + 3*k^3*w^4*e^(k*x) + k*w^6*e^(k*x))*x^3 + 3*(k^6*e^(k*x) + k^4*w^2*e^(k*x) -␣
↪k^2*w^4*e^(k*x) - w^6*e^(k*x))*x^2 - 6*(k^5*e^(k*x) - 2*k^3*w^2*e^(k*x) - 3*k*w^4*e^
↪(k*x))*x)*sin(w*x))/(k^8 + 4*k^6*w^2 + 6*k^4*w^4 + 4*k^2*w^6 + w^8)
>>> integrate(Integer(1)/x**Integer(2), x, Integer(1), infinity)
1
```

1.3.1 Convolution

You can find the convolution of any piecewise defined function with another (off the domain of definition, they are assumed to be zero). Here is f, $f * f$, and $f * f * f$, where $f(x) = 1, 0 < x < 1$:

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: f = piecewise([((0,1),1*x^0)])
```

(continues on next page)

(continued from previous page)

```
sage: g = f.convolution(f)
sage: h = f.convolution(g)
sage: set_verbose(-1)
sage: P = f.plot(); Q = g.plot(rgbcolor=(1,1,0)); R = h.plot(rgbcolor=(0,1,1))
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> f = piecewise([((Integer(0),Integer(1)),Integer(1)*x**Integer(0))])
>>> g = f.convolution(f)
>>> h = f.convolution(g)
>>> set_verbose(-Integer(1))
>>> P = f.plot(); Q = g.plot(rgbcolor=(Integer(1),Integer(1),Integer(0))); R = h.
↪plot(rgbcolor=(Integer(0),Integer(1),Integer(1)))
```

To view this, type `show(P+Q+R)`.

1.3.2 Riemann and trapezoid sums for integrals

Regarding numerical approximation of $\int_a^b f(x)\,dx$, where f is a piecewise defined function, can

- compute (for plotting purposes) the piecewise linear function defined by the trapezoid rule for numerical integration based on a subdivision into N subintervals
- the approximation given by the trapezoid rule,
- compute (for plotting purposes) the piecewise constant function defined by the Riemann sums (left-hand, right-hand, or midpoint) in numerical integration based on a subdivision into N subintervals,
- the approximation given by the Riemann sum approximation.

```
sage: f1(x) = x^2
sage: f2(x) = 5-x^2
sage: f = piecewise([[[0,1], f1], [RealSet.open_closed(1,2), f2]])
sage: t = f.trapezoid(2); t
piecewise(x|-->1/2*x on (0, 1/2), x|-->3/2*x - 1/2 on (1/2, 1), x|-->7/2*x - 5/2 on␣
↪(1, 3/2), x|-->-7/2*x + 8 on (3/2, 2); x)
sage: t.integral()
piecewise(x|-->1/4*x^2 on (0, 1/2), x|-->3/4*x^2 - 1/2*x + 1/8 on (1/2, 1), x|-->7/
↪4*x^2 - 5/2*x + 9/8 on (1, 3/2), x|-->-7/4*x^2 + 8*x - 27/4 on (3/2, 2); x)
sage: t.integral(definite=True)
9/4
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f1 = symbolic_expression(x**Integer(2)).function(x)
>>> __tmp__=var("x"); f2 = symbolic_expression(Integer(5)-x**Integer(2)).function(x)
>>> f = piecewise([[[Integer(0),Integer(1)], f1], [RealSet.open_closed(Integer(1),
↪Integer(2)), f2]])
>>> t = f.trapezoid(Integer(2)); t
piecewise(x|-->1/2*x on (0, 1/2), x|-->3/2*x - 1/2 on (1/2, 1), x|-->7/2*x - 5/2 on␣
↪(1, 3/2), x|-->-7/2*x + 8 on (3/2, 2); x)
>>> t.integral()
piecewise(x|-->1/4*x^2 on (0, 1/2), x|-->3/4*x^2 - 1/2*x + 1/8 on (1/2, 1), x|-->7/
↪4*x^2 - 5/2*x + 9/8 on (1, 3/2), x|-->-7/4*x^2 + 8*x - 27/4 on (3/2, 2); x)
```

(continues on next page)

(continued from previous page)

```
>>> t.integral(definite=True)
9/4
```

1.3.3 Laplace transforms

If you have a piecewise-defined polynomial function then there is a "native" command for computing Laplace transforms. This calls Maxima but it's worth noting that Maxima cannot handle (using the direct interface illustrated in the last few examples) this type of computation.

```
sage: var('x s')
(x, s)
sage: f1(x) = 1
sage: f2(x) = 1-x
sage: f = piecewise([((0,1),f1), ((1,2),f2)])
sage: f.laplace(x, s)
-e^(-s)/s + (s + 1)*e^(-2*s)/s^2 + 1/s - e^(-s)/s^2
```

```
>>> from sage.all import *
>>> var('x s')
(x, s)
>>> __tmp__=var("x"); f1 = symbolic_expression(Integer(1)).function(x)
>>> __tmp__=var("x"); f2 = symbolic_expression(Integer(1)-x).function(x)
>>> f = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2)])
>>> f.laplace(x, s)
-e^(-s)/s + (s + 1)*e^(-2*s)/s^2 + 1/s - e^(-s)/s^2
```

For other "reasonable" functions, Laplace transforms can be computed using the Maxima interface:

```
sage: var('k, s, t')
(k, s, t)
sage: f = 1/exp(k*t)
sage: f.laplace(t,s)
1/(k + s)
```

```
>>> from sage.all import *
>>> var('k, s, t')
(k, s, t)
>>> f = Integer(1)/exp(k*t)
>>> f.laplace(t,s)
1/(k + s)
```

is one way to compute LT's and

```
sage: var('s, t')
(s, t)
sage: f = t^5*exp(t)*sin(t)
sage: L = laplace(f, t, s); L
240*(3*s^5 - 15*s^4 + 20*s^3 - 12*s + 4)/(s^12 - 12*s^11 + 72*s^10 - 280*s^9 + 780*s^
↪8 - 1632*s^7 + 2624*s^6 - 3264*s^5 + 3120*s^4 - 2240*s^3 + 1152*s^2 - 384*s + 64)
```

```
>>> from sage.all import *
>>> var('s, t')
(s, t)
>>> f = t**Integer(5)*exp(t)*sin(t)
>>> L = laplace(f, t, s); L
240*(3*s^5 - 15*s^4 + 20*s^3 - 12*s + 4)/(s^12 - 12*s^11 + 72*s^10 - 280*s^9 + 780*s^
↪8 - 1632*s^7 + 2624*s^6 - 3264*s^5 + 3120*s^4 - 2240*s^3 + 1152*s^2 - 384*s + 64)
```

is another way.

1.4 Ordinary differential equations

Symbolically solving ODEs can be done using Sage interface with Maxima. See

```
sage:desolvers?
```

```
>>> from sage.all import *
sage:desolvers?
```

for available commands. Numerical solution of ODEs can be done using Sage interface with Octave (an experimental package), or routines in the GSL (Gnu Scientific Library).

An example, how to solve ODE's symbolically in Sage using the Maxima interface (do not type the `....:`):

```
sage: y=function('y')(x); desolve(diff(y,x,2) + 3*x == y, dvar = y, ics = [1,1,1])
3*x - 2*e^(x - 1)
sage: desolve(diff(y,x,2) + 3*x == y, dvar = y)
_K2*e^(-x) + _K1*e^x + 3*x
sage: desolve(diff(y,x) + 3*x == y, dvar = y)
(3*(x + 1)*e^(-x) + _C)*e^x
sage: desolve(diff(y,x) + 3*x == y, dvar = y, ics = [1,1]).expand()
3*x - 5*e^(x - 1) + 3

sage: f=function('f')(x); desolve_laplace(diff(f,x,2) == 2*diff(f,x)-f, dvar = f, ics␣
↪= [0,1,2])
x*e^x + e^x

sage: desolve_laplace(diff(f,x,2) == 2*diff(f,x)-f, dvar = f)
-x*e^x*f(0) + x*e^x*D[0](f)(0) + e^x*f(0)
```

```
>>> from sage.all import *
>>> y=function('y')(x); desolve(diff(y,x,Integer(2)) + Integer(3)*x == y, dvar = y,␣
↪ics = [Integer(1),Integer(1),Integer(1)])
3*x - 2*e^(x - 1)
>>> desolve(diff(y,x,Integer(2)) + Integer(3)*x == y, dvar = y)
_K2*e^(-x) + _K1*e^x + 3*x
>>> desolve(diff(y,x) + Integer(3)*x == y, dvar = y)
(3*(x + 1)*e^(-x) + _C)*e^x
>>> desolve(diff(y,x) + Integer(3)*x == y, dvar = y, ics = [Integer(1),Integer(1)]).
↪expand()
3*x - 5*e^(x - 1) + 3
```

(continues on next page)

(continued from previous page)

```
>>> f=function('f')(x); desolve_laplace(diff(f,x,Integer(2)) == Integer(2)*diff(f,x)-
↪f, dvar = f, ics = [Integer(0),Integer(1),Integer(2)])
x*e^x + e^x

>>> desolve_laplace(diff(f,x,Integer(2)) == Integer(2)*diff(f,x)-f, dvar = f)
-x*e^x*f(0) + x*e^x*D[0](f)(0) + e^x*f(0)
```

If you have `Octave` and `gnuplot` installed,

```
sage: octave.de_system_plot(['x+y','x-y'], [1,-1], [0,2]) # optional - octave
```

```
>>> from sage.all import *
>>> octave.de_system_plot(['x+y','x-y'], [Integer(1),-Integer(1)], [Integer(0),
↪Integer(2)]) # optional - octave
```

yields the two plots $(t, x(t)), (t, y(t))$ on the same graph (the t-axis is the horizontal axis) of the system of ODEs

$$x' = x + y, x(0) = 1; y' = x - y, y(0) = -1,$$

for $0 <= t <= 2$. The same result can be obtained by using `desolve_system_rk4`:

```
sage: x, y, t = var('x y t')
sage: P=desolve_system_rk4([x+y, x-y], [x,y], ics=[0,1,-1], ivar=t, end_points=2)
sage: p1 = list_plot([[i,j] for i,j,k in P], plotjoined=True)
sage: p2 = list_plot([[i,k] for i,j,k in P], plotjoined=True, color='red')
sage: p1+p2
Graphics object consisting of 2 graphics primitives
```

```
>>> from sage.all import *
>>> x, y, t = var('x y t')
>>> P=desolve_system_rk4([x+y, x-y], [x,y], ics=[Integer(0),Integer(1),-Integer(1)],␣
↪ivar=t, end_points=Integer(2))
>>> p1 = list_plot([[i,j] for i,j,k in P], plotjoined=True)
>>> p2 = list_plot([[i,k] for i,j,k in P], plotjoined=True, color='red')
>>> p1+p2
Graphics object consisting of 2 graphics primitives
```

Another way this system can be solved is to use the command `desolve_system`.

```
sage: t=var('t'); x=function('x',t); y=function('y',t)
sage: des = [diff(x,t) == x+y, diff(y,t) == x-y]
sage: desolve_system(des, [x,y], ics = [0, 1, -1])
[x(t) == cosh(sqrt(2)*t), y(t) == sqrt(2)*sinh(sqrt(2)*t) - cosh(sqrt(2)*t)]
```

```
>>> from sage.all import *
>>> t=var('t'); x=function('x',t); y=function('y',t)
>>> des = [diff(x,t) == x+y, diff(y,t) == x-y]
>>> desolve_system(des, [x,y], ics = [Integer(0), Integer(1), -Integer(1)])
[x(t) == cosh(sqrt(2)*t), y(t) == sqrt(2)*sinh(sqrt(2)*t) - cosh(sqrt(2)*t)]
```

The output of this command is *not* a pair of functions.

Finally, can solve linear DEs using power series:

```
sage: R.<t> = PowerSeriesRing(QQ, default_prec=10)
sage: a = 2 - 3*t + 4*t^2 + O(t^10)
sage: b = 3 - 4*t^2 + O(t^7)
sage: f = a.solve_linear_de(prec=5, b=b, f0=3/5)
sage: f
3/5 + 21/5*t + 33/10*t^2 - 38/15*t^3 + 11/24*t^4 + O(t^5)
sage: f.derivative() - a*f - b
O(t^4)
```

```
>>> from sage.all import *
>>> R = PowerSeriesRing(QQ, default_prec=Integer(10), names=('t',)); (t,) = R._first_
↪ngens(1)
>>> a = Integer(2) - Integer(3)*t + Integer(4)*t**Integer(2) + O(t**Integer(10))
>>> b = Integer(3) - Integer(4)*t**Integer(2) + O(t**Integer(7))
>>> f = a.solve_linear_de(prec=Integer(5), b=b, f0=Integer(3)/Integer(5))
>>> f
3/5 + 21/5*t + 33/10*t^2 - 38/15*t^3 + 11/24*t^4 + O(t^5)
>>> f.derivative() - a*f - b
O(t^4)
```

1.5 Fourier series of periodic functions

Let f be a real-valued periodic function of period $2L$. The Fourier series of f is

$$S(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

where

$$a_n = \frac{1}{L} \int_{-L}^{L} f(x) \cos\left(\frac{n\pi x}{L}\right) dx,$$

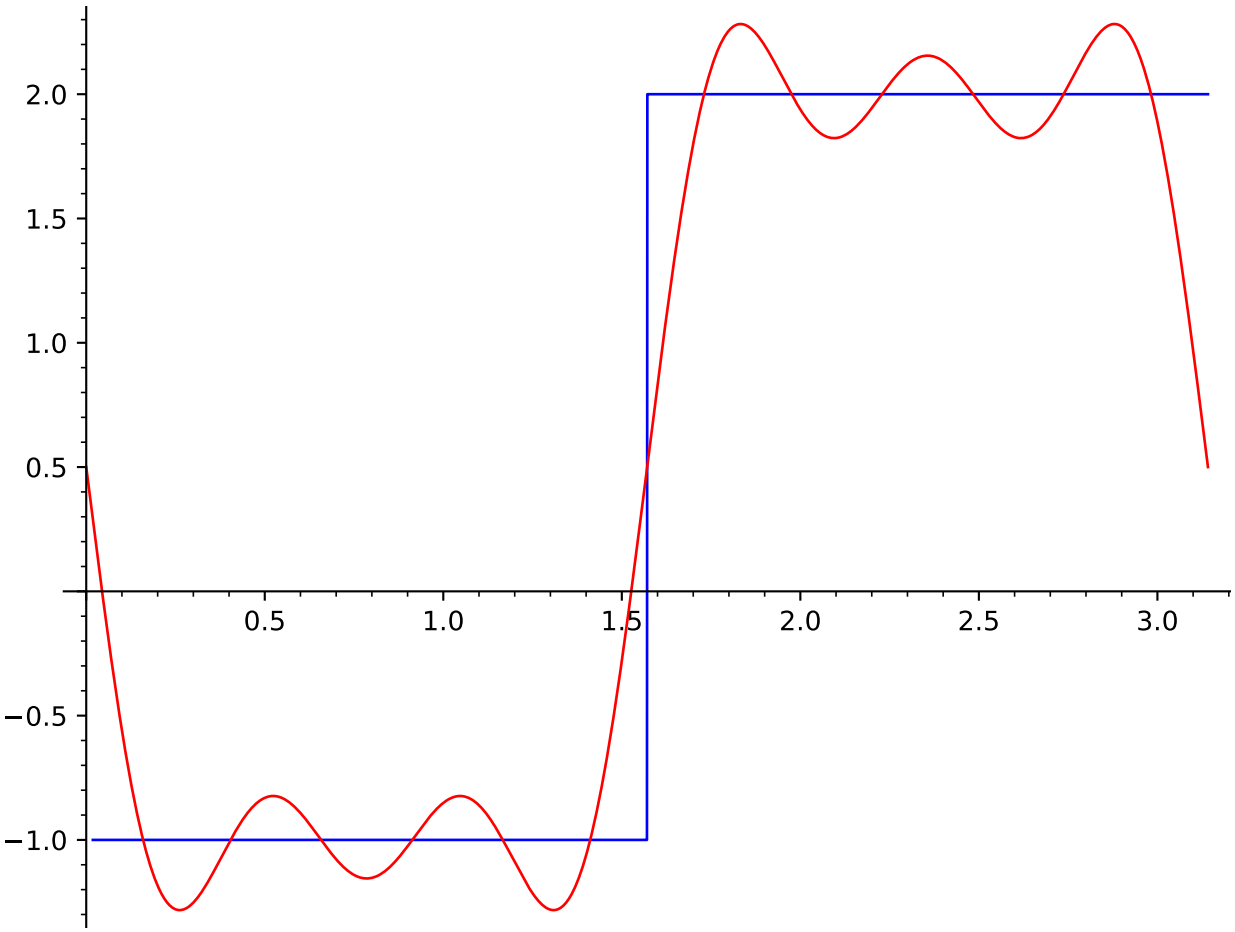
and

$$b_n = \frac{1}{L} \int_{-L}^{L} f(x) \sin\left(\frac{n\pi x}{L}\right) dx,$$

The Fourier coefficients a_n and b_n are computed by declaring f as a piecewise-defined function over one period and invoking the methods `fourier_series_cosine_coefficient` and `fourier_series_sine_coefficient`, while the partial sums are obtained via `fourier_series_partial_sum`:

```
sage: f = piecewise([((0,pi/2), -1), ((pi/2,pi), 2)])
sage: f.fourier_series_cosine_coefficient(0)
1
sage: f.fourier_series_sine_coefficient(5)
-6/5/pi
sage: s5 = f.fourier_series_partial_sum(5); s5
-6/5*sin(10*x)/pi - 2*sin(6*x)/pi - 6*sin(2*x)/pi + 1/2
sage: plot(f, (0,pi)) + plot(s5, (x,0,pi), color='red')
Graphics object consisting of 2 graphics primitives
```

```
>>> from sage.all import *
>>> f = piecewise([((Integer(0),pi/Integer(2)), -Integer(1)), ((pi/Integer(2),pi),␣
↪Integer(2))])
>>> f.fourier_series_cosine_coefficient(Integer(0))
1
>>> f.fourier_series_sine_coefficient(Integer(5))
-6/5/pi
>>> s5 = f.fourier_series_partial_sum(Integer(5)); s5
-6/5*sin(10*x)/pi - 2*sin(6*x)/pi - 6*sin(2*x)/pi + 1/2
>>> plot(f, (Integer(0),pi)) + plot(s5, (x,Integer(0),pi), color='red')
Graphics object consisting of 2 graphics primitives
```



CHAPTER

TWO

PLOTTING

Sage can plot using matplotlib, openmath, gnuplot, or surf but only matplotlib and openmath are included with Sage in the standard distribution. For surf examples, see *Plotting surfaces*.

Plotting in Sage can be done in many different ways. You can plot a function (in 2 or 3 dimensions) or a set of points (in 2-D only) via gnuplot, you can plot a solution to a differential equation via Maxima (which in turn calls gnuplot or openmath), or you can use Singular's interface with the plotting package surf (which does not come with Sage). `gnuplot` does not have an implicit plotting command, so if you want to plot a curve or surface using an implicit plot, it is best to use the Singular's interface to surf, as described in chapter ch:AG, Algebraic geometry.

2.1 Plotting functions in 2D

The default plotting method in uses the excellent `matplotlib` package.

To view any of these, type `P.save("<path>/myplot.png")` and then open it in a graphics viewer such as gimp.

You can plot piecewise-defined functions:

```
sage: f1 = 1
sage: f2 = 1-x
sage: f3 = exp(x)
sage: f4 = sin(2*x)
sage: f = piecewise([((0,1),f1), ((1,2),f2), ((2,3),f3), ((3,10),f4)])
sage: f.plot(x,0,10)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> f1 = Integer(1)
>>> f2 = Integer(1)-x
>>> f3 = exp(x)
>>> f4 = sin(Integer(2)*x)
>>> f = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2),␣
↪((Integer(2),Integer(3)),f3), ((Integer(3),Integer(10)),f4)])
>>> f.plot(x,Integer(0),Integer(10))
Graphics object consisting of 1 graphics primitive
```

Other function plots can be produced as well:

A red plot of the Jacobi elliptic function $\mathrm{sn}(x, 2)$, $-3 < x < 3$ (do not type the `....:`:

```
sage: L = [(i/100.0, maxima.eval('jacobi_sn (%s/100.0,2.0)'%i))
....:     for i in range(-300,300)]
sage: show(line(L, rgbcolor=(3/4,1/4,1/8)))
```

```
>>> from sage.all import *
>>> L = [(i/RealNumber('100.0'), maxima.eval('jacobi_sn (%s/100.0,2.0)'%i))
...      for i in range(-Integer(300),Integer(300))]
>>> show(line(L, rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(4),Integer(1)/
↪Integer(8))))
```

A red plot of J-Bessel function $J_2(x)$, $0 < x < 10$:

```
sage: L = [(i/10.0, maxima.eval('bessel_j (2,%s/10.0)'%i)) for i in range(100)]
sage: show(line(L, rgbcolor=(3/4,1/4,5/8)))
```

```
>>> from sage.all import *
>>> L = [(i/RealNumber('10.0'), maxima.eval('bessel_j (2,%s/10.0)'%i)) for i in␣
↪range(Integer(100))]
>>> show(line(L, rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(4),Integer(5)/
↪Integer(8))))
```

A purple plot of the Riemann zeta function $\zeta(1/2 + it)$, $0 < t < 30$:

```
sage: I = CDF.0
sage: show(line([zeta(1/2 + k*I/6) for k in range(180)], rgbcolor=(3/4,1/2,5/8)))
```

```
>>> from sage.all import *
>>> I = CDF.gen(0)
>>> show(line([zeta(Integer(1)/Integer(2) + k*I/Integer(6)) for k in␣
↪range(Integer(180))], rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(2),
↪Integer(5)/Integer(8))))
```

2.2 Plotting curves

To plot a curve in Sage, you can use Singular and surf (http://surf.sourceforge.net/, also available as an experimental package) or use matplotlib (included with Sage).

2.2.1 matplotlib

Here are several examples. To view them, type `p.save("<path>/my_plot.png")` (where `<path>` is a directory path which you have write permissions to where you want to save the plot) and view it in a viewer (such as GIMP).

A blue conchoid of Nicomedes:

```
sage: L = [[1+5*cos(pi/2+pi*i/100), tan(pi/2+pi*i/100)*
....:       (1+5*cos(pi/2+pi*i/100))] for i in range(1,100)]
sage: line(L, rgbcolor=(1/4,1/8,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[Integer(1)+Integer(5)*cos(pi/Integer(2)+pi*i/Integer(100)), tan(pi/
↪Integer(2)+pi*i/Integer(100))*
...      (Integer(1)+Integer(5)*cos(pi/Integer(2)+pi*i/Integer(100)))] for i in␣
↪range(Integer(1),Integer(100))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(8),Integer(3)/
```

(continues on next page)

(continued from previous page)

```
↪Integer(4)))
Graphics object consisting of 1 graphics primitive
```

A blue hypotrochoid (3 leaves):

```
sage: n = 4; h = 3; b = 2
sage: L = [[n*cos(pi*i/100)+h*cos((n/b)*pi*i/100),
....:      n*sin(pi*i/100)-h*sin((n/b)*pi*i/100)] for i in range(200)]
sage: line(L, rgbcolor=(1/4,1/4,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> n = Integer(4); h = Integer(3); b = Integer(2)
>>> L = [[n*cos(pi*i/Integer(100))+h*cos((n/b)*pi*i/Integer(100)),
...      n*sin(pi*i/Integer(100))-h*sin((n/b)*pi*i/Integer(100))] for i in␣
↪range(Integer(200))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(4),Integer(3)/
↪Integer(4)))
Graphics object consisting of 1 graphics primitive
```

A blue hypotrochoid (4 leaves):

```
sage: n = 6; h = 5; b = 2
sage: L = [[n*cos(pi*i/100)+h*cos((n/b)*pi*i/100),
....:      n*sin(pi*i/100)-h*sin((n/b)*pi*i/100)] for i in range(200)]
sage: line(L, rgbcolor=(1/4,1/4,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> n = Integer(6); h = Integer(5); b = Integer(2)
>>> L = [[n*cos(pi*i/Integer(100))+h*cos((n/b)*pi*i/Integer(100)),
...      n*sin(pi*i/Integer(100))-h*sin((n/b)*pi*i/Integer(100))] for i in␣
↪range(Integer(200))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(4),Integer(3)/
↪Integer(4)))
Graphics object consisting of 1 graphics primitive
```

A red limaçon of Pascal:

```
sage: L = [[sin(pi*i/100)+sin(pi*i/50),-(1+cos(pi*i/100)+cos(pi*i/50))]
....:      for i in range(-100,101)]
sage: line(L, rgbcolor=(1,1/4,1/2))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[sin(pi*i/Integer(100))+sin(pi*i/Integer(50)),-(Integer(1)+cos(pi*i/
↪Integer(100))+cos(pi*i/Integer(50)))]
...      for i in range(-Integer(100),Integer(101))]
>>> line(L, rgbcolor=(Integer(1),Integer(1)/Integer(4),Integer(1)/Integer(2)))
Graphics object consisting of 1 graphics primitive
```

A light green trisectrix of Maclaurin:

```
sage: L = [[2*(1-4*cos(-pi/2+pi*i/100)^2),10*tan(-pi/2+pi*i/100)*
....:       (1-4*cos(-pi/2+pi*i/100)^2)] for i in range(1,100)]
sage: line(L, rgbcolor=(1/4,1,1/8))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[Integer(2)*(Integer(1)-Integer(4)*cos(-pi/Integer(2)+pi*i/
↪Integer(100))**Integer(2)),Integer(10)*tan(-pi/Integer(2)+pi*i/Integer(100))*
...       (Integer(1)-Integer(4)*cos(-pi/Integer(2)+pi*i/Integer(100))**Integer(2))]␣
↪for i in range(Integer(1),Integer(100))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1),Integer(1)/Integer(8)))
Graphics object consisting of 1 graphics primitive
```

A green lemniscate of Bernoulli (we omit i==100 since that would give a 0 division error):

```
sage: v = [(1/cos(-pi/2+pi*i/100), tan(-pi/2+pi*i/100)) for i in range(1,200) if i!
↪=100 ]
sage: L = [(a/(a^2+b^2), b/(a^2+b^2)) for a,b in v]
sage: line(L, rgbcolor=(1/4,3/4,1/8))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> v = [(Integer(1)/cos(-pi/Integer(2)+pi*i/Integer(100)), tan(-pi/Integer(2)+pi*i/
↪Integer(100))) for i in range(Integer(1),Integer(200)) if i!=Integer(100) ]
>>> L = [(a/(a**Integer(2)+b**Integer(2)), b/(a**Integer(2)+b**Integer(2))) for a,b␣
↪in v]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(3)/Integer(4),Integer(1)/
↪Integer(8)))
Graphics object consisting of 1 graphics primitive
```

2.2.2 surf

In particular, since `surf` is only available on a UNIX type OS (and is not included with Sage), plotting using the commands below in Sage is only available on such an OS. Incidentally, surf is included with several popular Linux distributions.

```
sage: s = singular.eval
sage: s('LIB "surf.lib";')
...
sage: s("ring rr0 = 0,(x1,x2),dp;")
''
sage: s("ideal I = x1^3 - x2^2;")
''
sage: s("plot(I);")
...
```

```
>>> from sage.all import *
>>> s = singular.eval
>>> s('LIB "surf.lib";')
...
>>> s("ring rr0 = 0,(x1,x2),dp;")
''
>>> s("ideal I = x1^3 - x2^2;")
```

(continues on next page)

(continued from previous page)

```
''
>>> s("plot(I);")
...
```

Press `q` with the surf window active to exit from surf and return to Sage.

You can save this plot as a surf script. In the surf window which pops up, just choose `file`, `save as`, etc.. (Type `q` or select `file`, `quit`, to close the window.)

The plot produced is omitted but the gentle reader is encouraged to try it out.

2.3 openmath

Openmath is a TCL/Tk GUI plotting program written by W. Schelter.

The following command plots the function $\cos(2x) + 2e^{-x}$

```
sage: maximal.plot2d('cos(2*x) + 2*exp(-x)','[x,0,1]',  # not tested (pops up a window)
....:     '[plot_format,openmath]')
```

```
>>> from sage.all import *
>>> maximal.plot2d('cos(2*x) + 2*exp(-x)','[x,0,1]',  # not tested (pops up a window)
...     '[plot_format,openmath]')
```

(Mac OS X users: Note that these `openmath` commands were run in a session of started in an xterm shell, not using the standard Mac Terminal application.)

```
sage: maximal.eval('load("plotdf");')
.../share/maximal.../share/dynamics/plotdf.lisp...
sage: maximal.eval('plotdf(x+y,[trajectory_at,2,-0.1]); ')  # not tested
```

```
>>> from sage.all import *
>>> maximal.eval('load("plotdf");')
.../share/maximal.../share/dynamics/plotdf.lisp...
>>> maximal.eval('plotdf(x+y,[trajectory_at,2,-0.1]); ')  # not tested
```

This plots a direction field (the plotdf Maxima package was also written by W. Schelter.)

A 2D plot of several functions:

```
sage: maximal.plot2d('[x,x^2,x^3]','[x,-1,1]','[plot_format,openmath]')  # not tested
```

```
>>> from sage.all import *
>>> maximal.plot2d('[x,x^2,x^3]','[x,-1,1]','[plot_format,openmath]')  # not tested
```

Openmath also does 3D plots of surfaces of the form $z = f(x, y)$, as x and y range over a rectangle. For example, here is a "live" 3D plot which you can move with your mouse:

```
sage: maximal.plot3d ("sin(x^2 + y^2)", "[x, -3, 3]", "[y, -3, 3]",  # not tested
....:     '[plot_format, openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot3d ("sin(x^2 + y^2)", "[x, -3, 3]", "[y, -3, 3]",  # not tested
...      '[plot_format, openmath]')
```

By rotating this suitably, you can view the contour plot.

2.4 Tachyon 3D plotting

The ray-tracing package Tachyon is distributed with Sage. The 3D plots look very nice but tend to take a bit more setting up. Here is an example of a parametric space curve:

```
sage: f = lambda t: (t,t^2,t^3)
sage: t = Tachyon(camera_center=(5,0,4))
sage: t.texture('t')
sage: t.light((-20,-20,40), 0.2, (1,1,1))
sage: t.parametric_plot(f,-5,5,'t',min_depth=6)
```

```
>>> from sage.all import *
>>> f = lambda t: (t,t**Integer(2),t**Integer(3))
>>> t = Tachyon(camera_center=(Integer(5),Integer(0),Integer(4)))
>>> t.texture('t')
>>> t.light((-Integer(20),-Integer(20),Integer(40)), RealNumber('0.2'), (Integer(1),
↪Integer(1),Integer(1)))
>>> t.parametric_plot(f,-Integer(5),Integer(5),'t',min_depth=Integer(6))
```

Type `t.show()` to view this.

Other examples are in the Reference Manual.

2.5 gnuplot

You must have `gnuplot` installed to run these commands.

First, here's way to plot a function: {plot!a function}

```
sage: maxima.plot2d('sin(x)','[x,-5,5]')
sage: opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-plot.eps"]'
sage: maxima.plot2d('sin(x)','[x,-5,5]',opts)
sage: opts = '[gnuplot_term, ps], [gnuplot_out_file, "/tmp/sin-plot.eps"]'
sage: maxima.plot2d('sin(x)','[x,-5,5]',opts)
```

```
>>> from sage.all import *
>>> maxima.plot2d('sin(x)','[x,-5,5]')
>>> opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-plot.eps"]'
>>> maxima.plot2d('sin(x)','[x,-5,5]',opts)
>>> opts = '[gnuplot_term, ps], [gnuplot_out_file, "/tmp/sin-plot.eps"]'
>>> maxima.plot2d('sin(x)','[x,-5,5]',opts)
```

The eps file is saved by default to the current directory but you may specify a path if you prefer.

Here is an example of a plot of a parametric curve in the plane:

```
sage: maxima.plot2d_parametric(["sin(t)","cos(t)"], "t",[-3.1,3.1])
sage: opts = '[gnuplot_preamble, "set nokey"], [gnuplot_term, ps],
....:     [gnuplot_out_file, "circle-plot.eps"]'
sage: maxima.plot2d_parametric(["sin(t)","cos(t)"], "t", [-3.1,3.1], options=opts)
```

```
>>> from sage.all import *
>>> maxima.plot2d_parametric(["sin(t)","cos(t)"], "t",[-RealNumber('3.1'),RealNumber(
↪'3.1')])
>>> opts = '[gnuplot_preamble, "set nokey"], [gnuplot_term, ps],
...     [gnuplot_out_file, "circle-plot.eps"]'
>>> maxima.plot2d_parametric(["sin(t)","cos(t)"], "t", [-RealNumber('3.1'),RealNumber(
↪'3.1')], options=opts)
```

Here is an example of a plot of a parametric surface in 3-space: {plot!a parametric surface}

```
sage: maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
....:     [-3.2,3.2],[0,3])
sage: opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-cos-plot.eps"]'
sage: maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
....:     [-3.2,3.2],[0,3],opts)
```

```
>>> from sage.all import *
>>> maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
...     [-RealNumber('3.2'),RealNumber('3.2')],[Integer(0),Integer(3)])
>>> opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-cos-plot.eps"]'
>>> maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
...     [-RealNumber('3.2'),RealNumber('3.2')],[Integer(0),Integer(3)],opts)
```

To illustrate how to pass gnuplot options in , here is an example of a plot of a set of points involving the Riemann zeta function $\zeta(s)$ (computed using Pari but plotted using Maxima and Gnuplot): {plot!points} {Riemann zeta function}

```
sage: zeta_ptsx = [ (pari(1/2 + i*I/10).zeta().real()).precision(1)
....:     for i in range(70,150)]
sage: zeta_ptsy = [ (pari(1/2 + i*I/10).zeta().imag()).precision(1)
....:     for i in range(70,150)]
sage: maxima.plot_list(zeta_ptsx, zeta_ptsy)  # optional -- pops up a window.
sage: opts='[gnuplot_preamble, "set nokey"], [gnuplot_term, ps],
....:     [gnuplot_out_file, "zeta.eps"]'
sage: maxima.plot_list(zeta_ptsx, zeta_ptsy, opts) # optional -- pops up a window.
```

```
>>> from sage.all import *
>>> zeta_ptsx = [ (pari(Integer(1)/Integer(2) + i*I/Integer(10)).zeta().real()).
↪precision(Integer(1))
...     for i in range(Integer(70),Integer(150))]
>>> zeta_ptsy = [ (pari(Integer(1)/Integer(2) + i*I/Integer(10)).zeta().imag()).
↪precision(Integer(1))
...     for i in range(Integer(70),Integer(150))]
>>> maxima.plot_list(zeta_ptsx, zeta_ptsy)  # optional -- pops up a window.
>>> opts='[gnuplot_preamble, "set nokey"], [gnuplot_term, ps],
...     [gnuplot_out_file, "zeta.eps"]'
>>> maxima.plot_list(zeta_ptsx, zeta_ptsy, opts) # optional -- pops up a window.
```

2.6 Plotting surfaces

To plot a surface in is no different that to plot a curve, though the syntax is slightly different. In particular, you need to have `surf` loaded. {plot!surface using surf}

```
sage: singular.eval('ring rr1 = 0,(x,y,z),dp;')
''
sage: singular.eval('ideal I(1) = 2x2-1/2x3 +1-y+1;')
''
sage: singular.eval('plot(I(1));')
...
```

```
>>> from sage.all import *
>>> singular.eval('ring rr1 = 0,(x,y,z),dp;')
''
>>> singular.eval('ideal I(1) = 2x2-1/2x3 +1-y+1;')
''
>>> singular.eval('plot(I(1));')
...
```

CHAPTER

THREE

GROUPS

3.1 Permutation groups

A permutation group is a subgroup of some symmetric group S_n. Sage has a Python class `PermutationGroup`, so you can work with such groups directly:

```
sage: G = PermutationGroup(['(1,2,3)(4,5)'])
sage: G
Permutation Group with generators [(1,2,3)(4,5)]
sage: g = G.gens()[0]; g
(1,2,3)(4,5)
sage: g*g
(1,3,2)
sage: G = PermutationGroup(['(1,2,3)'])
sage: g = G.gens()[0]; g
(1,2,3)
sage: g.order()
3
```

```
>>> from sage.all import *
>>> G = PermutationGroup(['(1,2,3)(4,5)'])
>>> G
Permutation Group with generators [(1,2,3)(4,5)]
>>> g = G.gens()[Integer(0)]; g
(1,2,3)(4,5)
>>> g*g
(1,3,2)
>>> G = PermutationGroup(['(1,2,3)'])
>>> g = G.gens()[Integer(0)]; g
(1,2,3)
>>> g.order()
3
```

For the example of the Rubik's cube group (a permutation subgroup of S_{48}, where the non-center facets of the Rubik's cube are labeled $1, 2, ..., 48$ in some fixed way), you can use the GAP-Sage interface as follows.

```
sage: cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
```

(continues on next page)

(continued from previous page)

```
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
sage: gap(cube)
'permutation group with 6 generators'
sage: gap("Size(cubegp)")
43252003274489856000'
```

```
>>> from sage.all import *
>>> cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
>>> gap(cube)
'permutation group with 6 generators'
>>> gap("Size(cubegp)")
43252003274489856000'
```

Another way you can choose to do this:

- Create a file `cubegroup.py` containing the lines:

```
cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
```

  Then place the file in the subdirectory `$SAGE_ROOT/local/lib/python2.4/site-packages/sage` of your Sage home directory. Last, read (i.e., `import`) it into Sage:

```
sage: import sage.cubegroup
sage: sage.cubegroup.cube
'cubegp := Group(( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)
(11,35,27,19),( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)
( 6,22,46,35),(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)
( 8,30,41,11),(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)
( 8,33,48,24),(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)
( 1,14,48,27),(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)
(16,24,32,40) )'
sage: gap(sage.cubegroup.cube)
'permutation group with 6 generators'
sage: gap("Size(cubegp)")
'43252003274489856000'
```

```
>>> from sage.all import *
>>> import sage.cubegroup
>>> sage.cubegroup.cube
```

(continues on next page)

(continued from previous page)

```
'cubegp := Group(( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)
(11,35,27,19),( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)
( 6,22,46,35),(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)
( 8,30,41,11),(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)
( 8,33,48,24),(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)
( 1,14,48,27),(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)
(16,24,32,40) )'
>>> gap(sage.cubegroup.cube)
'permutation group with 6 generators'
>>> gap("Size(cubegp)")
'43252003274489856000'
```

(You will have line wrap instead of the above carriage returns in your Sage output.)

- Use the `CubeGroup` class:

```
sage: rubik = CubeGroup()
sage: rubik
The Rubik's cube group with generators R,L,F,B,U,D in SymmetricGroup(48).
sage: rubik.order()
43252003274489856000
```

```
>>> from sage.all import *
>>> rubik = CubeGroup()
>>> rubik
The Rubik's cube group with generators R,L,F,B,U,D in SymmetricGroup(48).
>>> rubik.order()
43252003274489856000
```

(1) has implemented classical groups (such as $GU(3, \mathbf{F}_5)$) and matrix groups over a finite field with user-defined generators.

(2) also has implemented finite and infinite (but finitely generated) abelian groups.

3.2 Conjugacy classes

You can compute conjugacy classes of a finite group using "natively":

```
sage: G = PermutationGroup(['(1,2,3)', '(1,2)(3,4)', '(1,7)'])
sage: CG = G.conjugacy_classes_representatives()
sage: gamma = CG[2]
sage: CG; gamma
[(), (4,7), (3,4,7), (2,3)(4,7), (2,3,4,7), (1,2)(3,4,7), (1,2,3,4,7)]
(3,4,7)
```

```
>>> from sage.all import *
>>> G = PermutationGroup(['(1,2,3)', '(1,2)(3,4)', '(1,7)'])
>>> CG = G.conjugacy_classes_representatives()
>>> gamma = CG[Integer(2)]
>>> CG; gamma
[(), (4,7), (3,4,7), (2,3)(4,7), (2,3,4,7), (1,2)(3,4,7), (1,2,3,4,7)]
(3,4,7)
```

You can use the Sage-GAP interface:

```
sage: libgap.eval("G := Group((1,2)(3,4),(1,2,3))")
Group([ (1,2)(3,4), (1,2,3) ])
sage: libgap.eval("CG := ConjugacyClasses(G)")
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
sage: libgap.eval("gamma := CG[3]")
(2,4,3)^G
sage: libgap.eval("g := Representative(gamma)")
(2,4,3)
```

```
>>> from sage.all import *
>>> libgap.eval("G := Group((1,2)(3,4),(1,2,3))")
Group([ (1,2)(3,4), (1,2,3) ])
>>> libgap.eval("CG := ConjugacyClasses(G)")
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
>>> libgap.eval("gamma := CG[3]")
(2,4,3)^G
>>> libgap.eval("g := Representative(gamma)")
(2,4,3)
```

Or, here's another (more "pythonic") way to do this type of computation:

```
sage: G = libgap.eval("Group([(1,2,3), (1,2)(3,4), (1,7)])")
sage: CG = G.ConjugacyClasses()
sage: gamma = CG[2]
sage: g = gamma.Representative()
sage: CG; gamma; g
[ ()^G, (4,7)^G, (3,4,7)^G, (2,3)(4,7)^G, (2,3,4,7)^G, (1,2)(3,4,7)^G, (1,2,3,4,7)^G ]
(3,4,7)^G
(3,4,7)
```

```
>>> from sage.all import *
>>> G = libgap.eval("Group([(1,2,3), (1,2)(3,4), (1,7)])")
>>> CG = G.ConjugacyClasses()
>>> gamma = CG[Integer(2)]
>>> g = gamma.Representative()
>>> CG; gamma; g
[ ()^G, (4,7)^G, (3,4,7)^G, (2,3)(4,7)^G, (2,3,4,7)^G, (1,2)(3,4,7)^G, (1,2,3,4,7)^G ]
(3,4,7)^G
(3,4,7)
```

3.3 Normal subgroups

If you want to find all the normal subgroups of a permutation group G (up to conjugacy), you can use Sage's interface to GAP:

```
sage: G = AlternatingGroup( 5 )
sage: libgap(G).NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

```
>>> from sage.all import *
>>> G = AlternatingGroup( Integer(5) )
>>> libgap(G).NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

or

```
sage: G = libgap.AlternatingGroup( 5 )
sage: G.NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

```
>>> from sage.all import *
>>> G = libgap.AlternatingGroup( Integer(5) )
>>> G.NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

Here's another way, working more directly with GAP:

```
sage: libgap.eval("G := AlternatingGroup( 5 )")
Alt( [ 1 .. 5 ] )
sage: libgap.eval("normal := NormalSubgroups( G )")
[ Alt( [ 1 .. 5 ] ), Group(()) ]
sage: G = libgap.eval("DihedralGroup( 10 )")
sage: G.NormalSubgroups().SortedList()
[ Group([  ]), Group([ f2 ]), <pc group of size 10 with 2 generators> ]
sage: libgap.eval("G := SymmetricGroup( 4 )")
Sym( [ 1 .. 4 ] )
sage: libgap.eval("normal := NormalSubgroups( G );")
[ Sym( [ 1 .. 4 ] ), Alt( [ 1 .. 4 ] ), Group([ (1,4)(2,3),  ... ]),
      Group(()) ]
```

```
>>> from sage.all import *
>>> libgap.eval("G := AlternatingGroup( 5 )")
Alt( [ 1 .. 5 ] )
>>> libgap.eval("normal := NormalSubgroups( G )")
[ Alt( [ 1 .. 5 ] ), Group(()) ]
>>> G = libgap.eval("DihedralGroup( 10 )")
>>> G.NormalSubgroups().SortedList()
[ Group([  ]), Group([ f2 ]), <pc group of size 10 with 2 generators> ]
>>> libgap.eval("G := SymmetricGroup( 4 )")
Sym( [ 1 .. 4 ] )
>>> libgap.eval("normal := NormalSubgroups( G );")
[ Sym( [ 1 .. 4 ] ), Alt( [ 1 .. 4 ] ), Group([ (1,4)(2,3),  ... ]),
      Group(()) ]
```

3.4 Centers

How do you compute the center of a group in Sage?

Although Sage calls GAP to do the computation of the group center, `center` is "wrapped" (i.e., Sage has a class PermutationGroup with associated class method "center"), so the user does not need to use the `libgap` command. Here's an example:

```
sage: G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
sage: G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
```

```
>>> from sage.all import *
>>> G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
>>> G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
```

A similar syntax for matrix groups also works:

```
sage: G = SL(2, GF(5) )
sage: G.center()
Subgroup with 1 generators (
[4 0]
[0 4]
) of Special Linear Group of degree 2 over Finite Field of size 5
sage: G = PSL(2, 5 )
sage: G.center()
Subgroup generated by [()] of (The projective special linear group of degree 2 over␣
↪Finite Field of size 5)
```

```
>>> from sage.all import *
>>> G = SL(Integer(2), GF(Integer(5)) )
>>> G.center()
Subgroup with 1 generators (
[4 0]
[0 4]
) of Special Linear Group of degree 2 over Finite Field of size 5
>>> G = PSL(Integer(2), Integer(5) )
>>> G.center()
Subgroup generated by [()] of (The projective special linear group of degree 2 over␣
↪Finite Field of size 5)
```

Note

`center` can be spelled either way in GAP, not so in Sage.

3.5 The group id database

The function `group_id` uses the Small Groups Library of E. A. O'Brien, B. Eick, and H. U. Besche, which is a part of GAP.

```
sage: G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
sage: G.order()
120
sage: G.group_id()
[120, 34]
```

```
>>> from sage.all import *
>>> G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
>>> G.order()
120
>>> G.group_id()
[120, 34]
```

Another example of using the small groups database: `group_id`

```
sage: gap_console()
┌─────────┐   GAP 4.10.0 of 01-Nov-2018
│  GAP  │   https://www.gap-system.org
└─────────┘   Architecture: x86_64-pc-linux-gnu-default64
Configuration:  gmp 6.0.0, readline
Loading the library and packages ...
Packages:   GAPDoc 1.6.2, PrimGrp 3.3.2, SmallGrp 1.3, TransGrp 2.0.4
Try '??help' for help. See also '?copyright', '?cite' and '?authors'
gap> G:=Group((4,6,5)(7,8,9),(1,7,2,4,6,9,5,3));
Group([ (4,6,5)(7,8,9), (1,7,2,4,6,9,5,3) ])
gap> StructureDescription(G);
"(C3 x C3) : GL(2,3)"
```

```
>>> from sage.all import *
>>> gap_console()
┌─────────┐   GAP 4.10.0 of 01-Nov-2018
│  GAP  │   https://www.gap-system.org
└─────────┘   Architecture: x86_64-pc-linux-gnu-default64
Configuration:  gmp 6.0.0, readline
Loading the library and packages ...
Packages:   GAPDoc 1.6.2, PrimGrp 3.3.2, SmallGrp 1.3, TransGrp 2.0.4
Try '??help' for help. See also '?copyright', '?cite' and '?authors'
gap> G:=Group((4,6,5)(7,8,9),(1,7,2,4,6,9,5,3));
Group([ (4,6,5)(7,8,9), (1,7,2,4,6,9,5,3) ])
gap> StructureDescription(G);
"(C3 x C3) : GL(2,3)"
```

3.6 Construction instructions for every group of order less than 32

AUTHORS:

- Davis Shurbert

Every group of order less than 32 is implemented in Sage as a permutation group. They can all be created easily. We will first show how to build direct products and semidirect products, then give the commands necessary to build all of these small groups.

Let `G1`, `G2`, ..., `Gn` be permutation groups already initialized in Sage. The following command can be used to take their direct product (where, of course, the ellipses are simply being used here as a notation, and you actually must enter every factor in your desired product explicitly).

```
sage: G = direct_product_permgroups([G1, G2, ..., Gn])
```

```
>>> from sage.all import *
>>> G = direct_product_permgroups([G1, G2, Ellipsis, Gn])
```

The semidirect product operation can be thought of as a generalization of the direct product operation. Given two groups, H and K, their semidirect product, $H \ltimes_\phi K$, (where $\phi : H \to Aut(K)$ is a homomorphism) is a group whose underlying set is the cartersian product of H and K, but with the operation:

$$(h_1, k_1)(h_2, k_2) = (h_1 h_2, k_1^{\phi(h_2)} k_2).$$

The output is not the group explicitly described in the definition of the operation, but rather an isomorphic group of permutations. In the routine below, assume `H` and `K` already have been defined and initialized in Sage. Also, `phi` is a list containing two sublists that define the underlying homomorphism by giving the images of a set of generators of `H`. For each semidirect product in the table below we will show you how to build `phi`, then assume you have read this passage and understand how to go from there.

```
sage: G = H.semidirect_product(K, phi)
```

```
>>> from sage.all import *
>>> G = H.semidirect_product(K, phi)
```

To avoid unnecessary repetition, we will now give commands that one can use to create the cyclic group of order n, C_n, and the dihedral group on n letters, D_n. We will present one more example of each to ensure that the reader understands the command, then it will be withheld.

```
sage: G = CyclicPermutationGroup(n)

sage: G = DihedralGroup(n)
```

```
>>> from sage.all import *
>>> G = CyclicPermutationGroup(n)

>>> G = DihedralGroup(n)
```

Note that exponential notation will be used for the direct product operation. For example, ${C_2}^2 = C_2 \times C_2$. This table was crafted with the help of *Group Tables*, by AD Thomas and GV Wood (1980, Shiva Publishing).

Order	Group Description	Command(s)	GAP ID

1	The Trivial Group	`sage: G = SymmetricGroup(1)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(1))`
			[1,1]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

2	C_2	`sage: G = SymmetricGroup(2)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(2))`
			[2,1]

3	C_3	`sage: G = CyclicPermutationGroup(3)`
		`>>> from sage.all import *`
		`>>> G = CyclicPermutationGroup(Integer(3))`
			[3,1]

4	C_4		[4,1]

4	$C_2 \times C_2$	`sage: G = KleinFourGroup()`
		`>>> from sage.all import *`
		`>>> G = KleinFourGroup()`
			[4,2]

5	C_5		[5,1]
6	C_6		[6,2]

6	S_3 (Symmetric Group on 3 letters)	`sage: G = SymmetricGroup(3)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(3))`
			[6,1]

7	C_7		[7,1]
8	C_8		[8,1]
8	$C_4 \times C_2$		[8,2]
8	$C_2 \times C_2 \times C_2$		[8,5]

8	D_4	`sage: G = DihedralGroup(4)`
		`>>> from sage.all import *`
		`>>> G = DihedralGroup(Integer(4))`
			[8,3]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

8	The Quaternion Group (Q)	`sage: G = QuaternionGroup()`
		`>>> from sage.all import *`
		`>>> G = QuaternionGroup()`
			[8,4]

9	C_9		[9,1]
9	$C_3 \times C_3$		[9,2]
10	C_{10}		[10,2]
10	D_5		[10,1]
11	C_{11}		[11,1]
12	C_{12}		[12,2]
12	$C_6 \times C_2$		[12,5]
12	D_6		[12,4]

12	A_4 (Alternating Group on 4 letters)	`sage: G = AlternatingGroup(4)`
		`>>> from sage.all import *`
		`>>> G = AlternatingGroup(Integer(4))`
			[12,3]

12	Q_6 (DiCyclic group of order 12)	`sage: G = DiCyclicGroup(3)`
		`>>> from sage.all import *`
		`>>> G = DiCyclicGroup(Integer(3))`
			[12,1]

13	C_{13}		[13,1]
14	C_{14}		[14,2]
14	D_7		[14,1]
15	C_{15}		[15,1]
16	C_{16}		[16,1]
16	$C_8 \times C_2$		[16,5]
16	$C_4 \times C_4$		[16,2]
16	$C_4 \times C_2 \times C_2$		[16,10]
16	${C_2}^4$		[16,14]
16	$D_4 \times C_2$		[16,11]
16	$Q \times C_2$		[16,12]
16	D_8		[16,7]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

16	Q_8 (Dicyclic group of order 16)	`sage: G = DiCyclicGroup(4)`
		`>>> from sage.all import *`
		`>>> G = DiCyclicGroup(Integer(4))`
			[16,9]

16	Semidihedral Group of order 2^4	`sage: G = SemidihedralGroup(4`
		`>>> from sage.all import *`
		`>>> G = SemidihedralGroup(Integer(4))`
			[16,8]

16	Split Metacyclic Group of order 2^4	`sage: G = SplitMetacyclicGroup(2,4)`
		`>>> from sage.all import *`
		`>>> G = SplitMetacyclicGroup(Integer(2),Integer(4))`
			[16,6]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

16	$(C_4 \times C_2) \rtimes_\phi C_2$	sage: C2 = SymmetricGroup(2); C4 = CyclicPermutationGroup(4)
		sage: A = direct_product_permgroups([C2, C4])
		sage: alpha = PermutationGroupMorphism(A,A,[A.gens()[0],A.gens()[0]^2*A.gens()[1]])
		sage: phi = [[(1,2)],[alpha]]
		>>> from sage.all import *
		>>> C2 = SymmetricGroup(Integer(2)); C4 = CyclicPermutationGroup(Integer(4))
		>>> A = direct_product_permgroups([C2, C4])
		>>> alpha = PermutationGroupMorphism(A,A,[A.gens()[Integer(0)],A.gens()[Integer(0)]**Integer(2)*A.gens()[Integer(1)]])
		>>> phi = [[(Integer(1),Integer(2))],[alpha]]
			[16,13]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

16	$(C_4 \times C_2) \rtimes_\phi C_2$	sage: C2 = SymmetricGroup(2); C4 = CyclicPermutationGroup(4)
		sage: A = direct_product_permgroups([C2, C4])
		sage: alpha = PermutationGroupMorphism(A,A,[A.gens()[0]^3*A.gens()[1], A.gens()[1]])
		sage: phi = [[(1,2)],[alpha]]
		
		>>> from sage.all import *
		>>> C2 = SymmetricGroup(Integer(2)); C4 = CyclicPermutationGroup(Integer(4))
		>>> A = direct_product_permgroups([C2, C4])
		>>> alpha = PermutationGroupMorphism(A,A,[A.gens()[Integer(0)]**Integer(3)*A.gens()[Integer(1)],A.gens()[Integer(1)]])
		>>> phi = [[(Integer(1),Integer(2))],[alpha]]
			[16,3]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID
16	$C_4 \rtimes_\phi C_4$	`sage: C4 = CyclicPe ↪rmutationGroup(4) sage: alpha = Permu ↪tationGroupMorphi ↪sm(C4,C4,[C4.gen( ↪).inverse()]) sage: phi = [[(1,2, ↪3,4)],[alpha]]` `>>> from sage.all␣ ↪import * >>> C4 = CyclicPerm ↪utationGroup(Inte ↪ger(4)) >>> alpha = Permuta ↪tionGroupMorphism ↪(C4,C4,[C4.gen(). ↪inverse()]) >>> phi =␣ ↪[[(Integer(1), ↪Integer(2), ↪Integer(3), ↪Integer(4))], ↪[alpha]]`	[16,4]
17	C_{17}		[17,1]
18	C_{18}		[18,2]
18	$C_6 \times C_3$		[18,5]
18	D_9		[18,1]
18	$S_3 \times C_3$		[18,3]
18	$Dih(C_3 \times C_3)$	`sage: G = GeneralDi ↪hedralGroup([3,3] ↪)` `>>> from sage.all␣ ↪import * >>> G = GeneralDihe ↪dralGroup([Intege ↪r(3),Integer(3)])`	[18,4]
19	C_{19}		[19,1]
20	C_{20}		[20,2]
20	$C_{10} \times C_2$		[20,5]
20	D_{10}		[20,4]
20	Q_{10} (Dicyclic Group of order 20)		[20,1]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

20	$Hol(C_5)$	`sage: C5 = CyclicPermutationGroup(5)`
		`sage: G = C5.holomorph()`
		
		`>>> from sage.all import *`
		`>>> C5 = CyclicPermutationGroup(Integer(5))`
		`>>> G = C5.holomorph()`
			[20,3]

21	C_{21}		[21,2]

21	$C_7 \rtimes_\phi C_3$	`sage: C7 = CyclicPermutationGroup(7)`
		`sage: alpha = PermutationGroupMorphism(C7,C7,[C7.gen()**4])`
		`sage: phi = [[(1,2,3)],[alpha]]`
		
		`>>> from sage.all import *`
		`>>> C7 = CyclicPermutationGroup(Integer(7))`
		`>>> alpha = PermutationGroupMorphism(C7,C7,[C7.gen()**Integer(4)])`
		`>>> phi = [[(Integer(1),Integer(2),Integer(3))],[alpha]]`
			[21,1]

22	C_{22}		[22,2]
22	D_{11}		[22,1]
23	C_{23}		[23,1]
24	C_{24}		[24,2]
24	D_{12}		[24,6]
24	Q_{12} (DiCyclic Group of order 24)		[24,4]
24	$C_{12} \times C_2$		[24,9]
24	$C_6 \times C_2 \times C_2$		[24,15]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

24	S_4 (Symmetric Group on 4 letters)	`sage: G = SymmetricGroup(4)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(4))`
			[24,12]

24	$S_3 \times C_4$		[24,5]
24	$S_3 \times C_2 \times C_2$		[24,14]
24	$D_4 \times C_3$		[24,10]
24	$Q \times C_3$		[24,11]
24	$A_4 \times C_2$		[24,13]
24	$Q_6 \times C_2$		[24,7]

24	$Q \rtimes_\phi C_3$	`sage: Q = QuaternionGroup()`
		`sage: alpha = PermutationGroupMorphism(Q,Q,[Q.gens()[0]*Q.gens()[1],Q.gens()[0].inverse()])`
		`sage: phi = [[(1,2,3)],[alpha]]`
		`>>> from sage.all import *`
		`>>> Q = QuaternionGroup()`
		`>>> alpha = PermutationGroupMorphism(Q,Q,[Q.gens()[Integer(0)]*Q.gens()[Integer(1)],Q.gens()[Integer(0)].inverse()])`
		`>>> phi = [[(Integer(1),Integer(2),Integer(3))],[alpha]]`
			[24,3]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

24	$C_3 \rtimes_\phi C_8$	sage: C3 = CyclicPe
		↪rmutationGroup(3)
		sage: alpha = Permu
		↪tationGroupMorphi
		↪sm(C3,C3,[C3.gen(
		↪).inverse()])
		sage: phi = [[(1,2,
		↪3,4,5,6,7,8)],
		↪[alpha]]
		>>> from sage.all␣
		↪import *
		>>> C3 = CyclicPerm
		↪utationGroup(Inte
		↪ger(3))
		>>> alpha = Permuta
		↪tionGroupMorphism
		↪(C3,C3,[C3.gen().
		↪inverse()])
		>>> phi =␣
		↪[[(Integer(1),
		↪Integer(2),
		↪Integer(3),
		↪Integer(4),
		↪Integer(5),
		↪Integer(6),
		↪Integer(7),
		↪Integer(8))],
		↪[alpha]]
			[24,1]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID

24	$C_3 \rtimes_\phi D_4$	sage: C3 = CyclicPermutationGroup(3)
		sage: alpha1 = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		sage: alpha2 = PermutationGroupMorphism(C3,C3,[C3.gen()])
		sage: phi = [[(1,2,3,4),(1,3)],[alpha1,alpha2]]
		>>> from sage.all import *
		>>> C3 = CyclicPermutationGroup(Integer(3))
		>>> alpha1 = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		>>> alpha2 = PermutationGroupMorphism(C3,C3,[C3.gen()])
		>>> phi = [[(Integer(1),Integer(2),Integer(3),Integer(4)),(Integer(1),Integer(3))],[alpha1,alpha2]]
			[24,8]

25	C_{25}		[25,1]
25	$C_5 \times C_5$		[25,2]
26	C_{26}		[26,2]
26	D_{13}		[26,1]
27	C_{27}		[27,1]
27	$C_9 \times C_3$		[27,2]
27	$C_3 \times C_3 \times C_3$		[27,5]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID
27	Split Metacyclic Group of order 3^3	`sage: G = SplitMeta ↪cyclicGroup(3,3)` `>>> from sage.all_ ↪import * >>> G = SplitMetacy ↪clicGroup(Integer ↪(3),Integer(3))`	[27,4]
27	$(C_3 \times C_3) \rtimes_\phi C_3$	`sage: C3 = CyclicPe ↪rmutationGroup(3) sage: A = direct_ ↪product_ ↪permgroups([C3, ↪C3]) sage: alpha = Permu ↪tationGroupMorphi ↪sm(A,A,[A.gens()[ ↪0]*A.gens()[1].in ↪verse(),A.gens()[ ↪1]]) sage: phi = [[(1,2, ↪3)],[alpha]]` `>>> from sage.all_ ↪import * >>> C3 = CyclicPerm ↪utationGroup(Inte ↪ger(3)) >>> A = direct_ ↪product_ ↪permgroups([C3, ↪C3]) >>> alpha = Permuta ↪tionGroupMorphism ↪(A,A,[A.gens()[In ↪teger(0)]*A.gens(↪)[Integer(1)].inv ↪erse(),A.gens()[I ↪nteger(1)]]) >>> phi =_ ↪[[(Integer(1), ↪Integer(2), ↪Integer(3))], ↪[alpha]]`	[27,3]
28	C_{28}		[28,2]
28	$C_{14} \times C_2$		[28,4]
28	D_{14}		[28,3]
28	Q_{14} (DiCyclic Group of order 28)		[28,1]

continues on next page

Table 1 – continued from previous page

Order	Group Description	Command(s)	GAP ID
29	C_{29}		[29,1]
30	C_{30}		[30,4]
30	D_{15}		[30,3]
30	$D_5 \times C_3$		[30,2]
30	$D_3 \times C_5$		[30,1]
31	C_{31}		[31,1]

Table By Kevin Halasz

3.7 Construction instructions for every finitely presented group of order 15 or less

Sage has the capability to easily construct every group of order 15 or less as a finitely presented group. We will begin with some discussion on creating finitely generated abelian groups, as well as direct and semidirect products of finitely presented groups.

All finitely generated abelian groups can be created using the `groups.presentation.FGAbelian(ls)` command, where `ls` is a list of non-negative integers which gets reduced to invariants defining the group to be returned. For example, to construct $C_4 \times C_2 \times C_2 \times C_2$ we can simply use:

```
sage: A = groups.presentation.FGAbelian([4,2,2,2])
```

```
>>> from sage.all import *
>>> A = groups.presentation.FGAbelian([Integer(4),Integer(2),Integer(2),Integer(2)])
```

The output for a given group is the same regardless of the input list of integers. The following example yields identical presentations for the cyclic group of order 30.

```
sage: A = groups.presentation.FGAbelian([2,3,5])
sage: B = groups.presentation.FGAbelian([30])
```

```
>>> from sage.all import *
>>> A = groups.presentation.FGAbelian([Integer(2),Integer(3),Integer(5)])
>>> B = groups.presentation.FGAbelian([Integer(30)])
```

If `G` and `H` are finitely presented groups, we can use the following code to create the direct product of `G` and `H`, $G \times H$.

```
sage: D = G.direct_product(H)
```

```
>>> from sage.all import *
>>> D = G.direct_product(H)
```

Suppose there exists a homomorphism ϕ from a group G to the automorphism group of a group H. Define the semidirect product of G with H via ϕ, as the Cartesian product of G and H, with the operation $(g_1, h_1)(g_2, h_2) = (g_1 g_2, \phi_{h_1}(g_2) h_2)$ where $\phi_h = \phi(h)$. To construct this product in Sage for two finitely presented groups, we must define ϕ manually using a pair of lists. The first list consists of generators of the group G, while the second list consists of images of the corresponding generators in the first list. These automorphisms are similarly defined as a pair of lists, generators in one and images in the other. As an example, we construct the dihedral group of order 16 as a semidirect product of cyclic groups.

```
sage: C2 = groups.presentation.Cyclic(2)
sage: C8 = groups.presentation.Cyclic(8)
sage: hom = (C2.gens(), [ ([C8([1])], [C8([-1])]) ])
sage: D = C2.semidirect_product(C8, hom)
```

```
>>> from sage.all import *
>>> C2 = groups.presentation.Cyclic(Integer(2))
>>> C8 = groups.presentation.Cyclic(Integer(8))
>>> hom = (C2.gens(), [ ([C8([Integer(1)])], [C8([-Integer(1)])]) ])
>>> D = C2.semidirect_product(C8, hom)
```

The following table shows the groups of order 15 or less, and how to construct them in Sage. Repeated commands have been omitted but instead are described by the following examples.

The cyclic group of order n can be created with a single command:

```
sage: C = groups.presentation.Cyclic(n)
```

```
>>> from sage.all import *
>>> C = groups.presentation.Cyclic(n)
```

Similarly for the dihedral group of order $2n$:

```
sage: D = groups.presentation.Dihedral(n)
```

```
>>> from sage.all import *
>>> D = groups.presentation.Dihedral(n)
```

This table was modeled after the preceding table created by Kevin Halasz.

Order	Group Description	Command(s)	GAP ID

1	The Trivial Group	`sage: G = groups.presentation.Symmetric(1)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(1))`
			[1,1]

2	C_2	`sage: G = groups.presentation.Symmetric(2)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(2))`
			[2,1]

3	C_3	`sage: G = groups.presentation.Cyclic(3)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Cyclic(Integer(3))`
			[3,1]

4	C_4		[4,1]

4	$C_2 \times C_2$	`sage: G = groups.presentation.Klein()`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Klein()`
			[4,2]

5	C_5		[5,1]
6	C_6		[6,2]

6	S_3 (Symmetric Group on 3 letters)	`sage: G = groups.presentation.Symmetric(3)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(3))`
			[6,1]

7	C_7		[7,1]
8	C_8		[8,1]
8	$C_4 \times C_2$	`sage: G = groups.`	[8,2]

CHAPTER

FOUR

LINEAR ALGEBRA

4.1 Choosing the base ring: exact vs. numerical

Every vector or matrix in Sage is defined over a *base ring*, and the same computation can behave very differently depending on which one you choose. The most common choices are:

Base ring	When to use it
`ZZ`	Integers: exact. Determinants, Smith/Hermite normal forms, lattice problems.
`QQ`	Rationals: exact. The default for structural questions: rank, kernel, echelon form, inverse, exact solving.
`GF(q)`	Finite fields: exact modular arithmetic, e.g. for coding theory and cryptography.
`RR`, `CC`	Floating-point with adjustable precision (53 bits by default, any precision via `RealField(prec)`), but without fast matrix decompositions.
`RDF`, `CDF`	Hardware double-precision floats: fast numerical linear algebra via LAPACK (SVD, QR, Cholesky, Schur, …).
`RBF`, `CBF`	Ball arithmetic: floating-point intervals with certified error bounds.

Exact rings give exact answers:

```
sage: A = matrix(QQ, [[1, 1/2], [1/3, 1/4]])
sage: A.det()
1/12
sage: A.inverse()
[ 3 -6]
[-4 12]
```

```
>>> from sage.all import *
>>> A = matrix(QQ, [[Integer(1), Integer(1)/Integer(2)], [Integer(1)/Integer(3),␣
↪Integer(1)/Integer(4)]])
>>> A.det()
1/12
>>> A.inverse()
[ 3 -6]
[-4 12]
```

The same matrix over `RDF` computes with floating-point rounding:

```
sage: A = matrix(QQ, [[1, 1/2], [1/3, 1/4]])
sage: Ad = A.change_ring(RDF)
```

(continues on next page)

(continued from previous page)

```
sage: Ad.det()  # abs tol 1e-15
0.08333333333333334
```

```
>>> from sage.all import *
>>> A = matrix(QQ, [[Integer(1), Integer(1)/Integer(2)], [Integer(1)/Integer(3),␣
↪Integer(1)/Integer(4)]])
>>> Ad = A.change_ring(RDF)
>>> Ad.det()  # abs tol 1e-15
0.08333333333333334
```

so exact equality tests that hold over `QQ` can fail over `RDF`:

```
sage: matrix(QQ, [[1/10]]) * 3 == matrix(QQ, [[3/10]])
True
sage: matrix(RDF, [[0.1]]) * 3 == matrix(RDF, [[0.3]])
False
```

```
>>> from sage.all import *
>>> matrix(QQ, [[Integer(1)/Integer(10)]]) * Integer(3) == matrix(QQ, [[Integer(3)/
↪Integer(10)]])
True
>>> matrix(RDF, [[RealNumber('0.1')]]) * Integer(3) == matrix(RDF, [[RealNumber('0.3
↪')]])
False
```

Over an exact ring, eigenvalues are returned as exact algebraic numbers (the trailing `?` marks an element of `QQbar`, printed to its known precision); over `RDF` they are ordinary floats:

```
sage: B = matrix(QQ, [[0, 1], [2, 0]])
sage: B.eigenvalues()
[-1.414213562373095?, 1.414213562373095?]
sage: sorted(B.change_ring(RDF).eigenvalues())  # abs tol 1e-14
[-1.414213562373095, 1.4142135623730951]
```

```
>>> from sage.all import *
>>> B = matrix(QQ, [[Integer(0), Integer(1)], [Integer(2), Integer(0)]])
>>> B.eigenvalues()
[-1.414213562373095?, 1.414213562373095?]
>>> sorted(B.change_ring(RDF).eigenvalues())  # abs tol 1e-14
[-1.414213562373095, 1.4142135623730951]
```

Over `RDF` and `CDF`, numerical decompositions such as `SVD`, `QR`, `cholesky` and `schur` are computed by optimized LAPACK routines, and some of them (`SVD`, `schur`) are available over no other ring:

```
sage: Ad = matrix(RDF, [[1, 1/2], [1/3, 1/4]])
sage: U, S, V = Ad.SVD()
sage: S.diagonal()  # abs tol 1e-14
[1.1910987498956362, 0.0699634126394936]
```

```
>>> from sage.all import *
>>> Ad = matrix(RDF, [[Integer(1), Integer(1)/Integer(2)], [Integer(1)/Integer(3),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(1)/Integer(4)]])
>>> U, S, V = Ad.SVD()
>>> S.diagonal()  # abs tol 1e-14
[1.1910987498956362, 0.0699634126394936]
```

For floating-point results carrying rigorous error bounds, use the ball fields `RBF`/`CBF`:

```
sage: M = matrix(RBF, [[1, 1/3], [1/3, 1]])
sage: M.det()
[0.888888888888889 +/- ...]
```

```
>>> from sage.all import *
>>> M = matrix(RBF, [[Integer(1), Integer(1)/Integer(3)], [Integer(1)/Integer(3),␣
↪Integer(1)]])
>>> M.det()
[0.888888888888889 +/- ...]
```

As a rule of thumb: start with `QQ` when you want exact structural answers; switch to `RDF`/`CDF` for speed on large matrices and for the numerical decompositions; use `RR`/`CC` when you need a nondefault precision; and use `RBF`/`CBF` when you need floating-point answers you can certify.

4.2 Vector spaces

The `VectorSpace` command creates a vector space class, from which one can create a subspace. Note the basis computed by Sage is "row reduced".

```
sage: V = VectorSpace(GF(2),8)
sage: S = V.subspace([V([1,1,0,0,0,0,0,0]),V([1,0,0,0,0,1,1,0])])
sage: S.basis()
[(1, 0, 0, 0, 0, 1, 1, 0), (0, 1, 0, 0, 0, 1, 1, 0)]
sage: S.dimension()
2
```

```
>>> from sage.all import *
>>> V = VectorSpace(GF(Integer(2)),Integer(8))
>>> S = V.subspace([V([Integer(1),Integer(1),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(0)]),V([Integer(1),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(1),Integer(1),Integer(0)])])
>>> S.basis()
[(1, 0, 0, 0, 0, 1, 1, 0), (0, 1, 0, 0, 0, 1, 1, 0)]
>>> S.dimension()
2
```

4.3 Matrix powers

How do I compute matrix powers in Sage? The syntax is illustrated by the example below.

```
sage: R = IntegerModRing(51)
sage: M = MatrixSpace(R,3,3)
sage: A = M([1,2,3, 4,5,6, 7,8,9])
```

(continues on next page)

(continued from previous page)

```
sage: A^1000*A^1007

[ 3  3  3]
[18  0 33]
[33 48 12]
sage: A^2007

[ 3  3  3]
[18  0 33]
[33 48 12]
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(51))
>>> M = MatrixSpace(R,Integer(3),Integer(3))
>>> A = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6),␣
↪Integer(7),Integer(8),Integer(9)])
>>> A**Integer(1000)*A**Integer(1007)
<BLANKLINE>
[ 3  3  3]
[18  0 33]
[33 48 12]
>>> A**Integer(2007)
<BLANKLINE>
[ 3  3  3]
[18  0 33]
[33 48 12]
```

4.4 Kernels

The kernel is computed by applying the kernel method to the matrix object. The following examples illustrate the syntax.

```
sage: M = MatrixSpace(IntegerRing(),4,2)(range(8))
sage: M.kernel()
Free module of degree 4 and rank 2 over Integer Ring
Echelon basis matrix:
[ 1  0 -3  2]
[ 0  1 -2  1]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(IntegerRing(),Integer(4),Integer(2))(range(Integer(8)))
>>> M.kernel()
Free module of degree 4 and rank 2 over Integer Ring
Echelon basis matrix:
[ 1  0 -3  2]
[ 0  1 -2  1]
```

A kernel of dimension one over $\mathbf{Q}$:

```
sage: A = MatrixSpace(RationalField(),3)(range(9))
sage: A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
```

(continues on next page)

(continued from previous page)

```
Basis matrix:
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(3))(range(Integer(9)))
>>> A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

A trivial kernel:

```
sage: A = MatrixSpace(RationalField(),2)([1,2,3,4])
sage: A.kernel()
Vector space of degree 2 and dimension 0 over Rational Field
Basis matrix:
[]
sage: M = MatrixSpace(RationalField(),0,2)(0)
sage: M
[]
sage: M.kernel()
Vector space of degree 0 and dimension 0 over Rational Field
Basis matrix:
[]
sage: M = MatrixSpace(RationalField(),2,0)(0)
sage: M.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(2))([Integer(1),Integer(2),Integer(3),
↪Integer(4)])
>>> A.kernel()
Vector space of degree 2 and dimension 0 over Rational Field
Basis matrix:
[]
>>> M = MatrixSpace(RationalField(),Integer(0),Integer(2))(Integer(0))
>>> M
[]
>>> M.kernel()
Vector space of degree 0 and dimension 0 over Rational Field
Basis matrix:
[]
>>> M = MatrixSpace(RationalField(),Integer(2),Integer(0))(Integer(0))
>>> M.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

Kernel of a zero matrix:

```
sage: A = MatrixSpace(RationalField(),2)(0)
sage: A.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(2))(Integer(0))
>>> A.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

Kernel of a non-square matrix:

```
sage: A = MatrixSpace(RationalField(),3,2)(range(6))
sage: A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(3),Integer(2))(range(Integer(6)))
>>> A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

The 2-dimensional kernel of a matrix over a cyclotomic field:

```
sage: K = CyclotomicField(12); a = K.gen()
sage: M = MatrixSpace(K,4,2)([1,-1, 0,-2, 0,-a^2-1, 0,a^2-1])
sage: M
[             1            -1]
[             0            -2]
[             0 -zeta12^2 - 1]
[             0  zeta12^2 - 1]
sage: M.kernel()
Vector space of degree 4 and dimension 2 over Cyclotomic Field of order 12
 and degree 4
Basis matrix:
[               0                1                0     -2*zeta12^2]
[               0                0                1 -2*zeta12^2 + 1]
```

```
>>> from sage.all import *
>>> K = CyclotomicField(Integer(12)); a = K.gen()
>>> M = MatrixSpace(K,Integer(4),Integer(2))([Integer(1),-Integer(1), Integer(0),-
↪Integer(2), Integer(0),-a**Integer(2)-Integer(1), Integer(0),a**Integer(2)-
↪Integer(1)])
```

(continues on next page)

(continued from previous page)

```
>>> M
[              1             -1]
[              0             -2]
[              0 -zeta12^2 - 1]
[              0  zeta12^2 - 1]
>>> M.kernel()
Vector space of degree 4 and dimension 2 over Cyclotomic Field of order 12
 and degree 4
Basis matrix:
[               0                1                0     -2*zeta12^2]
[               0                0                1 -2*zeta12^2 + 1]
```

A nontrivial kernel over a complicated base field.

```
sage: K = FractionField(PolynomialRing(RationalField(),2,'x'))
sage: M = MatrixSpace(K, 2)([[K.gen(1),K.gen(0)], [K.gen(1), K.gen(0)]])
sage: M
[x1 x0]
[x1 x0]
sage: M.kernel()
Vector space of degree 2 and dimension 1 over Fraction Field of Multivariate
Polynomial Ring in x0, x1 over Rational Field
Basis matrix:
 [ 1 -1]
```

```
>>> from sage.all import *
>>> K = FractionField(PolynomialRing(RationalField(),Integer(2),'x'))
>>> M = MatrixSpace(K, Integer(2))([[K.gen(Integer(1)),K.gen(Integer(0))], [K.
↪gen(Integer(1)), K.gen(Integer(0))]])
>>> M
[x1 x0]
[x1 x0]
>>> M.kernel()
Vector space of degree 2 and dimension 1 over Fraction Field of Multivariate
Polynomial Ring in x0, x1 over Rational Field
Basis matrix:
 [ 1 -1]
```

Other methods for integer matrices are `elementary_divisors`, `smith_form` (for the Smith normal form), `echelon_form` for the Hermite normal form, `frobenius` for the Frobenius normal form (rational canonical form).

There are many methods for matrices over a field such as **Q** or a finite field: `row_span`, `nullity`, `transpose`, `swap_rows`, `matrix_from_columns`, `matrix_from_rows`, among many others.

See the file `matrix.py` for further details.

4.5 Eigenvectors and eigenvalues

How do you compute eigenvalues and eigenvectors using Sage?

Sage has a full range of functions for computing eigenvalues and both left and right eigenvectors and eigenspaces. If our matrix is A, then the `eigenmatrix_right` (resp. `eightmatrix_left`) command also gives matrices D and P such that $AP = PD$ (resp. $PA = DP$.)

```
sage: A = matrix(QQ, [[1,1,0],[0,2,0],[0,0,3]])
sage: A
[1 1 0]
[0 2 0]
[0 0 3]
sage: A.eigenvalues()
[3, 2, 1]
sage: A.eigenvectors_right()
[(3, [(0, 0, 1)], 1), (2, [(1, 1, 0)], 1), (1, [(1, 0, 0)], 1)]

sage: A.eigenspaces_right()
[(3,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [0 0 1]),
 (2,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 0]),
 (1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 0 0])]

sage: D, P = A.eigenmatrix_right()
sage: D
[3 0 0]
[0 2 0]
[0 0 1]
sage: P
[0 1 1]
[0 1 0]
[1 0 0]
sage: A*P == P*D
True
```

```
>>> from sage.all import *
>>> A = matrix(QQ, [[Integer(1),Integer(1),Integer(0)],[Integer(0),Integer(2),
↪Integer(0)],[Integer(0),Integer(0),Integer(3)]])
>>> A
[1 1 0]
[0 2 0]
[0 0 3]
>>> A.eigenvalues()
[3, 2, 1]
>>> A.eigenvectors_right()
[(3, [(0, 0, 1)], 1), (2, [(1, 1, 0)], 1), (1, [(1, 0, 0)], 1)]

>>> A.eigenspaces_right()
[(3,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
```

(continues on next page)

(continued from previous page)

```
  [0 0 1]),
 (2,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 0]),
 (1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 0 0])]

>>> D, P = A.eigenmatrix_right()
>>> D
[3 0 0]
[0 2 0]
[0 0 1]
>>> P
[0 1 1]
[0 1 0]
[1 0 0]
>>> A*P == P*D
True
```

For eigenvalues outside the fraction field of the base ring of the matrix, you can choose to have all the eigenspaces output when the algebraic closure of the field is implemented, such as the algebraic numbers, `QQbar`. Or you may request just a single eigenspace for each irreducible factor of the characteristic polynomial, since the others may be formed through Galois conjugation. The eigenvalues of the matrix below are $pmsqrt{3}$ and we exhibit each possible output.

Also, currently Sage does not implement multiprecision numerical eigenvalues and eigenvectors, so calling the eigen functions on a matrix from `CC` or `RR` will probably give inaccurate and nonsensical results (a warning is also printed). Eigenvalues and eigenvectors of matrices with floating point entries (over `CDF` and `RDF`) can be obtained with the "eigenmatrix" commands.

```
sage: MS = MatrixSpace(QQ, 2, 2)
sage: A = MS([1,-4,1, -1])
sage: A.eigenspaces_left(format='all')
[(-1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                        1 -1 - 1.732050807568878?*I]),
 (1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                        1 -1 + 1.732050807568878?*I])]
sage: A.eigenspaces_left(format='galois')
[(a0,
  Vector space of degree 2 and dimension 1 over Number Field in a0 with defining␣
↪polynomial x^2 + 3
  User basis matrix:
  [     1 a0 - 1])]
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(QQ, Integer(2), Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> A = MS([Integer(1),-Integer(4),Integer(1), -Integer(1)])
>>> A.eigenspaces_left(format='all')
[(-1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                        1 -1 - 1.732050807568878?*I]),
 (1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                        1 -1 + 1.732050807568878?*I])]
>>> A.eigenspaces_left(format='galois')
[(a0,
  Vector space of degree 2 and dimension 1 over Number Field in a0 with defining␣
↪polynomial x^2 + 3
  User basis matrix:
  [     1 a0 - 1])]
```

Another approach is to use the interface with Maxima:

```
sage: A = maxima("matrix ([1, -4], [1, -1])")
sage: eig = A.eigenvectors()
sage: eig.sage()
[[[-I*sqrt(3), I*sqrt(3)], [1, 1]], [[[1, 1/4*I*sqrt(3) + 1/4]], [[1, -1/4*I*sqrt(3)␣
↪+ 1/4]]]]
```

```
>>> from sage.all import *
>>> A = maxima("matrix ([1, -4], [1, -1])")
>>> eig = A.eigenvectors()
>>> eig.sage()
[[[-I*sqrt(3), I*sqrt(3)], [1, 1]], [[[1, 1/4*I*sqrt(3) + 1/4]], [[1, -1/4*I*sqrt(3)␣
↪+ 1/4]]]]
```

This tells us that $\vec{v}_1 = [1, (\sqrt{3}i + 1)/4]$ is an eigenvector of $\lambda_1 = -\sqrt{3}i$ (which occurs with multiplicity one) and $\vec{v}_2 = [1, (-\sqrt{3}i + 1)/4]$ is an eigenvector of $\lambda_2 = \sqrt{3}i$ (which also occurs with multiplicity one).

Here are two more examples:

```
sage: A = maxima("matrix ([11, 0, 0], [1, 11, 0], [1, 3, 2])")
sage: A.eigenvectors()
[[[2,11],[1,2]],[[[0,0,1]],[[0,1,1/3]]]]
sage: A = maxima("matrix ([-1, 0, 0], [1, -1, 0], [1, 3, 2])")
sage: A.eigenvectors()
[[[-1,2],[2,1]],[[[0,1,-1]],[[0,0,1]]]]
```

```
>>> from sage.all import *
>>> A = maxima("matrix ([11, 0, 0], [1, 11, 0], [1, 3, 2])")
>>> A.eigenvectors()
[[[2,11],[1,2]],[[[0,0,1]],[[0,1,1/3]]]]
>>> A = maxima("matrix ([-1, 0, 0], [1, -1, 0], [1, 3, 2])")
>>> A.eigenvectors()
[[[-1,2],[2,1]],[[[0,1,-1]],[[0,0,1]]]]
```

Warning: Notice how the ordering of the output is reversed, though the matrices are almost the same.

Finally, you can use Sage's GAP interface as well to compute "rational" eigenvalues and eigenvectors:

```
sage: A = libgap([[1,2,3],[4,5,6],[7,8,9]]); A
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]
sage: libgap(QQ).Eigenvectors(A)
[ [ 1, -2, 1 ] ]
sage: libgap(QQ).Eigenvalues(A)
[ 0 ]
```

```
>>> from sage.all import *
>>> A = libgap([[Integer(1),Integer(2),Integer(3)],[Integer(4),Integer(5),Integer(6)],
↪[Integer(7),Integer(8),Integer(9)]]); A
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]
>>> libgap(QQ).Eigenvectors(A)
[ [ 1, -2, 1 ] ]
>>> libgap(QQ).Eigenvalues(A)
[ 0 ]
```

4.6 Row reduction

The row reduced echelon form of a matrix is computed as in the following example.

```
sage: M = MatrixSpace(RationalField(),2,3)
sage: A = M([1,2,3, 4,5,6])
sage: A
[1 2 3]
[4 5 6]
sage: A.parent()
Full MatrixSpace of 2 by 3 dense matrices over Rational Field
sage: A[0,2] = 389
sage: A
[  1   2 389]
[  4   5   6]
sage: A.echelon_form()
[      1       0 -1933/3]
[      0       1  1550/3]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(RationalField(),Integer(2),Integer(3))
>>> A = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6)])
>>> A
[1 2 3]
[4 5 6]
>>> A.parent()
Full MatrixSpace of 2 by 3 dense matrices over Rational Field
>>> A[Integer(0),Integer(2)] = Integer(389)
>>> A
[  1   2 389]
[  4   5   6]
>>> A.echelon_form()
[      1       0 -1933/3]
[      0       1  1550/3]
```

4.7 Characteristic polynomial

The characteristic polynomial is a Sage method for square matrices.

First a matrix over $\mathbf{Z}$:

```
sage: A = MatrixSpace(IntegerRing(),2)( [[1,2], [3,4]] )
sage: f = A.charpoly()
sage: f
x^2 - 5*x - 2
sage: f.parent()
Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> A = MatrixSpace(IntegerRing(),Integer(2))( [[Integer(1),Integer(2)], [Integer(3),
↪Integer(4)]] )
>>> f = A.charpoly()
>>> f
x^2 - 5*x - 2
>>> f.parent()
Univariate Polynomial Ring in x over Integer Ring
```

We compute the characteristic polynomial of a matrix over the polynomial ring $\mathbf{Z}[a]$:

```
sage: R = PolynomialRing(IntegerRing(),'a'); a = R.gen()
sage: M = MatrixSpace(R,2)([[a,1], [a,a+1]])
sage: M
[    a     1]
[    a a + 1]
sage: f = M.charpoly()
sage: f
x^2 + (-2*a - 1)*x + a^2
sage: f.parent()
Univariate Polynomial Ring in x over Univariate Polynomial Ring in a over
Integer Ring

sage: M.trace()
2*a + 1
sage: M.determinant()
a^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(IntegerRing(),'a'); a = R.gen()
>>> M = MatrixSpace(R,Integer(2))([[a,Integer(1)], [a,a+Integer(1)]])
>>> M
[    a     1]
[    a a + 1]
>>> f = M.charpoly()
>>> f
x^2 + (-2*a - 1)*x + a^2
>>> f.parent()
Univariate Polynomial Ring in x over Univariate Polynomial Ring in a over
Integer Ring
```

(continues on next page)

(continued from previous page)

```
>>> M.trace()
2*a + 1
>>> M.determinant()
a^2
```

We compute the characteristic polynomial of a matrix over the multi-variate polynomial ring $\mathbf{Z}[u, v]$:

```
sage: R.<u,v> = PolynomialRing(ZZ,2)
sage: A = MatrixSpace(R,2)([u,v,u^2,v^2])
sage: f = A.charpoly(); f
x^2 + (-v^2 - u)*x - u^2*v + u*v^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ,Integer(2), names=('u', 'v',)); (u, v,) = R._first_ngens(2)
>>> A = MatrixSpace(R,Integer(2))([u,v,u**Integer(2),v**Integer(2)])
>>> f = A.charpoly(); f
x^2 + (-v^2 - u)*x - u^2*v + u*v^2
```

It's a little difficult to distinguish the variables. To fix this, we might want to rename the indeterminate "Z", which we can easily do as follows:

```
sage: f = A.charpoly('Z'); f
Z^2 + (-v^2 - u)*Z - u^2*v + u*v^2
```

```
>>> from sage.all import *
>>> f = A.charpoly('Z'); f
Z^2 + (-v^2 - u)*Z - u^2*v + u*v^2
```

4.8 Solving systems of linear equations

Using maxima, you can easily solve linear equations:

```
sage: var('a,b,c')
(a, b, c)
sage: eqn = [a+b*c==1, b-a*c==0, a+b==5]
sage: s = solve(eqn, a,b,c); s
[[a == -1/4*I*sqrt(79) + 11/4, b == 1/4*I*sqrt(79) + 9/4, c == 1/10*I*sqrt(79) + 1/
↪10], [a == 1/4*I*sqrt(79) + 11/4, b == -1/4*I*sqrt(79) + 9/4, c == -1/10*I*sqrt(79)␣
↪+ 1/10]]
```

```
>>> from sage.all import *
>>> var('a,b,c')
(a, b, c)
>>> eqn = [a+b*c==Integer(1), b-a*c==Integer(0), a+b==Integer(5)]
>>> s = solve(eqn, a,b,c); s
[[a == -1/4*I*sqrt(79) + 11/4, b == 1/4*I*sqrt(79) + 9/4, c == 1/10*I*sqrt(79) + 1/
↪10], [a == 1/4*I*sqrt(79) + 11/4, b == -1/4*I*sqrt(79) + 9/4, c == -1/10*I*sqrt(79)␣
↪+ 1/10]]
```

You can even nicely typeset the solution in LaTeX:

```
sage.: print(latex(s))
...
```

To have the above appear onscreen via xdvi, type `view(s)`.

You can also solve linear equations symbolically using the `solve` command:

```
sage: var('x,y,z,a')
(x, y, z, a)
sage: eqns = [x + z == y, 2*a*x - y == 2*a^2, y - 2*z == 2]
sage: solve(eqns, x, y, z)
[[x == a + 1, y == 2*a, z == a - 1]]
```

```
>>> from sage.all import *
>>> var('x,y,z,a')
(x, y, z, a)
>>> eqns = [x + z == y, Integer(2)*a*x - y == Integer(2)*a**Integer(2), y -␣
↪Integer(2)*z == Integer(2)]
>>> solve(eqns, x, y, z)
[[x == a + 1, y == 2*a, z == a - 1]]
```

Here is a numerical Numpy example:

```
sage: from numpy import arange, eye, linalg
sage: A = eye(10)        ##   the 10x10 identity matrix
sage: b = arange(1,11)
sage: x = linalg.solve(A,b)
```

```
>>> from sage.all import *
>>> from numpy import arange, eye, linalg
>>> A = eye(Integer(10))        ##   the 10x10 identity matrix
>>> b = arange(Integer(1),Integer(11))
>>> x = linalg.solve(A,b)
```

Another way to solve a system numerically is to use Sage's octave interface:

```
sage: M33 = MatrixSpace(QQ,3,3)
sage: A   = M33([1,2,3,4,5,6,7,8,0])
sage: V3  = VectorSpace(QQ,3)
sage: b   = V3([1,2,3])
sage: octave.solve_linear_system(A,b)     # optional - octave
[-0.333333, 0.666667, 0]
```

```
>>> from sage.all import *
>>> M33 = MatrixSpace(QQ,Integer(3),Integer(3))
>>> A   = M33([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5),Integer(6),
↪Integer(7),Integer(8),Integer(0)])
>>> V3  = VectorSpace(QQ,Integer(3))
>>> b   = V3([Integer(1),Integer(2),Integer(3)])
>>> octave.solve_linear_system(A,b)     # optional - octave
[-0.333333, 0.666667, 0]
```

CHAPTER

FIVE

LINEAR CODES AND CIPHERS

5.1 Codes

A linear code of length n is a finite dimensional subspace of $GF(q)^n$. Sage can compute with linear error-correcting codes to a limited extent. It basically has some wrappers to GAP and GUAVA commands. GUAVA 2.8 is not included with Sage 4.0's install of GAP but can be installed as an optional package.

Sage can compute Hamming codes

```
sage: C = codes.HammingCode(GF(3), 3)
sage: C
[13, 10] Hamming Code over GF(3)
sage: C.minimum_distance()
3
sage: C.generator_matrix()
[1 0 0 0 0 0 0 0 0 0 1 2 0]
[0 1 0 0 0 0 0 0 0 0 0 1 2]
[0 0 1 0 0 0 0 0 0 0 1 0 2]
[0 0 0 1 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 0 0 0 0 1 1 2]
[0 0 0 0 0 1 0 0 0 0 2 0 2]
[0 0 0 0 0 0 1 0 0 0 1 2 1]
[0 0 0 0 0 0 0 1 0 0 2 1 1]
[0 0 0 0 0 0 0 0 1 0 2 2 0]
[0 0 0 0 0 0 0 0 0 1 0 1 1]
```

```
>>> from sage.all import *
>>> C = codes.HammingCode(GF(Integer(3)), Integer(3))
>>> C
[13, 10] Hamming Code over GF(3)
>>> C.minimum_distance()
3
>>> C.generator_matrix()
[1 0 0 0 0 0 0 0 0 0 1 2 0]
[0 1 0 0 0 0 0 0 0 0 0 1 2]
[0 0 1 0 0 0 0 0 0 0 1 0 2]
[0 0 0 1 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 0 0 0 0 1 1 2]
[0 0 0 0 0 1 0 0 0 0 2 0 2]
[0 0 0 0 0 0 1 0 0 0 1 2 1]
[0 0 0 0 0 0 0 1 0 0 2 1 1]
```

(continues on next page)

(continued from previous page)

```
[0 0 0 0 0 0 0 0 1 0 2 2 0]
[0 0 0 0 0 0 0 0 0 1 0 1 1]
```

the four Golay codes

```
sage: C = codes.GolayCode(GF(3))
sage: C
[12, 6, 6] Extended Golay code over GF(3)
sage: C.minimum_distance()
6
sage: C.generator_matrix()
[1 0 0 0 0 0 2 0 1 2 1 2]
[0 1 0 0 0 0 1 2 2 2 1 0]
[0 0 1 0 0 0 1 1 1 0 1 1]
[0 0 0 1 0 0 1 1 0 2 2 2]
[0 0 0 0 1 0 2 1 2 2 0 1]
[0 0 0 0 0 1 0 2 1 2 2 1]
```

```
>>> from sage.all import *
>>> C = codes.GolayCode(GF(Integer(3)))
>>> C
[12, 6, 6] Extended Golay code over GF(3)
>>> C.minimum_distance()
6
>>> C.generator_matrix()
[1 0 0 0 0 0 2 0 1 2 1 2]
[0 1 0 0 0 0 1 2 2 2 1 0]
[0 0 1 0 0 0 1 1 1 0 1 1]
[0 0 0 1 0 0 1 1 0 2 2 2]
[0 0 0 0 1 0 2 1 2 2 0 1]
[0 0 0 0 0 1 0 2 1 2 2 1]
```

as well as binary Reed-Muller codes, quadratic residue codes, quasi-quadratic residue codes, "random" linear codes, and a code generated by a matrix of full rank (using, as usual, the rows as the basis).

For a given code, C, Sage can return a generator matrix, a check matrix, and the dual code:

```
sage: C = codes.HammingCode(GF(2), 3)
sage: Cperp = C.dual_code()
sage: C; Cperp
[7, 4] Hamming Code over GF(2)
[7, 3] linear code over GF(2)
sage: C.generator_matrix()
  [1 0 0 0 0 1 1]
  [0 1 0 0 1 0 1]
  [0 0 1 0 1 1 0]
  [0 0 0 1 1 1 1]
sage: C.parity_check_matrix()
  [1 0 1 0 1 0 1]
  [0 1 1 0 0 1 1]
  [0 0 0 1 1 1 1]
sage: C.dual_code()
[7, 3] linear code over GF(2)
```

(continues on next page)

(continued from previous page)

```
sage: C = codes.HammingCode(GF(4,'a'), 3)
sage: C.dual_code()
[21, 3] linear code over GF(4)
```

```
>>> from sage.all import *
>>> C = codes.HammingCode(GF(Integer(2)), Integer(3))
>>> Cperp = C.dual_code()
>>> C; Cperp
[7, 4] Hamming Code over GF(2)
[7, 3] linear code over GF(2)
>>> C.generator_matrix()
  [1 0 0 0 0 1 1]
  [0 1 0 0 1 0 1]
  [0 0 1 0 1 1 0]
  [0 0 0 1 1 1 1]
>>> C.parity_check_matrix()
  [1 0 1 0 1 0 1]
  [0 1 1 0 0 1 1]
  [0 0 0 1 1 1 1]
>>> C.dual_code()
[7, 3] linear code over GF(2)
>>> C = codes.HammingCode(GF(Integer(4),'a'), Integer(3))
>>> C.dual_code()
[21, 3] linear code over GF(4)
```

For C and a vector $v \in GF(q)^n$, Sage can try to decode v (i.e., find the codeword $c \in C$ closest to v in the Hamming metric) using syndrome decoding. As of yet, no special decoding methods have been implemented.

```
sage: C = codes.HammingCode(GF(2), 3)
sage: MS = MatrixSpace(GF(2),1,7)
sage: F = GF(2); a = F.gen()
sage: v = vector([a,a,F(0),a,a,F(0),a])
sage: c = C.decode_to_code(v, "Syndrome"); c
(1, 1, 0, 1, 0, 0, 1)
sage: c in C
True
```

```
>>> from sage.all import *
>>> C = codes.HammingCode(GF(Integer(2)), Integer(3))
>>> MS = MatrixSpace(GF(Integer(2)),Integer(1),Integer(7))
>>> F = GF(Integer(2)); a = F.gen()
>>> v = vector([a,a,F(Integer(0)),a,a,F(Integer(0)),a])
>>> c = C.decode_to_code(v, "Syndrome"); c
(1, 1, 0, 1, 0, 0, 1)
>>> c in C
True
```

To plot the (histogram of) the weight distribution of a code, one can use the matplotlib package included with Sage:

```
sage: C = codes.HammingCode(GF(2), 4)
sage: C
[15, 11] Hamming Code over GF(2)
```

(continues on next page)

(continued from previous page)

```
sage: w = C.weight_distribution(); w
 [1, 0, 0, 35, 105, 168, 280, 435, 435, 280, 168, 105, 35, 0, 0, 1]
sage: J = range(len(w))
sage: W = IndexedSequence([ZZ(w[i]) for i in J],J)
sage: P = W.plot_histogram()
```

```
>>> from sage.all import *
>>> C = codes.HammingCode(GF(Integer(2)), Integer(4))
>>> C
[15, 11] Hamming Code over GF(2)
>>> w = C.weight_distribution(); w
 [1, 0, 0, 35, 105, 168, 280, 435, 435, 280, 168, 105, 35, 0, 0, 1]
>>> J = range(len(w))
>>> W = IndexedSequence([ZZ(w[i]) for i in J],J)
>>> P = W.plot_histogram()
```

Now type `show(P)` to view this.

There are several coding theory functions we are skipping entirely. Please see the reference manual or the file `coding/linear_codes.py` for examples.

Sage can also compute algebraic-geometric codes, called AG codes, via the Singular interface § sec:agcodes. One may also use the AG codes implemented in GUAVA via the Sage interface to GAP `gap_console()`. See the GUAVA manual for more details. {GUAVA}

5.2 Ciphers

5.2.1 LFSRs

A special type of stream cipher is implemented in Sage, namely, a linear feedback shift register (LFSR) sequence defined over a finite field. Stream ciphers have been used for a long time as a source of pseudo-random number generators. {linear feedback shift register}

S. Golomb {G} gives a list of three statistical properties a sequence of numbers $\mathbf{a} = \{a_n\}_{n=1}^{\infty}$, $a_n \in \{0, 1\}$, should display to be considered "random". Define the autocorrelation of $\mathbf{a}$ to be

$$C(k) = C(k, \mathbf{a}) = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} (-1)^{a_n + a_{n+k}}.$$

In the case where a is periodic with period P then this reduces to

Assume a is periodic with period P.

- balance: $|\sum_{n=1}^{P} (-1)^{a_n}| \leq 1$.
- low autocorrelation:

$$C(k) = \begin{cases} 1, & k = 0, \\ \epsilon, & k \neq 0. \end{cases}$$

  (For sequences satisfying these first two properties, it is known that $\epsilon = -1/P$ must hold.)

- proportional runs property: In each period, half the runs have length 1, one-fourth have length 2, etc. Moveover, there are as many runs of 1's as there are of 0's.

A sequence satisfying these properties will be called pseudo-random. {pseudo-random}

A general feedback shift register is a map $f : \mathbf{F}_q^d \to \mathbf{F}_q^d$ of the form

$$f(x_0, ..., x_{n-1}) = (x_1, x_2, ..., x_n),$$
$$x_n = C(x_0, ..., x_{n-1}),$$

where $C : \mathbf{F}_q^d \to \mathbf{F}_q$ is a given function. When C is of the form

$$C(x_0, ..., x_{n-1}) = c_0 x_0 + ... + c_{n-1} x_{n-1},$$

for some given constants $c_i \in \mathbf{F}_q$, the map is called a linear feedback shift register (LFSR). The sequence of coefficients c_i is called the key and the polynomial

$$C(x) = 1 + c_0 x + ... + c_{n-1} x^n$$

is sometimes called the connection polynomial.

Example: Over $GF(2)$, if $[c_0, c_1, c_2, c_3] = [1, 0, 0, 1]$ then $C(x) = 1 + x + x^4$,

The LFSR sequence is then

$$1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1,$$
$$1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, ...$$

The sequence of $0, 1$'s is periodic with period $P = 2^4 - 1 = 15$ and satisfies Golomb's three randomness conditions. However, this sequence of period 15 can be "cracked" (i.e., a procedure to reproduce $g(x)$) by knowing only 8 terms! This is the function of the Berlekamp-Massey algorithm {M}, implemented as `lfsr_connection_polynomial` (which produces the reverse of `berlekamp_massey`).

```
sage: F = GF(2)
sage: o = F(0)
sage: l = F(1)
sage: key = [l,o,o,l]; fill = [l,l,o,l]; n = 20
sage: s = lfsr_sequence(key,fill,n); s
[1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0]
sage: lfsr_autocorrelation(s,15,7)
4/15
sage: lfsr_autocorrelation(s,15,0)
8/15
sage: lfsr_connection_polynomial(s)
x^4 + x + 1
sage: from sage.matrix.berlekamp_massey import berlekamp_massey
sage: berlekamp_massey(s)
x^4 + x^3 + 1
```

```
>>> from sage.all import *
>>> F = GF(Integer(2))
>>> o = F(Integer(0))
>>> l = F(Integer(1))
>>> key = [l,o,o,l]; fill = [l,l,o,l]; n = Integer(20)
>>> s = lfsr_sequence(key,fill,n); s
[1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0]
>>> lfsr_autocorrelation(s,Integer(15),Integer(7))
4/15
>>> lfsr_autocorrelation(s,Integer(15),Integer(0))
8/15
```

(continues on next page)

(continued from previous page)

```
>>> lfsr_connection_polynomial(s)
x^4 + x + 1
>>> from sage.matrix.berlekamp_massey import berlekamp_massey
>>> berlekamp_massey(s)
x^4 + x^3 + 1
```

5.2.2 Classical ciphers

has a type for cryptosystems (created by David Kohel, who also wrote the examples below), implementing classical cryptosystems. The general interface is as follows:

```
sage: S = AlphabeticStrings()
sage: S
Free alphabetic string monoid on A-Z
sage: E = SubstitutionCryptosystem(S)
sage: E
Substitution cryptosystem on Free alphabetic string monoid on A-Z
sage: K = S([ 25-i for i in range(26) ])
sage: e = E(K)
sage: m = S("THECATINTHEHAT")
sage: e(m)
GSVXZGRMGSVSZG
```

```
>>> from sage.all import *
>>> S = AlphabeticStrings()
>>> S
Free alphabetic string monoid on A-Z
>>> E = SubstitutionCryptosystem(S)
>>> E
Substitution cryptosystem on Free alphabetic string monoid on A-Z
>>> K = S([ Integer(25)-i for i in range(Integer(26)) ])
>>> e = E(K)
>>> m = S("THECATINTHEHAT")
>>> e(m)
GSVXZGRMGSVSZG
```

Here's another example:

```
sage: S = AlphabeticStrings()
sage: E = TranspositionCryptosystem(S,15);
sage: m = S("THECATANDTHEHAT")
sage: G = E.key_space()
sage: G
Symmetric group of order 15! as a permutation group
sage: g = G([ 3, 2, 1, 6, 5, 4, 9, 8, 7, 12, 11, 10, 15, 14, 13 ])
sage: e = E(g)
sage: e(m)
EHTTACDNAEHTTAH
```

```
>>> from sage.all import *
>>> S = AlphabeticStrings()
```

(continues on next page)

(continued from previous page)

```
>>> E = TranspositionCryptosystem(S,Integer(15));
>>> m = S("THECATANDTHEHAT")
>>> G = E.key_space()
>>> G
Symmetric group of order 15! as a permutation group
>>> g = G([ Integer(3), Integer(2), Integer(1), Integer(6), Integer(5), Integer(4),␣
↪Integer(9), Integer(8), Integer(7), Integer(12), Integer(11), Integer(10),␣
↪Integer(15), Integer(14), Integer(13) ])
>>> e = E(g)
>>> e(m)
EHTTACDNAEHTTAH
```

The idea is that a cryptosystem is a map $E : KS \to \mathrm{Hom}_{\mathrm{Set}}(MS, CS)$ where KS, MS, and CS are the key space, plaintext (or message) space, and ciphertext space, respectively. E is presumed to be injective, so `e.key()` returns the pre-image key.

CHAPTER

SIX

REPRESENTATION THEORY

6.1 Ordinary characters

How can you compute character tables of a finite group in Sage? The Sage-GAP interface can be used to compute character tables.

You can construct the table of character values of a permutation group G as a Sage matrix, using the method `character_table` of the PermutationGroup class, or via the interface to the GAP command `CharacterTable`.

```
sage: G = PermutationGroup([[(1,2),(3,4)], [(1,2,3,4)]])
sage: G.order()
8
sage: G.character_table() # random
[ 1  1  1  1  1]
[ 1 -1 -1  1  1]
[ 1 -1  1 -1  1]
[ 1  1 -1 -1  1]
[ 2  0  0  0 -2]
sage: CT = libgap(G).CharacterTable()
sage: CT.Display() # random
CT1

 2  3  2  2  2  3

   1a 2a 2b 4a 2c
2P 1a 1a 1a 2c 1a
3P 1a 2a 2b 4a 2c

X.1     1  1  1  1  1
X.2     1 -1 -1  1  1
X.3     1 -1  1 -1  1
X.4     1  1 -1 -1  1
X.5     2  .  .  . -2
```

```
>>> from sage.all import *
>>> G = PermutationGroup([[(Integer(1),Integer(2)),(Integer(3),Integer(4))],␣
↪[(Integer(1),Integer(2),Integer(3),Integer(4))]])
>>> G.order()
8
>>> G.character_table() # random
[ 1  1  1  1  1]
```

(continues on next page)

(continued from previous page)

```
[ 1 -1 -1  1  1]
[ 1 -1  1 -1  1]
[ 1  1 -1 -1  1]
[ 2  0  0  0 -2]
>>> CT = libgap(G).CharacterTable()
>>> CT.Display() # random
CT1
<BLANKLINE>
 2  3  2  2  2  3
<BLANKLINE>
   1a 2a 2b 4a 2c
2P 1a 1a 1a 2c 1a
3P 1a 2a 2b 4a 2c
<BLANKLINE>
X.1     1  1  1  1  1
X.2     1 -1 -1  1  1
X.3     1 -1  1 -1  1
X.4     1  1 -1 -1  1
X.5     2  .  .  . -2
```

Here is another example:

```
sage: G = PermutationGroup([[(1,2),(3,4)], [(1,2,3)]])
sage: G.character_table() # random
[         1          1          1          1]
[         1 -zeta3 - 1      zeta3          1]
[         1      zeta3 -zeta3 - 1          1]
[         3          0          0         -1]
sage: G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: T = G.CharacterTable()
sage: T.Display() # random
CT2

     2  2  .  .  2
     3  1  1  1  .

       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a

X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

```
>>> from sage.all import *
>>> G = PermutationGroup([[(Integer(1),Integer(2)),(Integer(3),Integer(4))],␣
↪[(Integer(1),Integer(2),Integer(3))]])
```

(continues on next page)

(continued from previous page)

```
>>> G.character_table() # random
[         1          1          1          1]
[         1 -zeta3 - 1      zeta3          1]
[         1      zeta3 -zeta3 - 1          1]
[         3          0          0         -1]
>>> G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> T = G.CharacterTable()
>>> T.Display() # random
CT2
<BLANKLINE>
     2  2  .  .  2
     3  1  1  1  .
<BLANKLINE>
       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a
<BLANKLINE>
X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

where $E(3)$ denotes a cube root of unity, $ER(-3)$ denotes a square root of -3, say $i\sqrt{3}$, and $b3 = \frac{1}{2}(-1+i\sqrt{3})$. Note the added `print` Python command. This makes the output look much nicer.

```
sage: irr = G.Irr(); sorted(irr)
[Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3), E(3)^2, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, 0, 0, -1 ] )]
sage: irr.Display() # random
[ [       1,       1,       1,       1 ],
  [       1,  E(3)^2,    E(3),       1 ],
  [       1,    E(3),  E(3)^2,       1 ],
  [       3,       0,       0,      -1 ] ]
sage: CG = G.ConjugacyClasses(); CG
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
sage: gamma = CG[2]; gamma
(2,4,3)^G
sage: g = gamma.Representative(); g
(2,4,3)
sage: chi = irr[1]; chi # random
Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] )
sage: g^chi # random
E(3)
```

```
>>> from sage.all import *
>>> irr = G.Irr(); sorted(irr)
```

(continues on next page)

(continued from previous page)

```
[Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3), E(3)^2, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, 0, 0, -1 ] )]
>>> irr.Display() # random
[ [       1,       1,       1,       1 ],
  [       1,  E(3)^2,    E(3),       1 ],
  [       1,    E(3),  E(3)^2,       1 ],
  [       3,       0,       0,      -1 ] ]
>>> CG = G.ConjugacyClasses(); CG
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
>>> gamma = CG[Integer(2)]; gamma
(2,4,3)^G
>>> g = gamma.Representative(); g
(2,4,3)
>>> chi = irr[Integer(1)]; chi # random
Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] )
>>> g**chi # random
E(3)
```

This last quantity is the value of the character `chi` at the group element `g`.

Alternatively, if you turn IPython "pretty printing" off, then the table prints nicely.

```
sage: %Pprint
Pretty printing has been turned OFF
sage: G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: T = G.CharacterTable(); T
CharacterTable( Alt( [ 1 .. 4 ] ) )
sage: T.Display()
CT3

     2  2  2  .  .
     3  1  .  1  1

       1a 2a 3a 3b
    2P 1a 1a 3b 3a
    3P 1a 2a 1a 1a

X.1     1  1  1  1
X.2     1  1  A /A
X.3     1  1 /A  A
X.4     3 -1  .  .

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
sage: irr = G.Irr(); irr
[ Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3)^2, E(3) ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3), E(3)^2 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, -1, 0, 0 ] ) ]
sage: irr.Display()
```

(continues on next page)

(continued from previous page)

```
[ [       1,       1,       1,       1 ],
  [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       3,      -1,       0,       0 ] ]
sage: %Pprint
Pretty printing has been turned ON
```

```
>>> from sage.all import *
>>> %Pprint
Pretty printing has been turned OFF
>>> G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> T = G.CharacterTable(); T
CharacterTable( Alt( [ 1 .. 4 ] ) )
>>> T.Display()
CT3
<BLANKLINE>
     2  2  2  .  .
     3  1  .  1  1
<BLANKLINE>
       1a 2a 3a 3b
    2P 1a 1a 3b 3a
    3P 1a 2a 1a 1a
<BLANKLINE>
X.1     1  1  1  1
X.2     1  1  A /A
X.3     1  1 /A  A
X.4     3 -1  .  .
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
>>> irr = G.Irr(); irr
[ Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3)^2, E(3) ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3), E(3)^2 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, -1, 0, 0 ] ) ]
>>> irr.Display()
[ [       1,       1,       1,       1 ],
  [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       3,      -1,       0,       0 ] ]
>>> %Pprint
Pretty printing has been turned ON
```

6.2 Brauer characters

The Brauer character tables in GAP do not yet have a "native" interface. To access them you can directly interface with GAP using the `libgap.eval` command.

The example below using the GAP interface illustrates the syntax.

```
sage: G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: irr = G.IrreducibleRepresentations(GF(7)); irr   # random arch. dependent output
[ [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^4 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^2 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^0 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] ->
    [ [ [ Z(7)^2, Z(7)^5, Z(7) ], [ Z(7)^3, Z(7)^2, Z(7)^3 ],
        [ Z(7), Z(7)^5, Z(7)^2 ] ],
      [ [ 0*Z(7), Z(7)^0, 0*Z(7) ], [ 0*Z(7), 0*Z(7), Z(7)^0 ],
        [ Z(7)^0, 0*Z(7), 0*Z(7) ] ] ] ]
sage: brvals = [[chi.Image(c.Representative()).BrauerCharacterValue()
....:            for c in G.ConjugacyClasses()] for chi in irr]
sage: brvals          # random architecture dependent output
[ [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       1,       1,       1,       1 ],
  [       3,      -1,       0,       0 ] ]
sage: T = G.CharacterTable()
sage: T.Display() # random
CT3

     2  2  .  .  2
     3  1  1  1  .

       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a

X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

```
>>> from sage.all import *
>>> G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> irr = G.IrreducibleRepresentations(GF(Integer(7))); irr   # random arch.␣
↪dependent output
[ [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^4 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^2 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^0 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] ->
    [ [ [ Z(7)^2, Z(7)^5, Z(7) ], [ Z(7)^3, Z(7)^2, Z(7)^3 ],
        [ Z(7), Z(7)^5, Z(7)^2 ] ],
      [ [ 0*Z(7), Z(7)^0, 0*Z(7) ], [ 0*Z(7), 0*Z(7), Z(7)^0 ],
        [ Z(7)^0, 0*Z(7), 0*Z(7) ] ] ] ]
>>> brvals = [[chi.Image(c.Representative()).BrauerCharacterValue()
...            for c in G.ConjugacyClasses()] for chi in irr]
```

(continues on next page)

(continued from previous page)

```
>>> brvals          # random architecture dependent output
[ [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       1,       1,       1,       1 ],
  [       3,      -1,       0,       0 ] ]
>>> T = G.CharacterTable()
>>> T.Display() # random
CT3
<BLANKLINE>
     2  2  .  .  2
     3  1  1  1  .
<BLANKLINE>
       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a
<BLANKLINE>
X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

CHAPTER

SEVEN

RINGS

7.1 Matrix rings

How do you construct a matrix ring over a finite ring in Sage? The `MatrixSpace` constructor accepts any ring as a base ring. Here's an example of the syntax:

```
sage: R = IntegerModRing(51)
sage: M = MatrixSpace(R,3,3)
sage: M(0)
[0 0 0]
[0 0 0]
[0 0 0]
sage: M(1)
[1 0 0]
[0 1 0]
[0 0 1]
sage: 5*M(1)
[5 0 0]
[0 5 0]
[0 0 5]
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(51))
>>> M = MatrixSpace(R,Integer(3),Integer(3))
>>> M(Integer(0))
[0 0 0]
[0 0 0]
[0 0 0]
>>> M(Integer(1))
[1 0 0]
[0 1 0]
[0 0 1]
>>> Integer(5)*M(Integer(1))
[5 0 0]
[0 5 0]
[0 0 5]
```

7.2 Polynomial rings

How do you construct a polynomial ring over a finite field in Sage? Here's an example:

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: f = x^2+7
sage: f in R
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> f = x**Integer(2)+Integer(7)
>>> f in R
True
```

Here's an example using the Singular interface:

```
sage: R = singular.ring(97, '(a,b,c,d)', 'lp')
sage: I = singular.ideal(['a+b+c+d', 'ab+ad+bc+cd', 'abc+abd+acd+bcd', 'abcd-1'])
sage: R
polynomial ring, over a field, global ordering
// coefficients: ZZ/97...
// number of vars : 4
//        block   1 : ordering lp
//                  : names    a b c d
//        block   2 : ordering C
sage: I
a+b+c+d,
a*b+a*d+b*c+c*d,
a*b*c+a*b*d+a*c*d+b*c*d,
a*b*c*d-1
```

```
>>> from sage.all import *
>>> R = singular.ring(Integer(97), '(a,b,c,d)', 'lp')
>>> I = singular.ideal(['a+b+c+d', 'ab+ad+bc+cd', 'abc+abd+acd+bcd', 'abcd-1'])
>>> R
polynomial ring, over a field, global ordering
// coefficients: ZZ/97...
// number of vars : 4
//        block   1 : ordering lp
//                  : names    a b c d
//        block   2 : ordering C
>>> I
a+b+c+d,
a*b+a*d+b*c+c*d,
a*b*c+a*b*d+a*c*d+b*c*d,
a*b*c*d-1
```

Here is another approach using GAP:

```
sage: R = libgap.PolynomialRing(GF(97), 4); R
```

(continues on next page)

(continued from previous page)

```
GF(97)[x_1,x_2,x_3,x_4]
sage: I = R.IndeterminatesOfPolynomialRing(); I
[ x_1, x_2, x_3, x_4 ]
sage: x1, x2, x3, x4 = I
sage: f = x1*x2 + x3; f
x_1*x_2+x_3
sage: f.Value(I,[1,1,1,1])
Z(97)^34
```

```
>>> from sage.all import *
>>> R = libgap.PolynomialRing(GF(Integer(97)), Integer(4)); R
GF(97)[x_1,x_2,x_3,x_4]
>>> I = R.IndeterminatesOfPolynomialRing(); I
[ x_1, x_2, x_3, x_4 ]
>>> x1, x2, x3, x4 = I
>>> f = x1*x2 + x3; f
x_1*x_2+x_3
>>> f.Value(I,[Integer(1),Integer(1),Integer(1),Integer(1)])
Z(97)^34
```

7.3 p-adic numbers

How do you construct p-adics in Sage? A great deal of progress has been made on this (see SageDays talks by David Harvey and David Roe). Here only a few of the simplest examples are given.

To compute the characteristic and residue class field of the ring `Zp` of integers of `Qp`, use the syntax illustrated by the following examples.

```
sage: K = Qp(3)
sage: K.residue_class_field()
Finite Field of size 3
sage: K.residue_characteristic()
3
sage: a = K(1); a
1 + O(3^20)
sage: 82*a
1 + 3^4 + O(3^20)
sage: 12*a
3 + 3^2 + O(3^21)
sage: a in K
True
sage: b = 82*a
sage: b^4
1 + 3^4 + 3^5 + 2*3^9 + 3^12 + 3^13 + 3^16 + O(3^20)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(3))
>>> K.residue_class_field()
Finite Field of size 3
>>> K.residue_characteristic()
3
```

(continues on next page)

(continued from previous page)

```
>>> a = K(Integer(1)); a
1 + O(3^20)
>>> Integer(82)*a
1 + 3^4 + O(3^20)
>>> Integer(12)*a
3 + 3^2 + O(3^21)
>>> a in K
True
>>> b = Integer(82)*a
>>> b**Integer(4)
1 + 3^4 + 3^5 + 2*3^9 + 3^12 + 3^13 + 3^16 + O(3^20)
```

7.4 Quotient rings of polynomials

How do you construct a quotient ring in Sage?

We create the quotient ring $GF(97)[x]/(x^3+7)$, and demonstrate many basic functions with it.

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: S = R.quotient(x^3 + 7, 'a')
sage: a = S.gen()
sage: S
Univariate Quotient Polynomial Ring in a over Finite Field of size 97 with
modulus x^3 + 7
sage: S.is_field()
True
sage: a in S
True
sage: x in S
True
sage: S.polynomial_ring()
Univariate Polynomial Ring in x over Finite Field of size 97
sage: S.modulus()
x^3 + 7
sage: S.degree()
3
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> S = R.quotient(x**Integer(3) + Integer(7), 'a')
>>> a = S.gen()
>>> S
Univariate Quotient Polynomial Ring in a over Finite Field of size 97 with
modulus x^3 + 7
>>> S.is_field()
True
>>> a in S
True
>>> x in S
```

(continues on next page)

(continued from previous page)

```
True
>>> S.polynomial_ring()
Univariate Polynomial Ring in x over Finite Field of size 97
>>> S.modulus()
x^3 + 7
>>> S.degree()
3
```

In Sage, `in` means that there is a "canonical coercion" into the ring. So the integer x and a are both in S, although x really needs to be coerced.

You can also compute in quotient rings without actually computing then using the command `quo_rem` as follows.

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: f = x^7+1
sage: (f^3).quo_rem(x^7-1)
(x^14 + 4*x^7 + 7, 8)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> f = x**Integer(7)+Integer(1)
>>> (f**Integer(3)).quo_rem(x**Integer(7)-Integer(1))
(x^14 + 4*x^7 + 7, 8)
```

CHAPTER

EIGHT

POLYNOMIALS

8.1 Polynomial powers

How do I compute modular polynomial powers in Sage?

To compute $x^{2006} \pmod{x^3+7}$ in $GF(97)[x]$, we create the quotient ring $GF(97)[x]/(x^3+7)$, and compute x^{2006} in it. As a matter of Sage notation, we must distinguish between the x in $GF(97)[x]$ and the corresponding element (which we denote by a) in the quotient ring $GF(97)[x]/(x^3+7)$.

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: S = R.quotient(x^3 + 7, 'a')
sage: a = S.gen()
sage: S
Univariate Quotient Polynomial Ring in a over
Finite Field of size 97 with modulus x^3 + 7
sage: a^2006
4*a^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> S = R.quotient(x**Integer(3) + Integer(7), 'a')
>>> a = S.gen()
>>> S
Univariate Quotient Polynomial Ring in a over
Finite Field of size 97 with modulus x^3 + 7
>>> a**Integer(2006)
4*a^2
```

Another approach to this:

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: S = R.quotient(x^3 + 7, 'a')
sage: a = S.gen()
sage: a^20062006
80*a
sage: libgap.eval("R:= PolynomialRing( GF(97))")
GF(97)[x_1]
sage: libgap.eval("i:= IndeterminatesOfPolynomialRing(R)")
```

(continues on next page)

(continued from previous page)

```
[ x_1 ]
sage: libgap.eval("x:= i[1]"); libgap.eval("f:= x;")
x_1
x_1
sage: libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
sage: libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
sage: libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
sage: a^2006200620062006
43*a^2
sage: libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
sage: libgap.eval("Int(Z(97)^4)")
43
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> S = R.quotient(x**Integer(3) + Integer(7), 'a')
>>> a = S.gen()
>>> a**Integer(20062006)
80*a
>>> libgap.eval("R:= PolynomialRing( GF(97))")
GF(97)[x_1]
>>> libgap.eval("i:= IndeterminatesOfPolynomialRing(R)")
[ x_1 ]
>>> libgap.eval("x:= i[1]"); libgap.eval("f:= x;")
x_1
x_1
>>> libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
>>> libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
>>> libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
>>> a**Integer(2006200620062006)
43*a^2
>>> libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
>>> libgap.eval("Int(Z(97)^4)")
43
```

8.2 Factorization

You can factor a polynomial using Sage.

Using Sage to factor a univariate polynomial is a matter of applying the method `factor` to the PolynomialRingElement object f. In fact, this method actually calls Pari, so the computation is fairly fast.

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = (x^3 - 1)^2-(x^2-1)^2
sage: f.factor()
(x - 1)^2 * x^2 * (x^2 + 2*x + 2)
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = (x**Integer(3) - Integer(1))**Integer(2)-(x**Integer(2)-
↪Integer(1))**Integer(2)
>>> f.factor()
(x - 1)^2 * x^2 * (x^2 + 2*x + 2)
```

Using the Singular interface, Sage also factors multivariate polynomials.

```
sage: x, y = PolynomialRing(RationalField(), 2, ['x','y']).gens()
sage: f =  (9*y^6 - 9*x^2*y^5 - 18*x^3*y^4 - 9*x^5*y^4 + 9*x^6*y^2 + 9*x^7*y^3
....:     + 18*x^8*y^2 - 9*x^11)
sage: f.factor()
(9) * (-x^5 + y^2) * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

```
>>> from sage.all import *
>>> x, y = PolynomialRing(RationalField(), Integer(2), ['x','y']).gens()
>>> f =  (Integer(9)*y**Integer(6) - Integer(9)*x**Integer(2)*y**Integer(5) -␣
↪Integer(18)*x**Integer(3)*y**Integer(4) - Integer(9)*x**Integer(5)*y**Integer(4) +␣
↪Integer(9)*x**Integer(6)*y**Integer(2) + Integer(9)*x**Integer(7)*y**Integer(3)
...     + Integer(18)*x**Integer(8)*y**Integer(2) - Integer(9)*x**Integer(11))
>>> f.factor()
(9) * (-x^5 + y^2) * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

8.3 Polynomial GCD's

This example illustrates single variable polynomial GCD's:

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = 3*x^3 + x
sage: g = 9*x*(x+1)
sage: f.gcd(g)
x
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = Integer(3)*x**Integer(3) + x
>>> g = Integer(9)*x*(x+Integer(1))
>>> f.gcd(g)
x
```

This example illustrates multivariate polynomial GCD's:

```
sage: R.<x,y,z> = PolynomialRing(RationalField(), order='lex')
sage: f = 3*x^2*(x+y)
sage: g = 9*x*(y^2 - x^2)
```

(continues on next page)

(continued from previous page)

```
sage: f.gcd(g)
x^2 + x*y
```

```
>>> from sage.all import *
>>> R = PolynomialRing(RationalField(), order='lex', names=('x', 'y', 'z',)); (x, y,␣
↪z,) = R._first_ngens(3)
>>> f = Integer(3)*x**Integer(2)*(x+y)
>>> g = Integer(9)*x*(y**Integer(2) - x**Integer(2))
>>> f.gcd(g)
x^2 + x*y
```

Here's another way to do this:

```
sage: R2 = singular.ring(0, '(x,y,z)', 'lp')
sage: a = singular.new('3x2*(x+y)')
sage: b = singular.new('9x*(y2-x2)')
sage: g = a.gcd(b)
sage: g
x^2+x*y
```

```
>>> from sage.all import *
>>> R2 = singular.ring(Integer(0), '(x,y,z)', 'lp')
>>> a = singular.new('3x2*(x+y)')
>>> b = singular.new('9x*(y2-x2)')
>>> g = a.gcd(b)
>>> g
x^2+x*y
```

This example illustrates univariate polynomial GCD's via the GAP interface.

```
sage: R = libgap.PolynomialRing(GF(2)); R
GF(2)[x_1]
sage: i = R.IndeterminatesOfPolynomialRing(); i
[ x_1 ]
sage: x_1 = i[0]
sage: f = (x_1^3 - x_1 + 1)*(x_1 + x_1^2); f
x_1^5+x_1^4+x_1^3+x_1
sage: g = (x_1^3 - x_1 + 1)*(x_1 + 1); g
x_1^4+x_1^3+x_1^2+Z(2)^0
sage: f.Gcd(g)
x_1^4+x_1^3+x_1^2+Z(2)^0
```

```
>>> from sage.all import *
>>> R = libgap.PolynomialRing(GF(Integer(2))); R
GF(2)[x_1]
>>> i = R.IndeterminatesOfPolynomialRing(); i
[ x_1 ]
>>> x_1 = i[Integer(0)]
>>> f = (x_1**Integer(3) - x_1 + Integer(1))*(x_1 + x_1**Integer(2)); f
x_1^5+x_1^4+x_1^3+x_1
>>> g = (x_1**Integer(3) - x_1 + Integer(1))*(x_1 + Integer(1)); g
x_1^4+x_1^3+x_1^2+Z(2)^0
```

(continues on next page)

(continued from previous page)

```
>>> f.Gcd(g)
x_1^4+x_1^3+x_1^2+Z(2)^0
```

We can, of course, do the same computation in , which uses the NTL library (which does huge polynomial gcd's over finite fields very quickly).

```
sage: x = PolynomialRing(GF(2), 'x').gen()
sage: f = (x^3 - x + 1)*(x + x^2); f
x^5 + x^4 + x^3 + x
sage: g = (x^3 - x + 1)*(x + 1)
sage: f.gcd(g)
x^4 + x^3 + x^2 + 1
```

```
>>> from sage.all import *
>>> x = PolynomialRing(GF(Integer(2)), 'x').gen()
>>> f = (x**Integer(3) - x + Integer(1))*(x + x**Integer(2)); f
x^5 + x^4 + x^3 + x
>>> g = (x**Integer(3) - x + Integer(1))*(x + Integer(1))
>>> f.gcd(g)
x^4 + x^3 + x^2 + 1
```

8.4 Roots of polynomials

Sage can compute roots of a univariant polynomial.

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = x^3 - 1
sage: f.roots()
[(1, 1)]
sage: f = (x^3 - 1)^2
sage: f.roots()
[(1, 2)]
sage: x = PolynomialRing(CyclotomicField(3), 'x').gen()
sage: f = x^3 - 1
sage: f.roots()
[(1, 1), (zeta3, 1), (-zeta3 - 1, 1)]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = x**Integer(3) - Integer(1)
>>> f.roots()
[(1, 1)]
>>> f = (x**Integer(3) - Integer(1))**Integer(2)
>>> f.roots()
[(1, 2)]
>>> x = PolynomialRing(CyclotomicField(Integer(3)), 'x').gen()
>>> f = x**Integer(3) - Integer(1)
>>> f.roots()
[(1, 1), (zeta3, 1), (-zeta3 - 1, 1)]
```

The first of the pair is the root, the second of the pair is its multiplicity.

There are some situations where GAP does find the roots of a univariate polynomial but GAP does not do this generally. (The roots must generate either a finite field or a subfield of a cyclotomic field.) However, there is a GAP package called `RadiRoot`, which must be installed into 's installation of GAP, which does help to do this for polynomials with rational coefficients (`radiroot` itself requires other packages to be installed; please see its webpage for more details). The `Factors` command actually has an option which allows you to increase the groundfield so that a factorization actually returns the roots. Please see the examples given in section 64.10 "Polynomial Factorization" of the GAP Reference Manual for more details.

8.5 Evaluation of multivariate functions

You can evaluate polynomials in Sage as usual by substituting in points:

```
sage: x = PolynomialRing(RationalField(), 3, 'x').gens()
sage: f = x[0] + x[1] - 2*x[1]*x[2]
sage: f
-2*x1*x2 + x0 + x1
sage: f(1,2,0)
3
sage: f(1,2,5)
-17
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), Integer(3), 'x').gens()
>>> f = x[Integer(0)] + x[Integer(1)] - Integer(2)*x[Integer(1)]*x[Integer(2)]
>>> f
-2*x1*x2 + x0 + x1
>>> f(Integer(1),Integer(2),Integer(0))
3
>>> f(Integer(1),Integer(2),Integer(5))
-17
```

This also will work with rational functions:

```
sage: h = f /(x[1] + x[2])
sage: h
(-2*x1*x2 + x0 + x1)/(x1 + x2)
sage: h(1,2,3)
-9/5
```

```
>>> from sage.all import *
>>> h = f /(x[Integer(1)] + x[Integer(2)])
>>> h
(-2*x1*x2 + x0 + x1)/(x1 + x2)
>>> h(Integer(1),Integer(2),Integer(3))
-9/5
```

Sage also performs symbolic manipulation:

```
sage: var('x,y,z')
(x, y, z)
sage: f = (x + 3*y + x^2*y)^3; f
(x^2*y + x + 3*y)^3
```

(continues on next page)

(continued from previous page)

```
sage: f(x=1,y=2,z=3)
729
sage: f.expand()
x^6*y^3 + 3*x^5*y^2 + 9*x^4*y^3 + 3*x^4*y + 18*x^3*y^2 +
27*x^2*y^3 +
x^3 + 9*x^2*y + 27*x*y^2 + 27*y^3
sage: f(x = 5/z)
(3*y + 25*y/z^2 + 5/z)^3
sage: g = f.subs(x = 5/z); g
(3*y + 25*y/z^2 + 5/z)^3
sage: h = g.rational_simplify(); h
(27*y^3*z^6 + 135*y^2*z^5 + 225*(3*y^3 + y)*z^4 + 125*(18*y^2 + 1)*z^3 +
15625*y^3 + 9375*y^2*z + 1875*(3*y^3 + y)*z^2)/z^6
```

```
>>> from sage.all import *
>>> var('x,y,z')
(x, y, z)
>>> f = (x + Integer(3)*y + x**Integer(2)*y)**Integer(3); f
(x^2*y + x + 3*y)^3
>>> f(x=Integer(1),y=Integer(2),z=Integer(3))
729
>>> f.expand()
x^6*y^3 + 3*x^5*y^2 + 9*x^4*y^3 + 3*x^4*y + 18*x^3*y^2 +
27*x^2*y^3 +
x^3 + 9*x^2*y + 27*x*y^2 + 27*y^3
>>> f(x = Integer(5)/z)
(3*y + 25*y/z^2 + 5/z)^3
>>> g = f.subs(x = Integer(5)/z); g
(3*y + 25*y/z^2 + 5/z)^3
>>> h = g.rational_simplify(); h
(27*y^3*z^6 + 135*y^2*z^5 + 225*(3*y^3 + y)*z^4 + 125*(18*y^2 + 1)*z^3 +
15625*y^3 + 9375*y^2*z + 1875*(3*y^3 + y)*z^2)/z^6
```

8.6 Roots of multivariate polynomials

Sage (using the interface to Singular) can solve multivariate polynomial equations in some situations (they assume that the solutions form a zero-dimensional variety) using Gröbner bases. Here is a simple example:

```
sage: R = PolynomialRing(QQ, 2, 'ab', order='lp')
sage: a,b = R.gens()
sage: I = (a^2-b^2-3, a-2*b)*R
sage: B = I.groebner_basis(); B
[a - 2*b, b^2 - 1]
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), 'ab', order='lp')
>>> a,b = R.gens()
>>> I = (a**Integer(2)-b**Integer(2)-Integer(3), a-Integer(2)*b)*R
>>> B = I.groebner_basis(); B
[a - 2*b, b^2 - 1]
```

So $b = \pm 1$ and $a = 2b$.

8.7 Gröbner bases

This computation uses Singular behind the scenes to compute the Gröbner basis.

```
sage: R = PolynomialRing(QQ, 4, 'abcd', order='lp')
sage: a,b,c,d = R.gens()
sage: I = (a+b+c+d, a*b+a*d+b*c+c*d, a*b*c+a*b*d+a*c*d+b*c*d, a*b*c*d-1)*R; I
Ideal (a + b + c + d, a*b + a*d + b*c + c*d, a*b*c + a*b*d + a*c*d + b*c*d,
a*b*c*d - 1) of Multivariate Polynomial Ring in a, b, c, d over Rational Field
sage: B = I.groebner_basis(); B
[a + b + c + d,
 b^2 + 2*b*d + d^2,
 b*c - b*d + c^2*d^4 + c*d - 2*d^2,
 b*d^4 - b + d^5 - d,
 c^3*d^2 + c^2*d^3 - c - d,
 c^2*d^6 - c^2*d^2 - d^4 + 1]
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(4), 'abcd', order='lp')
>>> a,b,c,d = R.gens()
>>> I = (a+b+c+d, a*b+a*d+b*c+c*d, a*b*c+a*b*d+a*c*d+b*c*d, a*b*c*d-Integer(1))*R; I
Ideal (a + b + c + d, a*b + a*d + b*c + c*d, a*b*c + a*b*d + a*c*d + b*c*d,
a*b*c*d - 1) of Multivariate Polynomial Ring in a, b, c, d over Rational Field
>>> B = I.groebner_basis(); B
[a + b + c + d,
 b^2 + 2*b*d + d^2,
 b*c - b*d + c^2*d^4 + c*d - 2*d^2,
 b*d^4 - b + d^5 - d,
 c^3*d^2 + c^2*d^3 - c - d,
 c^2*d^6 - c^2*d^2 - d^4 + 1]
```

You can work with multiple rings without having to switch back and forth like in Singular. For example,

```
sage: a,b,c = QQ['a,b,c'].gens()
sage: X,Y = GF(7)['X,Y'].gens()
sage: I = ideal(a, b^2, b^3+c^3)
sage: J = ideal(X^10 + Y^10)

sage: I.minimal_associated_primes ()
[Ideal (c, b, a) of Multivariate Polynomial Ring in a, b, c over Rational Field]

sage: J.minimal_associated_primes ()      # slightly random output
[Ideal (Y^4 + 3*X*Y^3 + 4*X^2*Y^2 + 4*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^4 + 4*X*Y^3 + 4*X^2*Y^2 + 3*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^2 + X^2) of Multivariate Polynomial Ring in X, Y over Finite Field
of size 7]
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> a,b,c = QQ['a,b,c'].gens()
>>> X,Y = GF(Integer(7))['X,Y'].gens()
>>> I = ideal(a, b**Integer(2), b**Integer(3)+c**Integer(3))
>>> J = ideal(X**Integer(10) + Y**Integer(10))

>>> I.minimal_associated_primes ()
[Ideal (c, b, a) of Multivariate Polynomial Ring in a, b, c over Rational Field]

>>> J.minimal_associated_primes ()     # slightly random output
[Ideal (Y^4 + 3*X*Y^3 + 4*X^2*Y^2 + 4*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^4 + 4*X*Y^3 + 4*X^2*Y^2 + 3*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^2 + X^2) of Multivariate Polynomial Ring in X, Y over Finite Field
of size 7]
```

All the real work is done by Singular.

Sage also includes `gfan` which provides other fast algorithms for computing Gröbner bases. See the section on "Gröbner fans" in the Reference Manual for more details.

CHAPTER

NINE

ELEMENTARY NUMBER THEORY

9.1 Taking modular powers

How do I compute modular powers in Sage?

To compute $51^{2006} \pmod{97}$ in Sage, type

```
sage: R = Integers(97)
sage: a = R(51)
sage: a^2006
12
```

```
>>> from sage.all import *
>>> R = Integers(Integer(97))
>>> a = R(Integer(51))
>>> a**Integer(2006)
12
```

Instead of `R = Integers(97)` you can also type `R = IntegerModRing(97)`. Another option is to use the interface with GMP:

```
sage: 51.powermod(99203843984,97)
96
```

```
>>> from sage.all import *
>>> Integer(51).powermod(Integer(99203843984),Integer(97))
96
```

9.2 Discrete logs

To find a number x such that $b^x \equiv a \pmod{m}$ (the discrete log of $a \pmod{m}$), you can call 's `log` command:

```
sage: r = Integers(125)
sage: b = r.multiplicative_generator()^3
sage: a = b^17
sage: a.log(b)
17
```

```
>>> from sage.all import *
>>> r = Integers(Integer(125))
```

(continues on next page)

(continued from previous page)

```
>>> b = r.multiplicative_generator()**Integer(3)
>>> a = b**Integer(17)
>>> a.log(b)
17
```

This also works over finite fields:

```
sage: FF = FiniteField(16,"a")
sage: a = FF.gen()
sage: c = a^7
sage: c.log(a)
7
```

```
>>> from sage.all import *
>>> FF = FiniteField(Integer(16),"a")
>>> a = FF.gen()
>>> c = a**Integer(7)
>>> c.log(a)
7
```

9.3 Prime numbers

How do you construct prime numbers in Sage?

The class `Primes` allows for primality testing:

```
sage: 2^(2^12)+1 in Primes()
False
sage: 11 in Primes()
True
```

```
>>> from sage.all import *
>>> Integer(2)**(Integer(2)**Integer(12))+Integer(1) in Primes()
False
>>> Integer(11) in Primes()
True
```

The usage of `next_prime` is self-explanatory:

```
sage: next_prime(2005)
      2011
```

```
>>> from sage.all import *
>>> next_prime(Integer(2005))
      2011
```

The Pari command `primepi` is used via the command `pari(x).primepi()`. This returns the number of primes $\leq x$, for example:

```
sage: pari(10).primepi()
      4
```

```
>>> from sage.all import *
>>> pari(Integer(10)).primepi()
      4
```

Using `primes_first_n` or `primes` one can check that, indeed, there are 4 primes up to 10:

```
sage: primes_first_n(5)
[2, 3, 5, 7, 11]
sage: list(primes(1, 10))
[2, 3, 5, 7]
```

```
>>> from sage.all import *
>>> primes_first_n(Integer(5))
[2, 3, 5, 7, 11]
>>> list(primes(Integer(1), Integer(10)))
[2, 3, 5, 7]
```

9.4 Divisors

How do you compute the sum of the divisors of an integer in Sage?

Sage uses `divisors(n)` for the list of divisors of n, `number_of_divisors(n)` for the number of divisors of n and `sigma(n,k)` for the sum of the k-th powers of the divisors of n (so `number_of_divisors(n)` and `sigma(n,0)` are the same).

For example:

```
sage: divisors(28); sum(divisors(28)); 2*28
[1, 2, 4, 7, 14, 28]
56
56
sage: sigma(28,0); sigma(28,1); sigma(28,2)
6
56
1050
```

```
>>> from sage.all import *
>>> divisors(Integer(28)); sum(divisors(Integer(28))); Integer(2)*Integer(28)
[1, 2, 4, 7, 14, 28]
56
56
>>> sigma(Integer(28),Integer(0)); sigma(Integer(28),Integer(1)); sigma(Integer(28),
↪Integer(2))
6
56
1050
```

9.5 Quadratic residues

Try this:

```
sage: Q = quadratic_residues(23); Q
[0, 1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
sage: N = [x for x in range(22) if kronecker(x,23)==-1]; N
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

```
>>> from sage.all import *
>>> Q = quadratic_residues(Integer(23)); Q
[0, 1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
>>> N = [x for x in range(Integer(22)) if kronecker(x,Integer(23))==-Integer(1)]; N
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

Q is the set of quadratic residues mod 23 and N is the set of non-residues.

Here is another way to construct these using the `kronecker` command (which is also called the "Legendre symbol"):

```
sage: [x for x in range(22) if kronecker(x,23)==1]
[1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
sage: [x for x in range(22) if kronecker(x,23)==-1]
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

```
>>> from sage.all import *
>>> [x for x in range(Integer(22)) if kronecker(x,Integer(23))==Integer(1)]
[1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
>>> [x for x in range(Integer(22)) if kronecker(x,Integer(23))==-Integer(1)]
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

CHAPTER

TEN

MODULAR FORMS

One of SageMath's computational specialities is (the very technical field of) modular forms and can do a lot more than is even suggested in this very brief introduction.

10.1 Cusp forms

How do you compute the dimension of a space of cusp forms using Sage?

To compute the dimension of the space of cusp forms for Gamma use the command `dimension_cusp_forms`. Here is an example from section "Modular forms" in the Tutorial:

```
sage: from sage.modular.dims import dimension_cusp_forms
sage: dimension_cusp_forms(Gamma0(11),2)
1
sage: dimension_cusp_forms(Gamma0(1),12)
1
sage: dimension_cusp_forms(Gamma1(389),2)
6112
```

```
>>> from sage.all import *
>>> from sage.modular.dims import dimension_cusp_forms
>>> dimension_cusp_forms(Gamma0(Integer(11)),Integer(2))
1
>>> dimension_cusp_forms(Gamma0(Integer(1)),Integer(12))
1
>>> dimension_cusp_forms(Gamma1(Integer(389)),Integer(2))
6112
```

Related commands: `dimension_new__cusp_forms_gamma0` (for dimensions of newforms), `dimension_modular_forms` (for modular forms), and `dimension_eis` (for Eisenstein series). The syntax is similar - see the Reference Manual for examples.

10.2 Coset representatives

The explicit representation of fundamental domains of arithmetic quotients H/Γ can be determined from the cosets of Γ in $SL_2(\mathbf{Z})$. How are these cosets computed in Sage?

Here is an example of computing the coset representatives of $SL_2(\mathbf{Z})/\Gamma_0(11)$:

```
sage: G = Gamma0(11); G
Congruence Subgroup Gamma0(11)
```

(continues on next page)

(continued from previous page)

```
sage: list(G.coset_reps())
[
[1 0]  [ 0 -1]  [1 0]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[0 1], [ 1  0], [1 1], [ 1  2], [ 1  3], [ 1  4], [ 1  5], [ 1  6],

[ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[ 1  7], [ 1  8], [ 1  9], [ 1 10]
]
```

```
>>> from sage.all import *
>>> G = Gamma0(Integer(11)); G
Congruence Subgroup Gamma0(11)
>>> list(G.coset_reps())
[
[1 0]  [ 0 -1]  [1 0]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[0 1], [ 1  0], [1 1], [ 1  2], [ 1  3], [ 1  4], [ 1  5], [ 1  6],
<BLANKLINE>
[ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[ 1  7], [ 1  8], [ 1  9], [ 1 10]
]
```

10.3 Modular symbols and Hecke operators

Next we illustrate computation of Hecke operators on a space of modular symbols of level 1 and weight 12.

```
sage: M = ModularSymbols(1,12)
sage: M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
sage: t2 = M.T(2)
sage: f = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
sage: factor(f)
(x - 2049) * (x + 24)^2
sage: M.T(11).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

```
>>> from sage.all import *
>>> M = ModularSymbols(Integer(1),Integer(12))
>>> M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
>>> t2 = M.T(Integer(2))
>>> f = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
>>> factor(f)
(x - 2049) * (x + 24)^2
>>> M.T(Integer(11)).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

Here `t2` represents the Hecke operator T_2 on the space of Full Modular Symbols for $\Gamma_0(1)$ of weight 12 with sign 0 and dimension 3 over $\mathbf{Q}$.

```
sage: M = ModularSymbols(Gamma1(6),3,sign=0)
sage: M
Modular Symbols space of dimension 4 for Gamma_1(6) of weight 3 with sign 0
over Rational Field
sage: M.basis()
([X,(0,5)], [X,(3,5)], [X,(4,5)], [X,(5,5)])
sage: M._compute_hecke_matrix_prime(2).charpoly()
x^4 - 17*x^2 + 16
sage: M.integral_structure()
Free module of degree 4 and rank 4 over Integer Ring
Echelon basis matrix:
[1 0 0 0]
[0 1 0 0]
[0 0 1 0]
[0 0 0 1]
```

```
>>> from sage.all import *
>>> M = ModularSymbols(Gamma1(Integer(6)),Integer(3),sign=Integer(0))
>>> M
Modular Symbols space of dimension 4 for Gamma_1(6) of weight 3 with sign 0
over Rational Field
>>> M.basis()
([X,(0,5)], [X,(3,5)], [X,(4,5)], [X,(5,5)])
>>> M._compute_hecke_matrix_prime(Integer(2)).charpoly()
x^4 - 17*x^2 + 16
>>> M.integral_structure()
Free module of degree 4 and rank 4 over Integer Ring
Echelon basis matrix:
[1 0 0 0]
[0 1 0 0]
[0 0 1 0]
[0 0 0 1]
```

See the section on modular forms in the Tutorial or the Reference Manual for more examples.

10.4 Genus formulas

Sage can compute the genus of $X_0(N)$, $X_1(N)$, and related curves. Here are some examples of the syntax:

```
sage: from sage.modular.dims import dimension_cusp_forms
sage: dimension_cusp_forms(Gamma0(22))
2
sage: dimension_cusp_forms(Gamma0(30))
3
sage: dimension_cusp_forms(Gamma1(30))
9
```

```
>>> from sage.all import *
>>> from sage.modular.dims import dimension_cusp_forms
>>> dimension_cusp_forms(Gamma0(Integer(22)))
2
```

(continues on next page)

(continued from previous page)

```
>>> dimension_cusp_forms(Gamma0(Integer(30)))
3
>>> dimension_cusp_forms(Gamma1(Integer(30)))
9
```

See the code for computing dimensions of spaces of modular forms (in `sage/modular/dims.py`) or the paper by Oesterlé and Cohen {CO} for some details.

CHAPTER

ELEVEN

ELLIPTIC CURVES

11.1 Conductor

How do you compute the conductor of an elliptic curve (over $\mathbf{Q}$) in Sage?

Once you define an elliptic curve E in Sage, using the `EllipticCurve` command, the conductor is one of several "methods" associated to E. Here is an example of the syntax (borrowed from section 2.4 "Modular forms" in the tutorial):

```
sage: E = EllipticCurve([1,2,3,4,5])
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
sage: E.conductor()
10351
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
>>> E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
>>> E.conductor()
10351
```

11.2 j-invariant

How do you compute the j-invariant of an elliptic curve in Sage?

Other methods associated to the `EllipticCurve` class are `j_invariant`, `discriminant`, and `weierstrass_model`. Here is an example of their syntax.

```
sage: E = EllipticCurve([0, -1, 1, -10, -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
sage: E.j_invariant()
-122023936/161051
sage: E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 - 13392*x - 1080432 over Rational Field
sage: E.discriminant()
-161051
sage: E = EllipticCurve(GF(5),[0, -1, 1, -10, -20])
sage: E.short_weierstrass_model()
```

(continues on next page)

(continued from previous page)

```
Elliptic Curve defined by y^2  = x^3 + 3*x + 3 over Finite Field of size 5
sage: E.j_invariant()
4
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(0), -Integer(1), Integer(1), -Integer(10), -
↪Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
>>> E.j_invariant()
-122023936/161051
>>> E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 - 13392*x - 1080432 over Rational Field
>>> E.discriminant()
-161051
>>> E = EllipticCurve(GF(Integer(5)),[Integer(0), -Integer(1), Integer(1), -
↪Integer(10), -Integer(20)])
>>> E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 + 3*x + 3 over Finite Field of size 5
>>> E.j_invariant()
4
```

11.3 The $GF(q)$-rational points on E

How do you compute the number of points of an elliptic curve over a finite field?

Given an elliptic curve defined over $\mathbb{F} = GF(q)$, Sage can compute its set of $\mathbb{F}$-rational points

```
sage: E = EllipticCurve(GF(5),[0, -1, 1, -10, -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
sage: E.points()
[(0 : 1 : 0), (0 : 0 : 1), (0 : 4 : 1), (1 : 0 : 1), (1 : 4 : 1)]
sage: E.cardinality()
5
sage: G = E.abelian_group()
sage: G
Additive abelian group isomorphic to Z/5 embedded in Abelian group of points on␣
↪Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
sage: G.permutation_group()
Permutation Group with generators [(1,2,3,4,5)]
```

```
>>> from sage.all import *
>>> E = EllipticCurve(GF(Integer(5)),[Integer(0), -Integer(1), Integer(1), -
↪Integer(10), -Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
>>> E.points()
[(0 : 1 : 0), (0 : 0 : 1), (0 : 4 : 1), (1 : 0 : 1), (1 : 4 : 1)]
>>> E.cardinality()
5
```

(continues on next page)

(continued from previous page)

```
>>> G = E.abelian_group()
>>> G
Additive abelian group isomorphic to Z/5 embedded in Abelian group of points on␣
↪Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
>>> G.permutation_group()
Permutation Group with generators [(1,2,3,4,5)]
```

11.4 Modular form associated to an elliptic curve over Q

Let E be a "nice" elliptic curve whose equation has integer coefficients, let N be the conductor of E and, for each n, let a_n be the number appearing in the Hasse-Weil L-function of E. The Taniyama-Shimura conjecture (proven by Wiles) states that there exists a modular form of weight two and level N which is an eigenform under the Hecke operators and has a Fourier series $\sum_{n=0}^{\infty} a_n q^n$. Sage can compute the sequence a_n associated to E. Here is an example.

```
sage: E = EllipticCurve([0, -1, 1, -10, -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
sage: E.conductor()
11
sage: E.anlist(20)
[0, 1, -2, -1, 2, 1, 2, -2, 0, -2, -2, 1, -2, 4, 4, -1, -4, -2, 4, 0, 2]
sage: E.analytic_rank()
0
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(0), -Integer(1), Integer(1), -Integer(10), -
↪Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
>>> E.conductor()
11
>>> E.anlist(Integer(20))
[0, 1, -2, -1, 2, 1, 2, -2, 0, -2, -2, 1, -2, 4, 4, -1, -4, -2, 4, 0, 2]
>>> E.analytic_rank()
0
```

CHAPTER

TWELVE

NUMBER FIELDS

12.1 Ramification

How do you compute the number fields with given discriminant and ramification in Sage?

Sage can access the Jones database of number fields with bounded ramification and degree less than or equal to 6. It must be installed separately (`database_jones_numfield`).

First load the database:

```
sage: J = JonesDatabase()             # optional - database
sage: J                               # optional - database
John Jones's table of number fields with bounded ramification and degree <= 6
```

```
>>> from sage.all import *
>>> J = JonesDatabase()             # optional - database
>>> J                               # optional - database
John Jones's table of number fields with bounded ramification and degree <= 6
```

List the degree and discriminant of all fields in the database that have ramification at most at 2:

```
sage: [(k.degree(), k.disc()) for k in J.unramified_outside([2])] # optional -␣
↪database
[(4, -2048), (2, 8), (4, -1024), (1, 1), (4, 256), (2, -4), (4, 2048), (4, 512), (4,␣
↪2048), (2, -8), (4, 2048)]
```

```
>>> from sage.all import *
>>> [(k.degree(), k.disc()) for k in J.unramified_outside([Integer(2)])] # optional -␣
↪database
[(4, -2048), (2, 8), (4, -1024), (1, 1), (4, 256), (2, -4), (4, 2048), (4, 512), (4,␣
↪2048), (2, -8), (4, 2048)]
```

List the discriminants of the fields of degree exactly 2 unramified outside 2:

```
sage: [k.disc() for k in J.unramified_outside([2],2)] # optional - database
[8, -4, -8]
```

```
>>> from sage.all import *
>>> [k.disc() for k in J.unramified_outside([Integer(2)],Integer(2))] # optional -␣
↪database
[8, -4, -8]
```

List the discriminants of cubic field in the database ramified exactly at 3 and 5:

```
sage: [k.disc() for k in J.ramified_at([3,5],3)] # optional - database
[-6075, -6075, -675, -135]
sage: factor(6075)
3^5 * 5^2
sage: factor(675)
3^3 * 5^2
sage: factor(135)
3^3 * 5
```

```
>>> from sage.all import *
>>> [k.disc() for k in J.ramified_at([Integer(3),Integer(5)],Integer(3))] # optional -
↪ database
[-6075, -6075, -675, -135]
>>> factor(Integer(6075))
3^5 * 5^2
>>> factor(Integer(675))
3^3 * 5^2
>>> factor(Integer(135))
3^3 * 5
```

List all fields in the database ramified at 101:

```
sage: J.ramified_at(101)                    # optional - database
[Number Field in a with defining polynomial x^2 - 101,
 Number Field in a with defining polynomial x^4 - x^3 + 13*x^2 - 19*x + 361,
 Number Field in a with defining polynomial x^5 - x^4 - 40*x^3 - 93*x^2 - 21*x + 17,
 Number Field in a with defining polynomial x^5 + x^4 - 6*x^3 - x^2 + 18*x + 4,
 Number Field in a with defining polynomial x^5 + 2*x^4 + 7*x^3 + 4*x^2 + 11*x - 6]
```

```
>>> from sage.all import *
>>> J.ramified_at(Integer(101))                    # optional - database
[Number Field in a with defining polynomial x^2 - 101,
 Number Field in a with defining polynomial x^4 - x^3 + 13*x^2 - 19*x + 361,
 Number Field in a with defining polynomial x^5 - x^4 - 40*x^3 - 93*x^2 - 21*x + 17,
 Number Field in a with defining polynomial x^5 + x^4 - 6*x^3 - x^2 + 18*x + 4,
 Number Field in a with defining polynomial x^5 + 2*x^4 + 7*x^3 + 4*x^2 + 11*x - 6]
```

12.2 Class numbers

How do you compute the class number of a number field in Sage?

The `class_number` is a method associated to a QuadraticField object:

```
sage: K = QuadraticField(29, 'x')
sage: K.class_number()
1
sage: K = QuadraticField(65, 'x')
sage: K.class_number()
2
sage: K = QuadraticField(-11, 'x')
```

(continues on next page)

(continued from previous page)

```
sage: K.class_number()
1
sage: K = QuadraticField(-15, 'x')
sage: K.class_number()
2
sage: K.class_group()
Class group of order 2 with structure C2 of Number Field in x with defining␣
↪polynomial x^2 + 15 with x = 3.872983346207417?*I
sage: K = QuadraticField(401, 'x')
sage: K.class_group()
Class group of order 5 with structure C5 of Number Field in x with defining␣
↪polynomial x^2 - 401 with x = 20.024984394500793?
sage: K.class_number()
5
sage: K.discriminant()
401
sage: K = QuadraticField(-479, 'x')
sage: K.class_group()
Class group of order 25 with structure C25 of Number Field in x with defining␣
↪polynomial x^2 + 479 with x = 21.886068628239293?*I
sage: K.class_number()
25
sage: K.pari_polynomial()
x^2 + 479
sage: K.degree()
2
```

```
>>> from sage.all import *
>>> K = QuadraticField(Integer(29), 'x')
>>> K.class_number()
1
>>> K = QuadraticField(Integer(65), 'x')
>>> K.class_number()
2
>>> K = QuadraticField(-Integer(11), 'x')
>>> K.class_number()
1
>>> K = QuadraticField(-Integer(15), 'x')
>>> K.class_number()
2
>>> K.class_group()
Class group of order 2 with structure C2 of Number Field in x with defining␣
↪polynomial x^2 + 15 with x = 3.872983346207417?*I
>>> K = QuadraticField(Integer(401), 'x')
>>> K.class_group()
Class group of order 5 with structure C5 of Number Field in x with defining␣
↪polynomial x^2 - 401 with x = 20.024984394500793?
>>> K.class_number()
5
>>> K.discriminant()
401
>>> K = QuadraticField(-Integer(479), 'x')
```

(continues on next page)

(continued from previous page)

```
>>> K.class_group()
Class group of order 25 with structure C25 of Number Field in x with defining␣
↪polynomial x^2 + 479 with x = 21.886068628239292?*I
>>> K.class_number()
25
>>> K.pari_polynomial()
x^2 + 479
>>> K.degree()
2
```

Here's an example involving a more general type of number field:

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: K = NumberField(x^5+10*x+1, 'a')
sage: K
Number Field in a with defining polynomial x^5 + 10*x + 1
sage: K.degree()
5
sage: K.pari_polynomial()
x^5 + 10*x + 1
sage: K.discriminant()
25603125
sage: K.class_group()
Class group of order 1 of Number Field in a with defining
polynomial x^5 + 10*x + 1
sage: K.class_number()
1
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> K = NumberField(x**Integer(5)+Integer(10)*x+Integer(1), 'a')
>>> K
Number Field in a with defining polynomial x^5 + 10*x + 1
>>> K.degree()
5
>>> K.pari_polynomial()
x^5 + 10*x + 1
>>> K.discriminant()
25603125
>>> K.class_group()
Class group of order 1 of Number Field in a with defining
polynomial x^5 + 10*x + 1
>>> K.class_number()
1
```

- See also the link for class numbers at http://mathworld.wolfram.com/ClassNumber.html at the Math World site for tables, formulas, and background information.
- For cyclotomic fields, try:

```
sage: K = CyclotomicField(19)
sage: K.class_number()    # long time
1
```

```
>>> from sage.all import *
>>> K = CyclotomicField(Integer(19))
>>> K.class_number()     # long time
1
```

For further details, see the documentation strings in the `ring/number_field.py` file.

12.3 Integral basis

How do you compute an integral basis of a number field in Sage?

Sage can compute a list of elements of this number field that are a basis for the full ring of integers of a number field.

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: K = NumberField(x^5+10*x+1, 'a')
sage: K.integral_basis()
[1, a, a^2, a^3, a^4]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> K = NumberField(x**Integer(5)+Integer(10)*x+Integer(1), 'a')
>>> K.integral_basis()
[1, a, a^2, a^3, a^4]
```

Next we compute the ring of integers of a cubic field in which 2 is an "essential discriminant divisor", so the ring of integers is not generated by a single element.

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: K = NumberField(x^3 + x^2 - 2*x + 8, 'a')
sage: K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> K = NumberField(x**Integer(3) + x**Integer(2) - Integer(2)*x + Integer(8), 'a')
>>> K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

CHAPTER

THIRTEEN

ALGEBRAIC GEOMETRY

13.1 Point counting on curves

How do you count points on an elliptic curve over a finite field in Sage?

Over prime finite fields, includes both the baby step giant step method and the SEA (Schoof-Elkies-Atkin) algorithm (implemented in PARI by Christophe Doche and Sylvain Duquesne). An example taken form the Reference manual:

```
sage: E = EllipticCurve(GF(10007),[1,2,3,4,5])
sage: E.cardinality()
10076
```

```
>>> from sage.all import *
>>> E = EllipticCurve(GF(Integer(10007)),[Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(5)])
>>> E.cardinality()
10076
```

The command `E.points()` will return the actual list of rational points.

How do you count points on a plane curve over a finite field? The `rational_points` command produces points by a simple enumeration algorithm. Here is an example of the syntax:

```
sage: x,y,z = PolynomialRing(GF(5), 3, 'xyz').gens()
sage: C = Curve(y^2*z^7 - x^9 - x*z^8); C
Projective Plane Curve over Finite Field of size 5 defined by -x^9 + y^2*z^7 - x*z^8
sage: C.rational_points()
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
sage: C.rational_points(algorithm="bn")
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
```

```
>>> from sage.all import *
>>> x,y,z = PolynomialRing(GF(Integer(5)), Integer(3), 'xyz').gens()
>>> C = Curve(y**Integer(2)*z**Integer(7) - x**Integer(9) - x*z**Integer(8)); C
Projective Plane Curve over Finite Field of size 5 defined by -x^9 + y^2*z^7 - x*z^8
>>> C.rational_points()
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
>>> C.rational_points(algorithm="bn")
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
```

The option `algorithm="bn"` uses Sage's Singular interface and calls the `brnoeth` package.

Here is another example using Sage's `rational_points` applied to Klein's quartic over $GF(8)$.

```
sage: x, y, z = PolynomialRing(GF(8,'a'), 3, 'xyz').gens()
sage: f = x^3*y+y^3*z+x*z^3
sage: C = Curve(f); C
Projective Plane Curve over Finite Field in a of size 2^3 defined by x^3*y + y^3*z +␣
↪x*z^3
sage: C.rational_points()
[(0 : 0 : 1),
 (0 : 1 : 0),
 (1 : 0 : 0),
 (1 : a : 1),
 (1 : a^2 : 1),
 (1 : a^2 + a : 1),
 (a : 1 : 1),
 (a : a^2 : 1),
 (a : a^2 + 1 : 1),
 (a + 1 : a + 1 : 1),
 (a + 1 : a^2 : 1),
 (a + 1 : a^2 + a + 1 : 1),
 (a^2 : 1 : 1),
 (a^2 : a^2 + a : 1),
 (a^2 : a^2 + a + 1 : 1),
 (a^2 + 1 : a + 1 : 1),
 (a^2 + 1 : a^2 + 1 : 1),
 (a^2 + 1 : a^2 + a : 1),
 (a^2 + a : 1 : 1),
 (a^2 + a : a : 1),
 (a^2 + a : a + 1 : 1),
 (a^2 + a + 1 : a : 1),
 (a^2 + a + 1 : a^2 + 1 : 1),
 (a^2 + a + 1 : a^2 + a + 1 : 1)]
```

```
>>> from sage.all import *
>>> x, y, z = PolynomialRing(GF(Integer(8),'a'), Integer(3), 'xyz').gens()
>>> f = x**Integer(3)*y+y**Integer(3)*z+x*z**Integer(3)
>>> C = Curve(f); C
Projective Plane Curve over Finite Field in a of size 2^3 defined by x^3*y + y^3*z +␣
↪x*z^3
>>> C.rational_points()
[(0 : 0 : 1),
 (0 : 1 : 0),
 (1 : 0 : 0),
 (1 : a : 1),
 (1 : a^2 : 1),
 (1 : a^2 + a : 1),
 (a : 1 : 1),
 (a : a^2 : 1),
 (a : a^2 + 1 : 1),
 (a + 1 : a + 1 : 1),
 (a + 1 : a^2 : 1),
 (a + 1 : a^2 + a + 1 : 1),
 (a^2 : 1 : 1),
 (a^2 : a^2 + a : 1),
```

(continues on next page)

(continued from previous page)

```
(a^2 : a^2 + a + 1 : 1),
(a^2 + 1 : a + 1 : 1),
(a^2 + 1 : a^2 + 1 : 1),
(a^2 + 1 : a^2 + a : 1),
(a^2 + a : 1 : 1),
(a^2 + a : a : 1),
(a^2 + a : a + 1 : 1),
(a^2 + a + 1 : a : 1),
(a^2 + a + 1 : a^2 + 1 : 1),
(a^2 + a + 1 : a^2 + a + 1 : 1)]
```

13.1.1 Other methods

- For a plane curve, you can use Singular's `closed_points` command. The input is the vanishing ideal I of the curve X in a ring of 2 variables $F[x, y]$. The `closed_points` command returns a list of prime ideals (each a Gröbner basis), corresponding to the (distinct affine closed) points of $V(I)$. Here's an example:

```
sage: singular_console()
                     SINGULAR                                /  Development
 A Computer Algebra System for Polynomial Computations   /   version 3-0-1
                                                        0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann         \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern    \
// ** executing /home/wdj/sagefiles/sage-0.9.4/local/LIB/.singularrc
> LIB "brnoeth.lib";
> ring s = 2,(x,y),lp;
> ideal I = x4+x,y4+y;
> list L = closed_points(I);
> L;
[1]:
   _[1] = y
   _[2] = x
[2]:
   _[1] = y
   _[2] = x+1
[3]:
   _[1] = y
   _[2] = x2+x+1
[4]:
   _[1] = y+1
   _[2] = x
[5]:
   _[1] = y+1
   _[2] = x+1
[6]:
   _[1] = y+1
   _[2] = x2+x+1
[7]:
   _[1] = y2+y+1
   _[2] = x+1
[8]:
   _[1] = y2+y+1
```

(continues on next page)

(continued from previous page)

```
   _[2] = x
[9]:
   _[1] = y2+y+1
   _[2] = x+y
[10]:
   _[1] = y2+y+1
   _[2] = x+y+1
> Auf Wiedersehen.
```

```
>>> from sage.all import *
>>> singular_console()
                     SINGULAR                                /  Development
 A Computer Algebra System for Polynomial Computations   /   version 3-0-1
                                                       0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann        \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern    \
// ** executing /home/wdj/sagefiles/sage-0.9.4/local/LIB/.singularrc
> LIB "brnoeth.lib";
> ring s = 2,(x,y),lp;
> ideal I = x4+x,y4+y;
> list L = closed_points(I);
> L;
[1]:
   _[1] = y
   _[2] = x
[2]:
   _[1] = y
   _[2] = x+1
[3]:
   _[1] = y
   _[2] = x2+x+1
[4]:
   _[1] = y+1
   _[2] = x
[5]:
   _[1] = y+1
   _[2] = x+1
[6]:
   _[1] = y+1
   _[2] = x2+x+1
[7]:
   _[1] = y2+y+1
   _[2] = x+1
[8]:
   _[1] = y2+y+1
   _[2] = x
[9]:
   _[1] = y2+y+1
   _[2] = x+y
[10]:
   _[1] = y2+y+1
   _[2] = x+y+1
```

(continues on next page)

(continued from previous page)

```
> Auf Wiedersehen.
```

```
sage: singular.lib("brnoeth.lib")
sage: s = singular.ring(2,'(x,y)','lp')
sage: I = singular.ideal('x^4+x', 'y^4+y')
sage: L = singular.closed_points(I)
sage: # Here you have all the points :
sage: L       # random
[1]:
   _[1]=y+1
   _[2]=x+1
...
sage: l=[L[k].sage() for k in [1..10]]; len(l) # there are 10 points
10
sage: r=sorted(l[0].ring().gens()); r
[y, x]
sage: r in [t.gens() for t in l] #  one of them is given by [y,x]
True
```

```
>>> from sage.all import *
>>> singular.lib("brnoeth.lib")
>>> s = singular.ring(Integer(2),'(x,y)','lp')
>>> I = singular.ideal('x^4+x', 'y^4+y')
>>> L = singular.closed_points(I)
>>> # Here you have all the points :
>>> L       # random
[1]:
   _[1]=y+1
   _[2]=x+1
...
>>> l=[L[k].sage() for k in (ellipsis_range(Integer(1),Ellipsis,Integer(10)))];␣
↪len(l) # there are 10 points
10
>>> r=sorted(l[Integer(0)].ring().gens()); r
[y, x]
>>> r in [t.gens() for t in l] #  one of them is given by [y,x]
True
```

- Another way to compute rational points is to use Singular's `NSplaces` command. Here's the Klein quartic over $GF(8)$ done this way:

```
sage: singular.LIB("brnoeth.lib")
sage: s = singular.ring(2,'(x,y)','lp')
sage: f = singular.poly('x3y+y3+x')
sage: klein1 = f.Adj_div(); print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
```

(continues on next page)

(continued from previous page)

```
...
sage: # define a curve X = {f = 0} over GF(2)
sage: klein2 = singular.NSplaces(3,klein1)
sage: print(singular.eval('extcurve(3,%s)'%klein2.name()))
Total number of rational places : NrRatPl = 23
...
sage: klein3 = singular.extcurve(3, klein2)
```

```
>>> from sage.all import *
>>> singular.LIB("brnoeth.lib")
>>> s = singular.ring(Integer(2),'(x,y)','lp')
>>> f = singular.poly('x3y+y3+x')
>>> klein1 = f.Adj_div(); print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
...
>>> # define a curve X = {f = 0} over GF(2)
>>> klein2 = singular.NSplaces(Integer(3),klein1)
>>> print(singular.eval('extcurve(3,%s)'%klein2.name()))
Total number of rational places : NrRatPl = 23
...
>>> klein3 = singular.extcurve(Integer(3), klein2)
```

Above we defined a curve $X = \{f = 0\}$ over $GF(8)$ in Singular.

```
sage: print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
   [2]:
      // coefficients: ZZ/2...
// number of vars : 3
//        block   1 : ordering lp
//                  : names    x y z
//        block   2 : ordering C
[2]:
   4,3
[3]:
   [1]:
      1,1
   [2]:
      1,2
[4]:
```

(continues on next page)

(continued from previous page)

```
   0
[5]:
   [1]:
      [1]:
         // coefficients: ZZ/2...
// number of vars : 3
//         block   1 : ordering ls
//                   : names    x y t
//         block   2 : ordering C
      [2]:
         1,1
sage: print(klein1[3])
[1]:
   1,1
[2]:
   1,2
```

```
>>> from sage.all import *
>>> print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//         block   1 : ordering lp
//                   : names    x y
//         block   2 : ordering C
   [2]:
      // coefficients: ZZ/2...
// number of vars : 3
//         block   1 : ordering lp
//                   : names    x y z
//         block   2 : ordering C
[2]:
   4,3
[3]:
   [1]:
      1,1
   [2]:
      1,2
[4]:
   0
[5]:
   [1]:
      [1]:
         // coefficients: ZZ/2...
// number of vars : 3
//         block   1 : ordering ls
//                   : names    x y t
//         block   2 : ordering C
      [2]:
         1,1
>>> print(klein1[Integer(3)])
```

(continues on next page)

(continued from previous page)

```
[1]:
   1,1
[2]:
   1,2
```

For the places of degree 3:

```
sage: print(klein2[3])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

```
>>> from sage.all import *
>>> print(klein2[Integer(3)])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

Each point below is a pair: (degree, point index number).

```
sage: print(klein3[3])
[1]:
```

(continues on next page)

(continued from previous page)

```
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

```
>>> from sage.all import *
>>> print(klein3[Integer(3)])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

To actually get the points of $X(GF(8))$:

```
sage: R = klein3[1][5]
sage: R.set_ring()
sage: singular("POINTS;")
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
```

(continues on next page)

(continued from previous page)

```
   [1]:
      1
   [2]:
      0
   [3]:
      0
...
```

```
>>> from sage.all import *
>>> R = klein3[Integer(1)][Integer(5)]
>>> R.set_ring()
>>> singular("POINTS;")
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      1
   [2]:
      0
   [3]:
      0
...
```

plus 21 others (omitted). There are a total of 23 rational points.

13.2 Riemann-Roch spaces using Singular

To compute a basis of the Riemann-Roch space of a divisor D on a curve over a field F, one can use Sage's wrapper `riemann_roch_basis` of Singular's implementation of the Brill Noether algorithm. Note that this wrapper currently only works when F is prime and the divisor D is supported on rational points. Below are examples of how to use `riemann_roch_basis` and how to use Singular itself to help an understanding of how the wrapper works.

- Using `riemann_roch_basis`:

```
sage: x, y, z = PolynomialRing(GF(5), 3, 'xyz').gens()
sage: f = x^7 + y^7 + z^7
sage: X = Curve(f); pts = X.rational_points()
sage: D = X.divisor([ (3, pts[0]), (-1,pts[1]), (10, pts[5]) ])
sage: X.riemann_roch_basis(D)
[(-2*x + y)/(x + y), (-x + z)/(x + y)]
```

```
>>> from sage.all import *
>>> x, y, z = PolynomialRing(GF(Integer(5)), Integer(3), 'xyz').gens()
>>> f = x**Integer(7) + y**Integer(7) + z**Integer(7)
>>> X = Curve(f); pts = X.rational_points()
>>> D = X.divisor([ (Integer(3), pts[Integer(0)]), (-Integer(1),pts[Integer(1)]),␣
```

(continues on next page)

(continued from previous page)

```
↪(Integer(10), pts[Integer(5)]) ])
>>> X.riemann_roch_basis(D)
[(-2*x + y)/(x + y), (-x + z)/(x + y)]
```

- Using Singular's `BrillNoether` command (for details see the section Brill-Noether in the Singular online documentation (https://www.singular.uni-kl.de/Manual/4-3-0/sing_2254.htm and the paper {CF}):

```
sage: singular.LIB('brnoeth.lib')
sage: _ = singular.ring(5,'(x,y)','lp')
sage: print(singular.eval("list X = Adj_div(-x5+y2+x);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully

The genus of the curve is 2
sage: print(singular.eval("X = NSplaces(1,X);"))
Computing non-singular affine places of degree 1 ...
sage: print(singular("X[3];"))
[1]:
   1,1
[2]:
   1,2
[3]:
   1,3
[4]:
   1,4
[5]:
   1,5
[6]:
   1,6
```

```
>>> from sage.all import *
>>> singular.LIB('brnoeth.lib')
>>> _ = singular.ring(Integer(5),'(x,y)','lp')
>>> print(singular.eval("list X = Adj_div(-x5+y2+x);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully
<BLANKLINE>
The genus of the curve is 2
>>> print(singular.eval("X = NSplaces(1,X);"))
Computing non-singular affine places of degree 1 ...
>>> print(singular("X[3];"))
[1]:
   1,1
[2]:
```

(continues on next page)

(continued from previous page)

```
   1,2
[3]:
   1,3
[4]:
   1,4
[5]:
   1,5
[6]:
   1,6
```

The first integer of each pair in the above list is the degree d of a point. The second integer is the index of this point in the list POINTS of the ring X[5][d][1]. Note that the order of this latter list is different every time the algorithm is run, e.g. 1, 1 in the above list refers to a different rational point each time. A divisor is given by defining a list G of integers of the same length as X[3] such that if the k-th entry of X[3] is d, i, then the k-th entry of G is the multiplicity of the divisor at the i-th point in the list POINTS of the ring X[5][d][1]. Let us proceed by defining a "random" divisor of degree 12 and computing a basis of its Riemann-Roch space:

```
sage: singular.eval("intvec G = 4,4,4,0,0,0;")
''
sage: singular.eval("def R = X[1][2];")
''
sage: singular.eval("setring R;")
''
sage: print(singular.eval("list LG = BrillNoether(G,X);"))
Forms of degree 6 :
28

Vector basis successfully computed
```

```
>>> from sage.all import *
>>> singular.eval("intvec G = 4,4,4,0,0,0;")
''
>>> singular.eval("def R = X[1][2];")
''
>>> singular.eval("setring R;")
''
>>> print(singular.eval("list LG = BrillNoether(G,X);"))
Forms of degree 6 :
28
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
```

13.2.1 AG codes

Sage can compute an AG code $C = C_X(D, E)$ by calling Singular's BrillNoether to compute a basis of the Riemann Roch space $L(D) = L_X(D)$. In addition to the curve X and the divisor D, you must also specify the evaluation divisor E.

Note that this section has not been updated since the wrapper `riemann_roch_basis` has been fixed. See above for how to properly define a divisor for Singular's `BrillNoether` command.

Here's an example, one which computes a generator matrix of an associated AG code. This time we use Singular's

`AGCode_L` command.

```
sage: singular.LIB('brnoeth.lib')
sage: singular.eval("ring s = 2,(x,y),lp;")
''
sage: print(singular.eval("list HC = Adj_div(x3+y2+y);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully

The genus of the curve is 1
sage: print(singular.eval("list HC1 = NSplaces(1..2,HC);"))
Computing non-singular affine places of degree 1 ...
Computing non-singular affine places of degree 2 ...
sage: print(singular.eval("HC = extcurve(2,HC1);"))
Total number of rational places : NrRatPl = 9
```

```
>>> from sage.all import *
>>> singular.LIB('brnoeth.lib')
>>> singular.eval("ring s = 2,(x,y),lp;")
''
>>> print(singular.eval("list HC = Adj_div(x3+y2+y);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully
<BLANKLINE>
The genus of the curve is 1
>>> print(singular.eval("list HC1 = NSplaces(1..2,HC);"))
Computing non-singular affine places of degree 1 ...
Computing non-singular affine places of degree 2 ...
>>> print(singular.eval("HC = extcurve(2,HC1);"))
Total number of rational places : NrRatPl = 9
```

We set the following to `junk` to discard the output:

```
sage: junk = singular.eval("intvec G = 5;")      # the rational divisor G = 5*HC[3][1]
sage: junk = singular.eval("def R = HC[1][2];")
sage: singular.eval("setring R;")
''
```

```
>>> from sage.all import *
>>> junk = singular.eval("intvec G = 5;")      # the rational divisor G = 5*HC[3][1]
>>> junk = singular.eval("def R = HC[1][2];")
>>> singular.eval("setring R;")
''
```

The vector G represents the divisor "5 times the point at infinity".

Next, we compute the Riemann-Roch space.

```
sage: print(singular.eval("BrillNoether(G,HC);"))
Forms of degree 3 :
10

Vector basis successfully computed

[1]:
   _[1]=x
   _[2]=z
[2]:
   _[1]=y
   _[2]=z
[3]:
   _[1]=1
   _[2]=1
[4]:
   _[1]=y2+yz
   _[2]=xz
[5]:
   _[1]=y3+y2z
   _[2]=x2z
```

```
>>> from sage.all import *
>>> print(singular.eval("BrillNoether(G,HC);"))
Forms of degree 3 :
10
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
[1]:
   _[1]=x
   _[2]=z
[2]:
   _[1]=y
   _[2]=z
[3]:
   _[1]=1
   _[2]=1
[4]:
   _[1]=y2+yz
   _[2]=xz
[5]:
   _[1]=y3+y2z
   _[2]=x2z
```

That was the basis of the Riemann-Roch space, where each pair of functions represents the quotient (first function divided by second function). Each of these basis elements get evaluated at certain points to construct the generator matrix of the code. We next construct the points.

```
sage: singular.eval("def R = HC[1][5];")
'// ** redefining R **'
```

(continues on next page)

(continued from previous page)

```
sage: singular.eval("setring R;")
''
sage: print(singular.eval("POINTS;"))
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      0
   [2]:
      1
   [3]:
      1
[3]:
   [1]:
      0
   [2]:
      0
   [3]:
      1
[4]:
   [1]:
      (a+1)
   [2]:
      (a)
   [3]:
      1
...
```

```
>>> from sage.all import *
>>> singular.eval("def R = HC[1][5];")
'// ** redefining R **'
>>> singular.eval("setring R;")
''
>>> print(singular.eval("POINTS;"))
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      0
   [2]:
      1
   [3]:
```

(continues on next page)

(continued from previous page)

```
      1
[3]:
   [1]:
      0
   [2]:
      0
   [3]:
      1
[4]:
   [1]:
      (a+1)
   [2]:
      (a)
   [3]:
      1
...
```

plus 5 more, for a total of 9 rational points on the curve. We define our "evaluation divisor" D using a subset of these points (all but the first):

```
sage: singular.eval("def ER = HC[1][4];")
''
sage: singular.eval("setring ER;")
''
sage: # D = sum of the rational places no. 2..9 over F_4
sage: singular.eval("intvec D = 2..9;")
''
sage: # let us construct the corresponding evaluation AG code :
sage: print(singular.eval("matrix C = AGcode_L(G,D,HC);"))
Forms of degree 3 :
10

Vector basis successfully computed

sage: # here is a linear code of type [8,5,> = 3] over F_4
sage: print(singular.eval("print(C);"))
0,0,(a+1),(a),  1,  1,    (a),  (a+1),
1,0,(a),  (a+1),(a),(a+1),(a),  (a+1),
1,1,1,    1,    1,  1,    1,    1,
0,0,(a),  (a+1),1,  1,    (a+1),(a),
0,0,1,    1,    (a),(a+1),(a+1),(a)
```

```
>>> from sage.all import *
>>> singular.eval("def ER = HC[1][4];")
''
>>> singular.eval("setring ER;")
''
>>> # D = sum of the rational places no. 2..9 over F_4
>>> singular.eval("intvec D = 2..9;")
''
>>> # let us construct the corresponding evaluation AG code :
>>> print(singular.eval("matrix C = AGcode_L(G,D,HC);"))
```

(continues on next page)

(continued from previous page)

```
Forms of degree 3 :
10
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
>>> # here is a linear code of type [8,5,> = 3] over F_4
>>> print(singular.eval("print(C);"))
0,0,(a+1),(a),  1,  1,    (a),  (a+1),
1,0,(a),  (a+1),(a),(a+1),(a),  (a+1),
1,1,1,    1,    1,  1,    1,    1,
0,0,(a),  (a+1),1,  1,    (a+1),(a),
0,0,1,    1,    (a),(a+1),(a+1),(a)
```

This is, finally, our desired generator matrix, where `a` represents a generator of the field extension of degree 2 over the base field $GF(2)$.

Can this be "wrapped"?

CHAPTER

FOURTEEN

INTERFACE ISSUES

14.1 Background jobs

Yes, a Sage job can be run in the background on a UNIX system. The canonical thing to do is type

```
$ nohup sage < command_file  > output_file &
```

The advantage of nohup is that Sage will continue running after you log out.

Currently Sage will appear as "sage-ipython" or "python" in the output of the (unix) `top` command, but in future versions of Sage it will appears as `sage`.

14.2 Referencing Sage

See citing Sage.

14.3 Logging your Sage session

Yes you can log your sessions.

(a) You can write the output to a file, by running Sage in the background (*Background jobs*).

(b) Start in a KDE konsole (this only work in linux). Go to `Settings` → `History ...` and select unlimited. Start your session. When ready, go to `edit` → `save history as ...`.

Some interfaces (such as the interface to Singular or that to GAP) allow you to create a log file. For Singular, there is a logfile option (in `singular.py`). In GAP, use the command `LogTo`.

14.4 LaTeX conversion

Yes, you can output some of your results into LaTeX.

```
sage: M = MatrixSpace(RealField(),3,3)
sage: A = M([1,2,3, 4,5,6, 7,8,9])
sage: print(latex(A))
\left(\begin{array}{rrr}
    1.00000000000000 & 2.00000000000000 & 3.00000000000000 \\
    4.00000000000000 & 5.00000000000000 & 6.00000000000000 \\
    7.00000000000000 & 8.00000000000000 & 9.00000000000000
    \end{array}\right)
```

```
>>> from sage.all import *
>>> M = MatrixSpace(RealField(),Integer(3),Integer(3))
>>> A = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6),␣
↪Integer(7),Integer(8),Integer(9)])
>>> print(latex(A))
\left(\begin{array}{rrr}
    1.00000000000000 & 2.00000000000000 & 3.00000000000000 \\
    4.00000000000000 & 5.00000000000000 & 6.00000000000000 \\
    7.00000000000000 & 8.00000000000000 & 9.00000000000000
    \end{array}\right)
```

```
sage: view(A)
```

```
>>> from sage.all import *
>>> view(A)
```

At this point a dvi preview should automatically be called to display in a separate window the LaTeX output produced.

LaTeX previewing for multivariate polynomials and rational functions is also available:

```
sage: x = PolynomialRing(QQ,3, 'x').gens()
sage: f = x[0] + x[1] - 2*x[1]*x[2]
sage: h = f /(x[1] + x[2])
sage: print(latex(h))
\frac{-2 x_{1} x_{2} + x_{0} + x_{1}}{x_{1} + x_{2}}
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ,Integer(3), 'x').gens()
>>> f = x[Integer(0)] + x[Integer(1)] - Integer(2)*x[Integer(1)]*x[Integer(2)]
>>> h = f /(x[Integer(1)] + x[Integer(2)])
>>> print(latex(h))
\frac{-2 x_{1} x_{2} + x_{0} + x_{1}}{x_{1} + x_{2}}
```

14.5 Sage and other computer algebra systems

If `foo` is a Pari, GAP (without ending semicolon), Singular, Maxima command, resp., enter `gp("foo")` for Pari, `libgap.eval("foo")` `singular.eval("foo")`, `maxima("foo")`, resp.. These programs merely send the command string to the external program, execute it, and read the result back into Sage. Therefore, these will not work if the external program is not installed and in your PATH.

14.6 Command-line Sage help

If you know only part of the name of a Sage command and want to know where it occurs in Sage, just type `sage -grep <string>` to find all occurrences of `<string>` in the Sage source code. For example,

```
$ sage -grep berlekamp_massey
matrix/all.py:from berlekamp_massey import berlekamp_massey
matrix/berlekamp_massey.py:def berlekamp_massey(a):
matrix/matrix.py:import berlekamp_massey
matrix/matrix.py:            g =
berlekamp_massey.berlekamp_massey(cols[i].list())
```

Type `help(foo)` or `foo??` for help and `foo.[tab]` for searching of Sage commands. Type `help()` for Python commands.

For example

```
help(Matrix)
```

returns

```
Help on cython_function_or_method in module sage.matrix.constructor:

matrix(*args, **kwds)
    matrix(*args, **kwds)
    File: sage/matrix/constructor.pyx (starting at line 21)

        Create a matrix.

        This implements the ``matrix`` constructor::

            sage: matrix([[1,2],[3,4]])
            [1 2]
            [3 4]

        It also contains methods to create special types of matrices, see
        ``matrix.[tab]`` for more options. For example::
--More--
```

in a new screen. Type q to return to the Sage screen.

14.7 Reading and importing files into Sage

A file imported into Sage must end in `.py`, e.g., `foo.py` and contain legal Python syntax. For a simple example see *Permutation groups* with the Rubik's cube group example above.

Another way to read a file in is to use the `load` or `attach` command. Create a file called `example.sage` (located in the home directory of Sage) with the following content:

```
print("Hello World")
print(2^3)
```

Read in and execute `example.sage` file using the `load` command.

```
sage: load("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> load("example.sage")
Hello World
8
```

You can also `attach` a Sage file to a running session:

```
sage: attach("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> attach("example.sage")
Hello World
8
```

Now if you change `example.sage` and enter one blank line into Sage, then the contents of `example.sage` will be automatically reloaded into Sage:

```
sage: !emacs example.sage&     #change 2^3 to 2^4
sage:                          #hit return
**************************************************
               Reloading 'example.sage'
**************************************************
Hello World
16
```

```
>>> from sage.all import *
>>> !emacs example.sage&     #change 2^3 to 2^4
>>>                          #hit return
**************************************************
               Reloading 'example.sage'
**************************************************
Hello World
16
```

14.8 Python language program code for Sage commands

Let's say you want to know what the Python program is for the Sage command to compute the center of a permutation group. Use Sage's help interface to find the file name:

```
sage: PermutationGroup.center?
Type:           instancemethod
Base Class:     <class 'instancemethod'>
String Form:    <unbound method PermutationGroup.center>
Namespace:      Interactive
File:           /home/wdj/sage/local/lib/python2.4/site-packages/sage/groups/
↪permgroup.py
Definition:     PermutationGroup.center(self)
```

```
>>> from sage.all import *
>>> PermutationGroup.center?
Type:           instancemethod
Base Class:     <class 'instancemethod'>
String Form:    <unbound method PermutationGroup.center>
Namespace:      Interactive
File:           /home/wdj/sage/local/lib/python2.4/site-packages/sage/groups/
```

(continues on next page)

(continued from previous page)

```
↪permgroup.py
Definition:     PermutationGroup.center(self)
```

Now you know that the command is located in the `permgroup.py` file and you know the directory to look for that Python module. You can use an editor to read the code itself.

14.9 "Special functions" in Sage

Sage has many special functions (see the reference manual at http://doc.sagemath.org/html/en/reference/functions/), and most of them can be manipulated symbolically. Where this is not implemented, it is possible that other symbolic packages have the functionality.

Via Maxima, some symbolic manipulation is allowed:

```
sage: maxima.eval("f:bessel_y (v, w)")
'bessel_y(v,w)'
sage: maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
sage: maxima.eval("diff (jacobi_sn (u, m), u)")
'jacobi_cn(u,m)*jacobi_dn(u,m)'
sage: jsn = lambda x: jacobi("sn",x,1)
sage: P = plot(jsn,0,1, plot_points=20); Q = plot(lambda x:bessel_Y( 1, x), 1/2,1)
sage: show(P)
sage: show(Q)
```

```
>>> from sage.all import *
>>> maxima.eval("f:bessel_y (v, w)")
'bessel_y(v,w)'
>>> maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
>>> maxima.eval("diff (jacobi_sn (u, m), u)")
'jacobi_cn(u,m)*jacobi_dn(u,m)'
>>> jsn = lambda x: jacobi("sn",x,Integer(1))
>>> P = plot(jsn,Integer(0),Integer(1), plot_points=Integer(20)); Q = plot(lambda␣
↪x:bessel_Y( Integer(1), x), Integer(1)/Integer(2),Integer(1))
>>> show(P)
>>> show(Q)
```

In addition to `maxima`, `pari` and `octave` also have special functions (in fact, some of `pari`'s special functions are wrapped in Sage).

Here's an example using Sage's interface (located in sage/interfaces/octave.py) with `octave` (https://www.gnu.org/software/octave/doc/latest).

```
sage: octave("atanh(1.1)")    ## optional - octave
(1.52226,1.5708)
```

```
>>> from sage.all import *
>>> octave("atanh(1.1)")    ## optional - octave
(1.52226,1.5708)
```

Here's an example using Sage's interface to `pari`'s special functions.

```
sage: pari('2+I').besselk(3)
0.0455907718407551 + 0.0289192946582081*I
sage: pari('2').besselk(3)
0.0615104584717420
```

```
>>> from sage.all import *
>>> pari('2+I').besselk(Integer(3))
0.0455907718407551 + 0.0289192946582081*I
>>> pari('2').besselk(Integer(3))
0.0615104584717420
```

14.10 What is Sage?

Sage is a framework for number theory, algebra, and geometry computation that is initially being designed for computing with elliptic curves and modular forms. The long-term goal is to make it much more generally useful for algebra, geometry, and number theory. It is open source and freely available under the terms of the GPL. The section titles in the reference manual gives a rough idea of the topics covered in Sage.

14.10.1 History of Sage

Sage was started by William Stein while at Harvard University in the Fall of 2004, with version 0.1 released in January of 2005. That version included Pari, but not GAP or Singular. Version 0.2 was released in March, version 0.3 in April, version 0.4 in July. During this time, support for Cremona's database, multivariate polynomials and large finite fields was added. Also, more documentation was written. Version 0.5 beta was released in August, version 0.6 beta in September, and version 0.7 later that month. During this time, more support for vector spaces, rings, modular symbols, and windows users was added. As of 0.8, released in October 2005, Sage contained the full distribution of GAP, though some of the GAP databases have to be added separately, and Singular. Adding Singular was not easy, due to the difficulty of compiling Singular from source. Version 0.9 was released in November. This version went through 34 releases! As of version 0.9.34 (definitely by version 0.10.0), Maxima and clisp were included with Sage. Version 0.10.0 was released January 12, 2006. The release of Sage 1.0 was made early February, 2006. As of February 2008, the latest release is 2.10.2.

Many people have contributed significant code and other expertise, such as assistance in compiling on various OS's. Generally code authors are acknowledged in the AUTHOR section of the Python docstring of their file and the credits section of the Sage website.

CHAPTER

FIFTEEN

CONTRIBUTIONS TO THIS DOCUMENT

Besides William Stein, contributions to this part of the documentation were made by Gary Zablackis.

[CF] {CF} A. Campillo and J. I. Farran, *Symbolic Hamburger-Noether expressions of plane curves and applications to AG codes*, Math. Comp., vol 71(2002)1759-1780. http://www.ams.org/mcom/2002-71-240/S0025-5718-01-01390-4/home.html

[CO] {CO} H. Cohen, J. Oesterlé, *Dimensions des espaces de formes modulaires*, p. 69-78 in Modular functions in one variable VI. Lecture Notes in Math. 627, Springer-Verlag, NewYork, 1977.

[GAP] {GAP4} The GAP Group, *GAP - Groups, Algorithms, and Programming, Version 4.4*; 2005, https://www.gap-system.org

[G] {G} Solomon Golomb, *Shift register sequences*, Aegean Park Press, Laguna Hills, Ca, 1967

[Sing] {GPS05} G.-M. Greuel, G. Pfister, and H. Schönemann. Singular 3.0. A Computer Algebra System for Polynomial Computations. Centre for Computer Algebra, University of Kaiserslautern (2005). https://www.singular.uni-kl.de

[Pari] {PARI2} PARI/GP, version 2.1.5, Bordeaux, 2004, https://pari.math.u-bordeaux.fr/

[M] {M} James L. Massey, *Shift-Register Synthesis and BCH Decoding*, IEEE Trans. on Information Theory, vol. 15(1), pp. 122-127, Jan 1969.

[SAGE] {SJ} William Stein, David Joyner, SAGE: System for Algebra and Geometry Experimentation, Comm. Computer Algebra 39(2005)61-64. (SIGSAM Bull. June 2005) https://sagemath.org/ https://github.com/sagemath/sage/issues/

CHAPTER

SIXTEEN

INDICES AND TABLES

- genindex
- search

INDEX

A

algebraic-geometric
    codes, 122
assumptions
    calculus, 3
attach into Sage, 131

B

background, running Sage in, 129
base ring
    matrix, 47
Brauer
    character, 73

C

calculus
    assumptions, 3
    critical points, 6
    differentiation, 5
    integration, 8
center
    groups, 29
character
    Brauer, 73
    modular representation, 73
characteristic polynomial
    matrix, 57
check matrix
    codes, 62
ciphers
    connection polynomial, 65
class_number
    number field, 106
codes
    algebraic-geometric, 122
    check matrix, 62
    dual, 62
    generator matrix, 62
    Golay, 62
conjugacy classes
    group, 27
connection polynomial
    ciphers, 65
cosets of Gamma_0, 97
critical points
    calculus, 6
cyclotomic
    number field, 108

D

database
    number field, 105
differentiation
    calculus, 5
discrete logs, 93
discriminant
    number field, 105
dual
    codes, 62

E

eigenvalues, 53
eigenvectors, 53
elliptic curve
    modular form, 103
    point counting, 111
elliptic curves, 100, 102
evaluation
    polynomial, 88

F

factorization
    polynomial, 84
Frobenius normal form, 53

G

gcd
    polynomial, 85
generator matrix
    codes, 62
Golay
    codes, 62
group
    conjugacy classes, 27

    normal subgroups, 28
    permutation, 25
    Rubik's cube, 25
groups
    center, 29

H

Hecke operators, 98
help in Sage, 130
Hermite normal form, 53
history
    Sage, 134

I

importing into Sage, 131
integral basis
    number field, 109
integration
    calculus, 8

L

LaTeX output, 129
linear equations
    solve, 59
load into Sage, 131

M

matrix
    base ring, 47
    characteristic polynomial, 57
    ring, 77
modular form
    elliptic curve, 103
modular forms, 96
modular representation
    character, 73
modular symbols, 98

N

normal subgroups
    group, 28
number field
    class_number, 106
    cyclotomic, 108
    database, 105
    discriminant, 105
    integral basis, 109

P

p-adics, 79
permutation
    group, 25
plot
    a function, 22
    a parametric curve, 22
    curve using surf, 20
point counting
    elliptic curve, 111
polynomial
    evaluation, 88
    factorization, 84
    gcd, 85
    powers, 83
    quotient ring, 80
    ring, 77
    roots, 87
    symbolic manipulation, 88
power series, 7
powers
    polynomial, 83
Python and Sage, 132

Q

quadratic residues, 95
quotient ring
    polynomial, 80

R

rational canonical form, 53
referencing
    Sage, 129
Riemann-Roch space, 120, 123
ring
    matrix, 77
    polynomial, 77
roots
    polynomial, 87
Rubik's cube
    group, 25

S

Sage
    history, 134
    referencing, 129
Smith normal form, 53
solve
    linear equations, 59
special functions in Sage, 133
symbolic manipulation
    polynomial, 88

T

Taylor series, 7